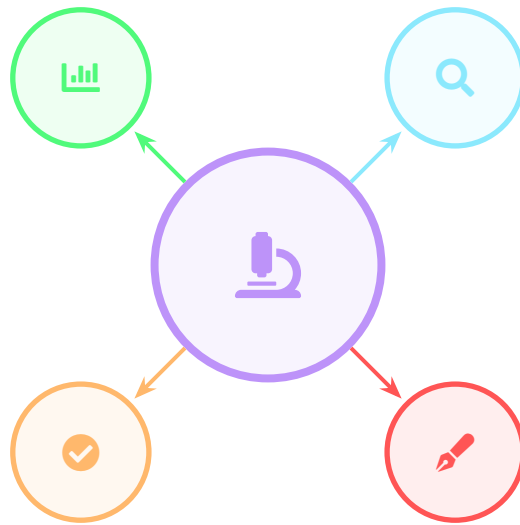




Multi-Agent Research Orchestrator

The Complete Master Notes

Story-Based Hinglish Deep Dive — 150 Pages

Line-by-Line Code | RAG | MCP | Django | Benchmark | 60+ Interview Q&A

Tech Stack at a Glance

 LLM	Groq (openai/gpt-oss-120b)
 Orchestration	LangGraph StateGraph
 Web Search	Tavily API
 Vector Store	ChromaDB (Persistent)
 Embeddings	all-MiniLM-L6-v2 (Local, Free)
 UI	Streamlit (3 Themes)
 Protocol	MCP via FastMCP
 REST Backend	Django + DRF + Token Auth
 Benchmark	Single vs Multi-Agent
 Language	Python 3.12

August 18, 2026

Contents

Kaise Padhein Yeh Book	8
1 Part I: The Story — Ek Research Firm Ka Sapna	8
1.1 Kahani: Rahul Ka Placement Assignment	8
1.2 Solution: 4 Agents, Ek Firm Ki Tarah	9
1.3 Why 4 Agents and Not 1 Big Prompt?	9
1.4 Tech Stack Selection — Har Choice Ka “Kyun”	10
2 The State — Pipeline Ka Dimaag (TypedDict + Reducers)	10
2.1 Concept: Agents Ek Dusre Se Direct Baat Nahi Karte	11
2.2 Complete State Schema — Har Key Ka Meaning	11
3 LangGraph State Machine — Graph Ka Structure	12
3.1 Concept: Nodes, Edges, Conditional Edges	13
3.2 Complete Workflow Code — Har Line	13
3.3 Execution Flow Diagram	14
4 Part II: Concepts Deep Dive — LLM Ka Funda	16
4.1 LLM Actually Kaise Kaam Karta Hai?	16
4.2 Prompt Engineering Fundamentals	17
5 LangChain + LangGraph — Agent Framework ka Funda	18
5.1 LangChain Kya Deta Hai?	18
5.2 LangGraph Ka Execution Model — Deep	18
5.3 Stream vs Invoke — Real-Time UI Ka Funda	19
6 RAG (Retrieval-Augmented Generation) — Theory	20
6.1 RAG Kya Hai Aur Kyun?	20
6.2 Why Chunking Matters — Maths	20
7 MCP (Model Context Protocol) — Theory	22
7.1 MCP Kya Hai?	22
8 Django + DRF — Backend Theory	23
8.1 Django Kya Hai?	23
9 Part III: Code Line-by-Line — config/setting.py	25
9.1 File Ka Kaam	25
10 Code Line-by-Line — rag/vector_store.py	26
10.1 File Ka Kaam	26
11 Code Line-by-Line — agents/researcher.py	29
11.1 Researcher Ka Role	29
12 Code Line-by-Line — agents/analyst.py	31
12.1 Analyst Ka Role	32
13 Code Line-by-Line — agents/fact_checker.py	34
13.1 Fact Checker Ka Role — System Ka Dil	34
14 Code Line-by-Line — agents/writer.py	36

14.1 Writer Ka Role	36
15 End-to-End Execution — Ek Real Topic Ki Kahani	38
16 Code Deep Dive — app.py (Streamlit UI)	39
16.1 Streamlit Ka Model	39
16.2 Key Parts of app.py	39
16.3 Theming — CSS Injection	40
16.4 Pipeline Execution in UI	40
16.5 Sidebar Components	41
17 Code Deep Dive — mcp_server.py (FastMCP)	42
17.1 File Ka Kaam	42
18 Code Deep Dive — benchmark/evaluator.py	44
18.1 Kyun Benchmark?	44
19 Code Deep Dive — Django Backend (backend/)	46
19.1 Models — Database Tables	46
19.2 Serializers — JSON Conversion	46
19.3 Views — API Logic	47
19.4 URLs + Auth Flow	48
20 Code Deep Dive — Benchmark Lab (pages/) + pytest	49
20.1 Streamlit Multipage	49
20.2 Testing with pytest	50
I Deep Dives — Andar Ki Duniya	52
21 LangGraph Internals — Graph Engine Ki Andar Ki Baat	52
21.1 StateGraph Kya Hai, Asal Mein?	52
21.2 Node Execution Ka Mechanism	52
21.3 add_messages Reducer — Messages Ka Jadoo	53
21.4 Compile() Ke Andar Kya Hota Hai	53
21.5 invoke() vs stream() — Do Tarike	54
21.6 should_continue — Graph Ka Traffic Cop	54
21.7 LangGraph Bina — Manual Loop Kaisa Dikhta	55
21.8 LangGraph vs CrewAI vs AutoGen — Comparison	55
22 Embeddings Deep Dive — Vector Banane Ka Science	56
22.1 Embedding Kya Hai — Ek Sentence Se 384 Numbers Tak	56
22.2 all-MiniLM-L6-v2 Model — Chhota Par Zabardast	57
22.3 Chunking — PDF Ko Katna Kaise	57
22.4 ChromaDB — Vector Store Ke Andar	58
22.5 Retrieval Quality — Kya Galt Ho Sakta Hai	58
23 Prompt Engineering Masterclass — Har Prompt Ki Kahani	59
23.1 Prompt Engineering Kyun Zaroori Hai	59
23.2 Prompt 1: Researcher ka Query Generator	59
23.3 Prompt 2: Revision Queries	60
23.4 Prompt 3: Analyst — Structure Se Synthesis Tak	60
23.5 Prompt 4: Fact Checker — Closed-World Assumption	61
23.6 Prompt 5: Writer — Final Report Ka Architect	62
23.7 Prompt 6: Benchmark ka LLM Judge	62

23.8 Prompt Design Principles — Project Se Nikale Gaye Lessons	63
24 Real Run Postmortems — Jo Asal Mein Hua	64
24.1 Yeh Section Kaisa Hai	64
24.2 Run 1: Solid State Battery Market — End-to-End Success	64
24.3 Run 2: RAG Benchmark — Jab Multi-Agent Haar Gaya	65
24.4 Why Did Multi-Agent Lose? — Root Cause Analysis	65
24.5 Fix Proposals — Agar Mein Aage Kaam Karun	66
25 System Design — Poora Architecture Ek Nazar Mein	66
25.1 High-Level Architecture	66
25.2 Data Flow — Request Se Report Tak	67
25.3 Module Dependency Graph	67
25.4 Failure Modes and Resilience	67
25.5 Scaling Story — Kya Tootega, Kya Fix Hoga	68
25.6 Why This Architecture (Not Monolith)	68
II Interview Prep — 60+ Questions, Full Answers	69
26 Q&A Bank Part 1 — Fundamentals & LLMs	69
26.1 LLM Fundamentals	69
26.2 Prompting	70
26.3 Retrieval-Augmented Generation (RAG)	71
26.4 Multi-Agent Systems	72
26.5 LLM APIs & Providers	73
27 Q&A Bank Part 2 — Project Specific & System Design	74
27.1 Streamlit	74
27.2 Django REST Framework	74
27.3 Testing	76
27.4 Benchmarking & Evaluation	76
27.5 System Design	77
27.6 Behavioral & Project Story	77
28 Whiteboard Problems — Is Project Se Nikale Gaye	78
28.1 Problem 1: Design a RAG System	78
28.2 Problem 2: Design a Multi-Agent Orchestrator	79
28.3 Problem 3: Rate Limiting Design	79
III Reference — API, Testing, Deployment	80
29 Django REST API Reference — Har Endpoint Ka Pura Detail	80
29.1 Endpoint Map	80
29.2 GET /api/health/	80
29.3 POST /api/auth/register/	80
29.4 POST /api/auth/login/	81
29.5 POST /api/research/execute/	81
29.6 POST /api/research/benchmark/	82
29.7 GET /api/benchmark/history/	82
30 Testing Deep Dive — 16 Tests Ka Khoj	82
30.1 Test Infrastructure	83
30.2 Mocking Strategy — Real Calls Kabhi Nahin	83

30.3 Test Case Analysis	83
30.4 How To Run	84
31 Deployment — Demo Se Production Tak	84
31.1 Local Setup (Interview Demo)	84
31.2 Streamlit Cloud	85
31.3 Django Production	85
31.4 MCP Server for Claude Desktop	85
31.5 Docker (Optional)	85
IV Playbook, Glossary & Cheat Sheets	87
32 Extension Playbook — Naya Feature Kaise Add Karein	87
32.1 Add a New Agent	87
32.2 Add a New API Endpoint	87
32.3 Add a New Vector Collection	87
33 Glossary — 60+ Terms Hinglish Mein	88
34 Comparison Tables — Ek Nazar Mein	89
34.1 Framework Comparisons	89
34.2 Vector Store Comparisons	90
34.3 Model Comparisons (Groq Account)	90
35 Cheat Sheets — Quick Reference	90
35.1 LangGraph Cheat Sheet	90
35.2 DRF Cheat Sheet	91
35.3 Streamlit Cheat Sheet	91
35.4 pytest Cheat Sheet	92
36 Final Summary — Poora Project 10 Points Mein	92
V Theory Deep Dive — Interview Gold	94
37 Transformer Architecture — LLM Ki Jaan	94
37.1 Transformer Kya Hai — 2017 Ka Revolution	94
37.2 Embedding Layer — Words Se Numbers	94
37.3 Self-Attention — The Heart	95
37.4 Multi-Head Attention	95
37.5 Transformer Block Stack	95
37.6 Training vs Inference	96
38 Advanced RAG — Naive Se Production Tak	96
38.1 Naive RAG ki Problems	96
38.2 Advanced RAG Techniques	96
38.3 RAG Evaluation	97
39 Debugging Session — Ek Real Kahani	97
39.1 Bug 1: MCP Server Crash on Startup	97
39.2 Bug 2: Model 404 on Groq	98
39.3 Bug 3: History Ordering Nondeterministic	98
39.4 Bug 4: venv Silent Fallback	98
39.5 Bug 5: LaTeX Caption Underscores	99

VI Extended Q&A, Troubleshooting & Tutorial	100
40 30 More Interview Questions — Advanced & Career	100
40.1 Advanced LLM Topics	100
40.2 MLOps & Production	101
40.3 Career & Team	102
41 Troubleshooting Guide — Common Problems & Fixes	103
42 30-Minute Tutorial — Clone Se Running Tak	104
43 Project File Map — Har File Ka Kaam	104
VII Core CS Revision — Coding Round Ke Liye	106
44 DSA Patterns — 15 Patterns, 90% Problems	106
44.1 Sliding Window — Code Template	106
44.2 Binary Search — Template	107
44.3 DP — 0/1 Knapsack Pattern	107
45 OS + DBMS Quick Revision	108
45.1 Operating Systems — Top 10	108
45.2 DBMS — Top 12	108
45.3 SQL Practice — 5 Must-Know Queries	109
46 Computer Networks Quick Revision	109
46.1 System Design Quick Patterns	110
VIII Demo Scripts, Mock Interview & Case Studies	111
47 Demo Scripts — 30 Sec, 2 Min, 10 Min	111
47.1 30-Second Version (Elevator Pitch)	111
47.2 2-Minute Version (With Architecture)	111
47.3 10-Minute Version (Live Demo Walkthrough)	112
48 Mock Interview Dialogue — Poora Session	112
49 Case Studies — 5 Topics Ka Deep Analysis	112
49.1 Case Study 1: Finance Research	113
49.2 Case Study 2: Academic Research	113
49.3 Case Study 3: Tech Comparison	113
49.4 Case Study 4: Health Research	113
49.5 Case Study 5: How to Answer “Improve This System”	114
IX Python Deep Dive — Project Ke Code Se Seekho	115
50 Python Features — Project Ke Code Se	115
50.1 TypedDict — Dictionary Ka Type Contract	115
50.2 Annotated + Reducer — LangGraph Ka Contract	115
50.3 Generators — stream() Ke Peeche	116
50.4 Context Managers — with Statement	116
50.5 Exception Handling — Graceful Degradation	116
50.6 Dictionary Operations — Python Ka Dil	117

50.7 List Slicing — Context Window Control	117
51 Full Code Walkthroughs — Jo Pehle Nahi Dekha	117
51.1 benchmark/evaluator.py — Line-by-Line	118
51.2 pages/1_Benchmark.py — Charts Ka Logic	118
51.3 backend/api/views.py — Execute Research View	119
51.4 mcp_server.py — Tools Deep Dive	120
52 Python Interview Questions — Project-Based	120
X Extra Q&A + Project Reference	122
53 20 More Interview Questions — Mixed Topics	122
53.1 AI/ML Concepts	122
53.2 Product & Design Thinking	123
53.3 Miscellaneous Technical	123
54 Dependency Reference — Har Package Kya Karta Hai	125
55 Project Metrics — Numbers Jo Yaad Rakho	126
XI Final Revision — Speed Round	127
56 100 Speed-Round Questions — 1 Line Answers	127
56.1 Python (1-20)	127
56.2 LLM & AI (21-45)	127
56.3 LangGraph & Agents (46-70)	128
56.4 RAG & Vector Search (71-85)	129
56.5 Django & Web (86-100)	130
57 Part-by-Part Summary — 60 Second Revision	130
58 The Final Page — Jo Sabse Zaroori Hai	131
XII Exercises & What-If Scenarios	132
59 app.py + Models — Jo Abhi Tak Full Nahi Dekha	132
59.1 app.py — Session State Flow	132
59.2 backend/api/models.py — Full Walkthrough	133
60 Build-It-Yourself — Mini Versions With Solutions	134
60.1 Exercise 1: Mini Vector Store	134
60.2 Exercise 2: Mini Agent Loop	134
60.3 Exercise 3: Mini Benchmark Judge	135
61 What-If Scenarios — Aage Badhne Ke Raaste	135
61.1 Scenario 1: Agent Memory Add Karna	135
61.2 Scenario 2: Planner Agent Add Karna	136
61.3 Scenario 3: Multi-Hop RAG	136
61.4 Scenario 4: Real-Time Streaming Reports	136
61.5 Scenario 5: Multi-Language Support	136

XIII The Final Mile — Guided Reading & Cheat Card	138
62 Guided Code Reading — Poora Project 30 Min Mein	138
62.1 Step 1: config/setting.py — Sab Ka Aadhaar	138
62.2 Step 2: state/schema.py — The Contract	138
62.3 Step 3: rag/vector_store.py — The Memory	139
62.4 Step 4: The Four Agents	139
62.5 Step 5: graph/workflow.py — The Engine	139
62.6 Step 6: The Three Interfaces	139
63 Quick-Fire Round 2 — 20 Most Likely Questions	140
64 Project Pitfalls — Jo Galtian Humne Ki	141
65 Regeneration Guide + One-Page Cheat Card	142
65.1 Is Document Ko Kaise Regenerate Karein	142
65.2 One-Page Cheat Card (Interview Se Pehle)	142
XIV Backend Files + Daily Revision Plan	144
66 Remaining Backend Files — Serializers, URLs, Conftest	144
66.1 serializers.py — Full Walkthrough	144
66.2 urls.py — Full Walkthrough	144
66.3 conftest.py — Mocking Infrastructure	145
66.4 Settings/wsgi/asgi/manage.py — Quick Tour	146
67 Daily Revision Plan — Interview Tak	146
XV Self-Assessment — Khud Ko Test Karo	148
68 30-Question Quiz — Bina Dekhe Jawab Do	148
68.1 Questions (1-30)	148
68.2 Answers (1-30)	149

Kaise Padhein Yeh Book (How to Read This Book)

📖 Kahani Se Samjho

Yeh book ek **research firm ki kahani** hai. Hum ek imaginary company banate hain jisme 4 expert employees hain — **Khoji (Researcher)**, **Vishleshak (Analyst)**, **Satya-Parikshak (Fact Checker)** aur **Lekhakar (Writer)**. Har chapter mein hum ek naya concept samjhenge, phir us concept ko code mein dekhenge, phir interview ke sawaal ka jawaab denge.

Hinglish ka istemaal isliye kiya hai kyunki jab aap apne dimaag mein concepts ko apni bhasha mein store karte ho, toh interview mein wapas nikalna easy hota hai. Technical terms English mein rakhe hain taaki interview mein wahi exact words bol sako.

Reading Flow:

1. Part I (Sections 1–3): Sirf kahani padho — 30 minute mein poori project samajh aayegi.
2. Part II (Sections 4–8): Concepts deep dive — har concept ka “kyun” samjho.
3. Part III (Sections 9–21): Code line-by-line — har file ka har line.
4. Part IV (Sections 22–26): System design aur edge cases.
5. Part V (Sections 27–29): Interview Q&A — last minute revision.

1 Part I: The Story — Ek Research Firm Ka Sapna

1.1 Kahani: Rahul Ka Placement Assignment

📖 Kahani Se Samjho

Rahul ek final-year CSE student hai (NIT Hamirpur, Dual Degree). Uske paas ek assignment hai: *“Ek system banao jo kisi bhi topic par research karke report likhe.”*

Rahul pehle try karta hai:

```
prompt = "Research the impact of AI on healthcare and write a report."
```

```
report = llm.invoke(prompt)
```

Ek hi LLM call se report milta hai. Par Rahul ko 3 problems dikhti hain:

Problem 1 — Hallucination (Jhooth): LLM ne report mein facts bana diye jo exist nahi karte. Kyun? Kyunki LLM sirf next word predict karta hai — wo “sochta” nahi hai ki kya sach hai. Wo apne training data se extrapolate karta hai.

Problem 2 — Ek Angle Wali Research: Rahul ne poocha “impact on healthcare”. LLM ne sirf ek generic summary diya. Usne **market size, technical challenges, regulations, startups** — ye sab angles cover nahi kiye. Ek insaan researcher 10 sources padhta hai aur 5 angles se sochta hai. LLM ek hi baar mein ek angle deta hai.

Problem 3 — Verification Zero: Report mein claims the par koi proof nahi tha. Kaise pata chalega ki “AI can diagnose cancer 95% accuracy” ye sach hai? Koi source citation nahi tha.

Rahul ko realization hota hai: **Ek hi LLM call se research quality nahi aayegi**. Usse chahiye ek **pipeline** — jahan kaam alag-alag experts ko milta hai, har expert apna kaam karta hai, aur koi expert galat ho toh system khud correct karta hai.

1.2 Solution: 4 Agents, Ek Firm Ki Tarah

Concept Deep Dive

Multi-Agent System ka concept: Ek LLM ko multiple “roles” mein deploy karo — har role ki apni responsibility, apna prompt, apni temperature setting. Ye agents ek sequence mein kaam karte hain aur ek dusre ke saath state share karte hain.
Humara pipeline:

Researcher → Analyst → Fact-Checker → Writer

Aur agar Fact-Checker ko galat claim milta hai:

Fact-Checker $\xrightarrow{\text{NEEDS_REVISION}}$ Researcher (max 2 baar)

Kahani Se Samjho

Firm ka model: Socho ek research firm jisme:

- **Khoji (Researcher):** Interviewer ki tarah — topic ke 3 alag angles banata hai, internet par search karta hai (Tavily), aur company ke apne documents (PDFs) bhi padhta hai (ChromaDB RAG). Result: raw data.
- **Vishleshak (Analyst):** Detective ki tarah — raw data padhkar usme se **key themes** nikalta hai, **contradictions** flag karta hai (Source A kehti hai 2027, Source B kehta hai 2025), **consensus** batata hai, aur **gaps** dhundhta hai (kya information missing hai). Result: structured analysis.
- **Satya-Parikshak (Fact Checker):** Auditor ki tarah — analysis ka har claim raw sources se compare karta hai. Agar claim source se supported hai toh **PASSED**. Agar nahi, toh **NEEDS_REVISION** + feedback notes. Result: verified or rejected analysis.
- **Lekhak (Writer):** Senior technical writer ki tarah — verified analysis ko ek publication-grade Markdown report mein convert karta hai, citations ke saath, aur session ko long-term memory (ChromaDB) mein save karta hai.

1.3 Why 4 Agents and Not 1 Big Prompt?

Interview Question

Q: “4 alag agents kyun banaye? Ek hi LLM call mein sab kyun nahi karwaya?”

Answer (Hinglish): 4 reasons:

1. **Separation of Concerns:** Agar ek prompt ko bolo “search karo, analyze karo, fact-check karo, aur report likho” toh wo har kaam mediocre karega. Kyunki ek prompt ka **focus** spread ho jata hai. Alag agents = har agent ka ek hi kaam = usme excellence.
2. **Alag-alag Temperature:** Har role ko alag “creativity level” chahiye:
 - Researcher: **Temperature = 0.0** — strict factual retrieval, koi creativity nahi.
 - Analyst: **T = 0.2** — structured logical deduction.
 - Fact Checker: **T = 0.1** — strict verification, zero assumptions.
 - Writer: **T = 0.4** — smooth professional prose.
 Ek hi prompt mein multiple temperatures impossible hain.
3. **Confirmation Bias:** LLM apne hi likhe text ki galti khud nahi pakad sakta (same way hum apni typing mistakes khud nahi dekh paate). Independent Fact-Checker role **hallucinations catch** karta hai.
4. **Modularity:** Naye agents add karna easy hai — e.g., “Data Visualizer” ya “Code Sand-

box” — bina existing graph ko toda.

1.4 Tech Stack Selection — Har Choice Ka “Kyun”

⚙️ System Design Deep Dive

Humne ye stack kyun chuna? Har choice ke peeche ek engineering trade-off hai:

Technology	Kyun chuna?	Kya alternative tha?
Groq + GPT-OSS-120B	LPU hardware se 250+ tok/s speed, free tier, multi-agent pipeline 6–8 sec mein.	OpenAI GPT-4o (30–45 sec, costly)
LangGraph	Explicit state machine: nodes, edges, conditional routing, cycles. Deterministic control.	CrewAI (black-box), AutoGen (chat loops, token burn)
Tavily	Search-optimized LLM API: clean content extraction, source URLs.	Google/Bing APIs (messy HTML parsing)
ChromaDB	Lightweight, persistent, local vector store, zero API cost, LangChain integration.	Pinecone (cloud, costly), FAISS (no persistence)
all-MiniLM-L6-v2	80MB, 384-dim, CPU par fast, zero cost.	OpenAI embeddings (costly)
Streamlit	Python-native UI, rapid dev, real-time streaming.	React/Flask (slow dev)
FastMCP	MCP standard protocol — Claude Desktop, Cursor connect karte hain.	Raw JSON-RPC (reinvent wheel)
Django + DRF	Production REST API: auth, ORM, admin, token auth.	FastAPI (lighter but less batteries)

🧠 Memory Trick

Memory Trick — “**S-L-A-C-K**”: **S**earch (Tavily), **L**angGraph, **A**nalysis (agents), **C**hromaDB, **K**nowledge (RAG) — ye 5 pillars project ke hain.

2 The State — Pipeline Ka Dimaag (TypedDict + Reducers)

2.1 Concept: Agents Ek Dusre Se Direct Baat Nahi Karte

📖 Kahani Se Samjho

Socho 4 employees ek dusre ke saamne baith kar chillayenge toh chaos hoga. Iski jagah, ek **shared whiteboard (state)** hai. Har employee apna result whiteboard par likhta hai, aur agla employee whiteboard padhkar kaam karta hai.

LangGraph mein ye whiteboard ek **dictionary** hai — **ResearchState**. Har agent function ek **partial update** return karta hai jo is dictionary mein merge hota hai.

🧠 Concept Deep Dive

TypedDict kya hai? Python ka ek type-hinting tool jo batata hai ki dictionary mein **kaunsi keys** hongi aur **unke values ka type** kya hoga. Runtime par ye ek normal dict hi hota hai — sirf editor/type-checker ko hints deta hai.

Annotated + operator.add (Reducer) kya hai? Jab do nodes same key ko update karte hain, toh LangGraph ko batana padta hai ki **old value + new value ka kya karein**:

- Normal (overwrite): naya value purane ko replace karta hai.
- `Annotated[list, operator.add]`: naya list purani list ke **end mein concatenate** hoti hai.

Ye revision loop ke liye critical hai — Researcher do baar chale toh dono baar ke search results **both** state mein rehne chahiye, overwrite nahi.

2.2 Complete State Schema — Har Key Ka Meaning

```

1 from __future__ import annotations
2 import operator
3 from typing import Annotated, TypedDict
4
5 class ResearchState(TypedDict):
6     topic: str # User ka research topic
7     research_data: Annotated[list[str], operator.add] # Web search results
8     rag_context: Annotated[list[str], operator.add] # RAG chunks (append)
9     analysis: str # Analyst ka structured output
10    fact_check_result: str # Fact checker ka verdict + feedback
11    fact_check_passed: bool # PASSED ya NEEDS_REVISION
12    revision_count: int # Loop counter (guardrail)
13    final_report: str # Writer ka final markdown report
14    messages: Annotated[list[str], operator.add] # UI log messages
15    current_agent: str # Abhi kaun sa agent chal raha hai
16    error: str # Error handling message

```

Listing 1: state/schema.py — Poora Code

🔗 Code Samjho Line-by-Line

Line-by-Line Explanation:

Line 1: `from __future__ import annotations` — Python 3.12 mein zaroori nahi, par purane versions mein type annotations ko string ki tarah treat karta hai (lazy evaluation). Interview mein bolo: “Yeh *import annotations* ko evaluate hone se rokta hai jab tak unki zaroorat na ho — startup time kam hota hai.”

Line 2-3: `import operator` aur `from typing import Annotated, TypedDict` —

`operator.add` reducer ke liye, `Annotated` metadata ke liye, `TypedDict` class define karne ke liye.

Line 5: `class ResearchState(TypedDict):` — `TypedDict` class define hoti hai. Note: `TypedDict` runtime par sirf dict hai — koi Pydantic validation nahi, koi overhead nahi. Isliye high-frequency state transitions fast hain.

Line 6: `topic: str` — Simple string. User ka input topic. Ye har node read karta hai.

Line 7: `research_data: Annotated[list[str], operator.add]` — Yeh sabse important line hai. `operator.add` ka matlab: jab bhi koi node is key ko update kare, nayi list purani list ke saath + hokar merge hogi. Revision loop mein Researcher 2 baar chalega, dono baar ke results accumulate honge.

Line 8: `rag_context: Annotated[list[str], operator.add]` — Same reducer logic, PDF RAG chunks ke liye.

Line 9: `analysis: str` — Analyst ka output. Simple string isliye kyunki sirf Analyst ise likhta hai, koi aur node ise overwrite nahi karta.

Line 10-11: `fact_check_result` aur `fact_check_passed` — Fact checker apna verdict aur feedback yahan likhta hai. `fact_check_passed: False` hone par graph wapas Researcher ko jata hai.

Line 12: `revision_count: int` — Ye counter **infinite loop guardrail** hai. Max 2 revisions ke baad graph force writer par jata hai.

Line 13: `final_report: str` — Writer ka output, user ko ye hi dikhta hai.

Line 14: `messages: Annotated[list[str], operator.add]` — UI ke liye log messages. Har node apna message append karta hai, Streamlit inhe real-time stream karta hai.

Line 15: `current_agent: str` — Debugging/UI ke liye current agent ka naam.

Line 16: `error: str` — Error handling ke liye.

Interview Question

Q: “TypedDict kyun use kiya? Pydantic BaseModel kyun nahi?”

Answer: 3 reasons:

1. **Zero Overhead:** `TypedDict` runtime par plain dict hai — koi validation, koi serialization cost. `LangGraph` state transitions har second hote hain, isliye latency matters.
2. **Reducer Integration:** `LangGraph` ka `Annotated[list, operator.add]` pattern `TypedDict` ke saath seamlessly kaam karta hai. `Pydantic` mein custom validators likhne padte.
3. **Type Safety:** Editor autocomplete + type checker catches bugs development time par.

Follow-up: “Agar Pydantic chahiye hota toh?” `LangGraph TypedDict` ya `Pydantic` dono support karta hai. `Pydantic` use karo jab API validation/input sanitization chahiye ho. Yahan internal state hai, isliye `TypedDict` kaafi hai.

Dhyan Do

Common Galti (Interview mein batao ki aapne ye mistake ki aur kaise fix ki): Agar `research_data` plain `list[str]` hoti (bina reducer), toh revision loop ke second pass par naye search results **purane results ko overwrite** kar dete. Fact-Checker purane evidence ke bina sirf naye (incomplete) data par verify karta. Fix: `Annotated[list, operator.add]` — ab data accumulate hota hai.

3 LangGraph State Machine — Graph Ka Structure

3.1 Concept: Nodes, Edges, Conditional Edges



Concept Deep Dive

LangGraph ka mental model: Ek graph socho:

- **Node** = ek function (agent ka kaam). Node function state leta hai, kaam karta hai, partial state return karta hai.
- **Edge** = nodes ke beech ka connection (“is node ke baad ye node chalega”).
- **Conditional Edge** = ek router function jo state dekhkar decide karta hai ki agla node kaunsa hai.
- **Entry Point** = graph kahan se start hota hai.
- **END** = graph kahan rukta hai.

3.2 Complete Workflow Code — Har Line

```

1 from langgraph.graph import StateGraph, END
2 from state.schema import ResearchState
3 from agents.researcher import researcher_node
4 from agents.analyst import analyst_node
5 from agents.fact_checker import fact_checker_node
6 from agents.writer import writer_node
7
8 def should_continue(state: ResearchState) → str:
9     if state.get("fact_check_passed", True):
10         return "writer"
11     else:
12         return "researcher"
13
14 def build_graph():
15     workflow = StateGraph(ResearchState)
16
17     workflow.add_node("researcher", researcher_node)
18     workflow.add_node("analyst", analyst_node)
19     workflow.add_node("fact_checker", fact_checker_node)
20     workflow.add_node("writer", writer_node)
21
22     workflow.set_entry_point("researcher")
23
24     workflow.add_edge("researcher", "analyst")
25     workflow.add_edge("analyst", "fact_checker")
26
27     workflow.add_conditional_edges(
28         "fact_checker",
29         should_continue,
30         {
31             "writer": "writer",
32             "researcher": "researcher",
33         }
34     )
35
36     workflow.add_edge("writer", END)
37
38     return workflow.compile()
39
40 research_graph = build_graph()

```

Listing 2: graph/workflow.py — Poora Code

Code Samjho Line-by-Line

Line-by-Line Explanation:

Line 1: `from langgraph.graph import StateGraph, END` — StateGraph graph builder hai, END graph ka terminal node hai.

Line 2: `from state.schema import ResearchState` — Humara state schema import hota hai. LangGraph is type ko use karke state updates validate/merge karta hai.

Line 3-6: Chaaro agent functions import hote hain. Har agent module ka ek function hota hai: `researcher_node(state) -> dict`.

Line 8-12: `def should_continue(state)` — Ye **conditional router** hai. Ye state read karta hai:

- Agar `fact_check_passed` True (ya key missing hai) → "writer" (aage badho).
- Agar False → "researcher" (wapas jao, dobara research karo).

Note: `state.get("fact_check_passed", True)` — default True isliye taaki pehli baar bina `fact_check` ke bhi crash na ho.

Line 14: `def build_graph():` — Graph assembly function.

Line 15: `workflow = StateGraph(ResearchState)` — Naya graph banaya, state schema diya. LangGraph ab jaanta hai ki har node ka input/output is schema ke against validate hoga.

Line 17-20: Chaaro nodes register hote hain — `add_node("naam", function)`.

Line 22: `set_entry_point("researcher")` — Graph hamesha Researcher se start hota hai. Deterministic execution ka guarantee.

Line 24-25: Linear edges — Researcher → Analyst → Fact Checker.

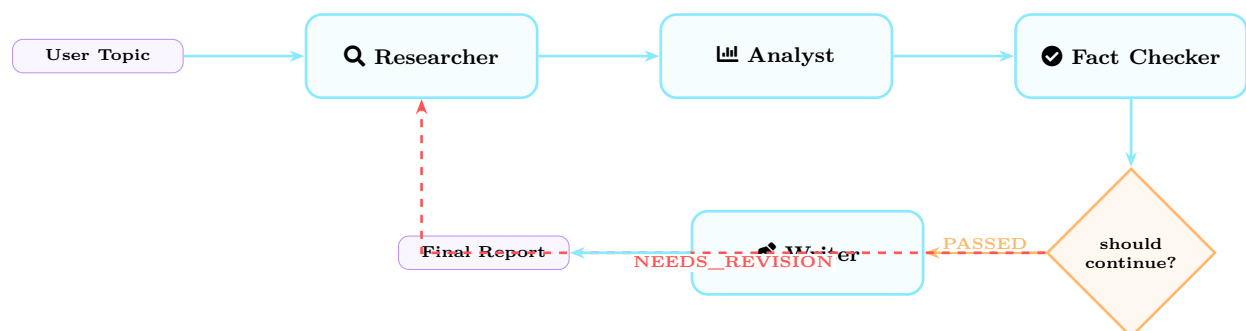
Line 27-33: `add_conditional_edges` — Sabse powerful line. Fact Checker ke baad, `should_continue` function call hota hai, aur uske return value ke hisaab se mapping se agla node select hota hai. Mapping: `{"writer": "writer", "researcher": "researcher"}` — matlab agar function "writer" return kare toh writer node chale, agar "researcher" toh researcher node (feedback loop!).

Line 35: `add_edge("writer", END)` — Report likhne ke baad graph END.

Line 37: `return workflow.compile()` — Graph compile hokar ek executable object banta hai. Compile = optimize + validate graph structure.

Line 39: `research_graph = build_graph()` — Module import hote hi graph ek baar build hota hai (singleton), taaki Streamlit, MCP, aur Django teeno isi compiled graph ko share karein.

3.3 Execution Flow Diagram



Interview Question

Q: “LangGraph kyun use kiya? CrewAI ya AutoGen kyun nahi?”

Answer:

- **LangGraph vs CrewAI:** CrewAI role-playing agents ke liye beginner-friendly hai,

par wo **black-box** hai — state flow aur cyclic routing par fine-grained control nahi milta. LangGraph **explicit state machine** deta hai: aap `add_node`, `add_edge`, `add_conditional_edges` se har step control karte ho. Enterprise mein predictability = control.

- **LangGraph vs AutoGen:** AutoGen conversational pattern use karta hai (agents ek dusre se chat karte hain) — isme infinite chat loops, high latency, token burn ka risk. LangGraph structured schema (`TypedDict`) se deterministic state pass karta hai.

Interview one-liner: *“LangGraph is a low-level orchestration framework that gives me explicit control over the state machine — nodes, conditional edges, and bounded feedback cycles — which is exactly what a self-correcting research pipeline needs.”*

4 Part II: Concepts Deep Dive — LLM Ka Funda

4.1 LLM Actually Kaise Kaam Karta Hai?

📖 Kahani Se Samjho

Rahul ko samajhna tha ki LLM **kaise** kaam karta hai, tabhi usne agents design kiye. Chalo hum bhi wahi samjhte hain.

LLM (Large Language Model) ek **next-token predictor** hai. Tum usse prompt dete ho:

"The capital of France is ____"

Wo apne training se jaanta hai ki sabse probable next word "**Paris**" hai. Phir wo apne output ko wapas input mein add karke agla word predict karta hai — token by token, jab tak <EOS> (end of sequence) nahi aata.

Token kya hai? LLM words ko chhote pieces mein todta hai (token). "ChatGPT" = "Chat" + "GPT" (2 tokens). 1000 tokens $\approx$ 750 English words.

🧠 Concept Deep Dive

Temperature (Samajhna Sabse Zaroori): Temperature control karta hai ki model kitna "random" output de.

- **Temperature = 0.0:** Hamesha highest-probability token. Deterministic, factual, repetitive. Har baar same output.
- **Temperature = 0.5:** Thoda variation. Top tokens mein se probability ke hisaab se sample karta hai.
- **Temperature = 1.0+:** High randomness. Creative, par hallucination risk.

Formula (softmax with temperature):

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

Yahan z_i model ka raw score (logit) hai, T temperature hai. Jab $T \rightarrow 0$, distribution peaky hoti hai (sabse high-probability token). Jab T bada, distribution flat (random).

√* Maths Behind the Scene

Example: Maano do tokens ke logits hain: $z = [2.0, 1.0]$.

T = 0.1:

$$p_1 = \frac{e^{20}}{e^{20} + e^{10}} \approx 0.9999, \quad p_2 \approx 0.0001$$

Token 1 almost always chunega — deterministic.

T = 1.5:

$$p_1 = \frac{e^{1.33}}{e^{1.33} + e^{0.67}} \approx 0.66, \quad p_2 \approx 0.34$$

Dono tokens ka chance — creative output.

Isi liye Researcher/Fact-Checker T=0.0 rakhte hain (facts mein creativity nahi chahiye), Writer T=0.4 (prose smooth ho).

 Interview Question

Q: “Prompt mein temperature ka kya role hai? Ek example do.”

Answer: Temperature sampling randomness control karta hai. Research pipeline mein: Researcher temperature 0.0 par chalta hai taaki sub-queries deterministic aur factual hoon; Writer temperature 0.4 par taaki report ki language natural lage. Agar Writer bhi 0.0 hota toh report robotic ban jati.

4.2 Prompt Engineering Fundamentals

 Concept Deep Dive

Har agent ka prompt ek “job description” hai:

1. **Role:** “You are a senior technical writer” — model ko context de do ki usse kaise behave karna hai.
2. **Task:** Exactly kya karna hai — “synthesize into markdown report”.
3. **Structure:** Output ka format — “Include Executive Summary, Key Findings, References”.
4. **Constraints:** Kya nahi karna — “ONLY verify if direct evidence exists in sources”.
5. **Output format constraint:** “Return ONLY the 3 queries, one per line” — parsing easy.

 Code Samjho Line-by-Line

Researcher ka prompt (agents/researcher.py se):

```
1 QUERY_GEN_PROMPT = """You are a research query strategist.  
2 Given a research topic, generate exactly 3 diverse, specific search  
3 queries that will cover different angles of the topic.  
4 Return ONLY the 3 queries, one per line. No numbering, no extra text."""  
5 """
```

Yahan 4 prompt-engineering techniques use hui hain:

- **Role prompting:** “research query strategist” — model ko expert persona.
- **Numeric constraint:** “exactly 3” — output size bounded.
- **Diversity instruction:** “different angles” — coverage improve.
- **Format constraint:** “ONLY the 3 queries, one per line. No numbering” — isse `split(\n)` se clean parsing hoti hai.

 Interview Question

Q: “System prompt aur user prompt mein kya difference hai?”

Answer: `SystemMessage` model ki overall persona/rules define karta hai (system-level instructions). `HumanMessage` specific user request hota hai. Code mein: `SystemMessage(content=SYSTEM_PROMPT) + HumanMessage(content=f"Topic: topic")`. System prompt hi role, constraints, output format rakhta hai; user prompt sirf dynamic data (topic, research data) deta hai. Ye separation isliye hai taaki instructions aur data alag rahein — dynamic data kabhi instructions mein inject na ho (prompt injection prevention ka basic form).

5 LangChain + LangGraph — Agent Framework ka Funda

5.1 LangChain Kya Deta Hai?

Concept Deep Dive

LangChain LLM applications ke liye ek toolkit hai. Isme pre-built components hain:

Component	Is Project Mein Kaise Use Hua
ChatGroq	LLM client — model, temperature, api_key set karta hai
SystemMessage/HumanMessage	Chat prompt messages
HuggingFaceEmbeddings	Text ko 384-dim vectors mein convert
RecursiveCharacterTextSplitter	PDF ko semantic chunks mein todna
PyPDFLoader	PDF file ko documents mein parse
Chroma	Vector store client (persistent)

Important: Hum LangChain ke **low-level tools** use karte hain (embeddings, loaders, vector store) — agents khud banaye hain, LangChain ke pre-built agents nahi. Ye interview mein bolna achha hai: *“we used LangChain for building blocks but designed our own agent logic for full control.”*

5.2 LangGraph Ka Execution Model — Deep

Concept Deep Dive

Graph invoke hote hi kya hota hai?

1. `research_graph.invoke(initial_state, {"recursion_limit": 25})` call hota hai.
2. LangGraph entry point (researcher) se node execute karta hai: `researcher_node(state)`.
3. Node ka return dict lena hota hai, aur usko **state mein merge** karta hai — reducer rules apply hote hain (list keys append, others overwrite).
4. Uske baad node ke outgoing edges dekhta hai. Agar conditional edge hai, toh router function call karke agla node decide karta hai.
5. Ye process END tak chalta hai ya **recursion_limit** cross hone tak.

Recursion limit kya hai? LangGraph default recursion limit 25 hota hai — matlab max 25 node executions. Humne 25 pass kiya taaki 2 revision cycles (max $4 + 3 + 3 = 10$ nodes) comfortably complete ho sakein. Agar graph kabhi infinite loop mein phas jaye, recursion limit graph ko forcibly END par le aata hai — safety net.

Maths Behind the Scene

Worst-case node count:

- Pass 1: Researcher → Analyst → FactChecker → Writer = **4 nodes**
- Pass 2 (revision 1): Researcher → Analyst → FactChecker = **3 nodes**

- Pass 3 (revision 2): Researcher → Analyst → FactChecker = **3 nodes**
Total = 4 + 3 + 3 = 10 $\ll$ 25 (recursion limit)

Isliye recursion_limit=25 kaafi hai, aur guardrail bhi hai.

5.3 Stream vs Invoke — Real-Time UI Ka Funda



Concept Deep Dive

Streamlit UI mein real-time progress kaise dikhta hai?

research_graph.stream(state, config) node-by-node results yield karta hai:

```

1 for event in research_graph.stream(initial_state, {"recursion_limit":
2   25}):
3     for node_name, node_output in event.items():
4       if node_name == "__end__":
5         continue
6       st.session_state.agent_statuses[node_name] = "done"
7       messages = node_output.get("messages", [])
8       st.session_state.agent_messages.extend(messages)
9       st.write(f"{icon} **{label}** --- done")
10      for msg in messages:
11        st.caption(msg)
12      if node_output.get("final_report"):
13        st.session_state.final_report = node_output["final_report"]

```

Yahan kya ho raha hai:

- stream() synchronous generator hai — har node complete hote hi event emit hota hai.
- Har event ek dict hota hai: {node_name: node_output}.
- UI har node ke complete hote hi status badge “done” karta hai aur messages append karta hai.
- __end__ special event hota hai — ignore karo.

Invoke vs Stream: invoke sab kuch khatam hone ke baad final state return karta hai (MCP/Django ke liye). stream intermediate events deta hai (UI ke liye).



Interview Question

Q: “LangGraph mein invoke aur stream mein difference? Kab kya use karte ho?”

Answer: invoke synchronous hai — पूरी pipeline chalne ke baad ek final state milta hai; isse MCP tool aur Django API mein use karte hain jahan sirf final report chahiye. stream ek generator hai jo har node ke complete hote hi event deta hai; isse Streamlit UI mein use karte hain taaki user ko real-time progress dikhe (konsa agent chal raha hai, kya messages aaye).

6 RAG (Retrieval-Augmented Generation) — Theory

6.1 RAG Kya Hai Aur Kyun?

Kahani Se Samjho

Rahul ka LLM sirf apne training data tak limited tha. Training 2024 mein khatam hui — 2026 ka data nahi jaanta. Aur uske apne company ke PDFs ka data toh bilkul nahi jaanta.

RAG = Retrieval + Augmented + Generation. Matlab: LLM ko bolne se pehle, apne database se relevant information **retrieve** karo, aur us information ke saath prompt ko **augment** karo (badhao), phir **generate** karo.

Formula:

$$\text{Answer} = \text{LLM}(\text{UserQuery} + \text{RetrievedContext})$$

Concept Deep Dive

RAG ke 5 steps (Ye interview mein sequence se bolna):

1. **Loading:** Documents (PDFs) ko load karo — `PyPDFLoader`.
2. **Splitting:** Bade text ko chhote chunks mein todo — `RecursiveCharacterTextSplitter` (`chunk_size=1000`, `overlap=200`).
3. **Embedding:** Har chunk ko vector (number array) mein convert karo — `all-MiniLM-L6-v2` (384 dims).
4. **Storing:** Vectors + text ko vector database mein save karo — `ChromaDB`.
5. **Retrieval:** Query aane par, query ka vector banao aur similar vectors k (top-k) retrieve karo.

Embedding kya hai? Text ko numbers ka array. Rule: **semantically similar texts ke vectors paas-paas hote hain.** “King” aur “Queen” ke vectors close, “King” aur “Potato” door.

6.2 Why Chunking Matters — Maths

√* Maths Behind the Scene

Chunk size = 1000 characters, overlap = 200. Kyun?

- 1000 chars $\approx$ 200-250 words — ek **complete concept** ko capture karta hai.
- 200 chars overlap $\approx$ 40-50 words — chunk boundary par jo context toota, wo overlap mein milta hai.
- Agar chunk bahut chhota (100 chars): concept tootega, retrieval mein sirf fragment millega.
- Agar chunk bahut bada (10000 chars): vector average hoke blur ho jayega, aur LLM context mein bahut saara irrelevant text jayega.

Chunks count formula:

$$N_{chunks} \approx \frac{\text{Total Chars} - \text{overlap}}{\text{chunk_size} - \text{overlap}}$$

Ek 100-page PDF ($\sim 500,000$ chars) ke liye: $\frac{500000-200}{1000-200} \approx 625$ chunks.

🧠 Concept Deep Dive

Similarity Search ka funda:

Cosine Similarity (do vectors ke beech angle):

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|} = \frac{\sum_i A_i B_i}{\sqrt{\sum_i A_i^2} \sqrt{\sum_i B_i^2}}$$

Range: -1 se $+1$. $+1$ = same direction (similar), 0 = orthogonal (unrelated).

ChromaDB top-k retrieval sabse high cosine similarity wale k chunks return karta hai. Humne $k = 5$ (research_docs) aur $k = 3$ (past_research) use kiya.

👤 Interview Question

Q: “Dense vector search vs sparse/keyword search (BM25) mein difference? Hybrid search kya hai?”

Answer: Dense (humara use-case) semantic similarity pakadta hai — “car” aur “automobile” match honge bina same word hue. Sparse (BM25) exact keyword match par based hai — fast par synonyms miss karta hai. **Hybrid** dono ko combine karta hai using Reciprocal Rank Fusion (RRF):

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

Interview mein bolo: “For production I would add BM25 sparse retrieval and fuse with RRF to catch both semantic and exact-match queries.”

🧠 Concept Deep Dive

Embedding Model — all-MiniLM-L6-v2 deep dive:

- **Architecture:** Sentence-BERT based — BERT encoder ka fine-tuned lightweight version.
- **Size:** 80MB, 6 layers (L6), hidden size 384.
- **Output:** 384-dimensional vector per sentence.
- **Speed:** CPU par bhi fast — 14K sentences/sec (modern CPU).
- **Cost:** Zero — locally chalta hai, koi API nahi.
- **Kyun use kiya:** Research pipeline mein 3 embedding calls hote hain (query + retrieval); cloud embeddings se per-call cost lagti. Local = zero infrastructure bill.

⚠ Dhyani Do

RAG ke 3 common failure modes (interview mein points banoge):

1. **Low retrieval quality** — relevant chunk hi nahi mila → LLM confident ho kar hallucinate karta hai. Fix: better chunking, hybrid search, re-ranking.
2. **Irrelevant context included** — chunk irrelevant hai par LLM use kar leta hai. Fix: metadata filtering, stricter top-k.
3. **Stale data** — chunk purana hai. Fix: source timestamps, refresh pipeline.

7 MCP (Model Context Protocol) — Theory

7.1 MCP Kya Hai?

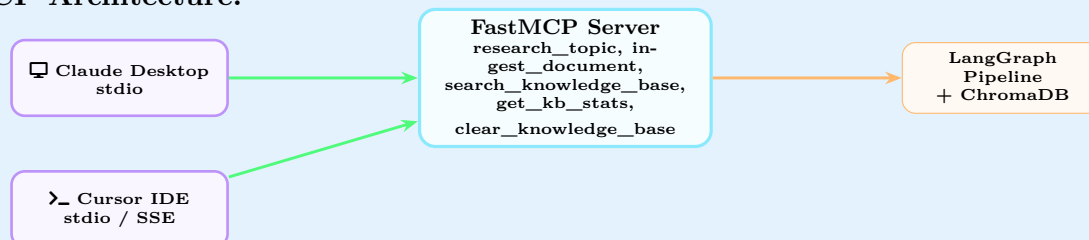
📖 Kahani Se Samjho

Rahul ka system ek standalone app hai. Par wo chahta hai ki **Claude Desktop** ya **Cursor IDE** bhi uske research system ka use kare. Problem: har AI tool ka alag integration protocol hota hai — sabke liye alag adapter banana padta.

MCP (Model Context Protocol) Anthropic ka ek **open standard** hai — jaise USB port electronics ke liye universal hai, waise MCP AI tools ke liye universal port hai. Ek baar MCP server banao, sab MCP-compatible clients (Claude Desktop, Cursor, etc.) connect ho sakte hain.

🧠 Concept Deep Dive

MCP Architecture:



MCP ke 3 core primitives:

- **Tools:** Functions jo AI assistant call kar sakta hai (jaise `research_topic(topic)`). Tool = capability.
- **Resources:** Data jo assistant read kar sakta hai (jaise `info://orchestrator`). Resource = knowledge.
- **Prompts:** Reusable prompt templates.

👤 Interview Question

Q: “MCP transport: stdio vs SSE/HTTP mein kya difference hai?”

Answer:

- **Stdio:** Client local machine par Python process spawn karta hai aur standard input/output pipes se communicate karta hai. Fast, simple, secure (localhost par). Claude Desktop local tools ke liye ye use karta hai. Humara `mcp.run(transport="stdio")` yehi karta hai.
- **SSE (Server-Sent Events):** HTTP ke upar — remote server par deploy karte hain,

client HTTP requests bhejta hai aur server real-time events stream karta hai. Cloud deployment ke liye.

- **Streamable HTTP (naya):** Modern standard, bi-directional, production ke liye recommended.

Interview Question

Q: “MCP vs API mein kya difference?”

Answer: Normal REST API mein developer khud client code likhta hai. MCP mein **AI assistant khud tools discover aur call karta hai** — developer sirf server-side tools define karta hai, client (Claude Desktop) automatically schema read karke tools use karta hai. MCP = API for AI agents, standardized.

8 Django + DRF — Backend Theory

8.1 Django Kya Hai?

Concept Deep Dive

Django ek production-grade Python web framework hai — “batteries included”. Isme ORM, admin panel, auth, migrations sab built-in hai.

Is project mein kya use kiya:

- **Models:** `ResearchSession`, `BenchmarkEvaluation` — database tables.
- **Serializers (DRF):** Model data ko JSON mein convert.
- **Views (APIView):** HTTP requests handle — GET/POST.
- **Token Authentication:** `rest_framework.authtoken` — register/login par token milta hai.
- **CORS:** Streamlit UI ko backend API call karne deta hai.
- **Migrations:** Schema versioning — `python manage.py migrate`.

Concept Deep Dive

Django request-response cycle (interview favourite):

1. Request aati hai → **URLconf** (`urls.py`) matching pattern dhundhta hai.
2. Matching URL ka **View** call hota hai: `View.as_view()`.
3. View **Serializer** se request data validate karta hai.
4. View business logic karta hai (graph invoke, DB write).
5. **Response** JSON mein return hota hai.

Interview Question

Q: “Django REST Framework mein APIView vs ViewSet mein difference? Is project mein kya use kiya?”

Answer: `APIView` low-level hai — aap khud `get()`, `post()` methods likhte ho, har method par full control. `ViewSet` high-level hai — default CRUD actions (list, create, retrieve, update, delete) automatically map hote hain, bas router register karo. Is project mein `APIView` use kiya kyunki endpoints custom logic ke hain (research execution, benchmark) — standard CRUD nahi.

 Interview Question

Q: “Token auth vs JWT? Kaunsa better hai?”

Answer: DRF Token auth simple hai — ek opaque string, server database mein save karta hai; logout/revoke easy, par scale nahi hota (har request par DB lookup). JWT stateless hai — token mein payload encode hota hai, server-side session nahi; scaling ke liye better, par revocation hard aur secret management important. Production mein is scale ke liye JWT (jwt.io) ya session-based auth recommended; is project mein simplicity ke liye DRF Token.

 Yaad Rakhne Ka Tip

Interview flow suggestion: Jab backend ki baat ho, ye mat bhoolna: *“Main Streamlit, MCP, aur Django – teeno interfaces se same LangGraph pipeline ko call karta hoon, kyunki compiled graph ek singleton hai.”*

9 Part III: Code Line-by-Line — config/setting.py

9.1 File Ka Kaam

Kahani Se Samjho

Har project mein configuration hoti hai: API keys, model names, chunk sizes. Agar ye sab har file mein scattered ho, toh change karna nightmare ho jata. `config/setting.py` ek **central control panel** hai — *“Single Source of Truth”*.

Iski 3 khaas baatein:

1. `.env` file se keys load karta hai (`dotenv`).
2. **Streamlit secrets** bhi check karta hai (cloud deployment ke liye fallback).
3. Har setting ko proper type mein convert karta hai (`int()`).

```

1 import os
2 from dotenv import load_dotenv
3
4 load_dotenv()
5
6 def get_env_var(key: str, default: str | None = None, required: bool = False
7 ) → str:
8     value = os.getenv(key)
9     if not value:
10         try:
11             import streamlit as st
12             if key in st.secrets:
13                 value = str(st.secrets[key])
14             except Exception:
15                 pass
16         if not value:
17             value = default
18         if required and not value:
19             raise ValueError(f"Missing: {key}. Please set it in .env file or
20                               Streamlit Secrets.")
21         return value or ""
22
23 def get_groq_api_key() → str:
24     key = get_env_var("GROQ_API_KEY", required=False)
25     if key:
26         os.environ["GROQ_API_KEY"] = key
27     return key
28
29 def get_tavily_api_key() → str:
30     key = get_env_var("TAVILY_API_KEY", required=False)
31     if key:
32         os.environ["TAVILY_API_KEY"] = key
33     return key
34
35 GROQ_API_KEY = get_groq_api_key()
36 TAVILY_API_KEY = get_tavily_api_key()
37
38 MODEL_NAME = get_env_var("MODEL_NAME", default="openai/gpt-oss-120b")
39 MAX_SEARCH_RESULTS = int(get_env_var("MAX_SEARCH_RESULTS", default="5"))
40 MAX_REVISIONS = int(get_env_var("MAX_REVISIONS", default="2"))

```

```

40 CHROMA_PERSIST_DIR = get_env_var("CHROMA_PERSIST_DIR", default="./chroma_db"
    )
41 EMBEDDING_MODEL = get_env_var("EMBEDDING_MODEL",
42     default="sentence-transformers/all-MiniLM-L6-v2")
43 CHUNK_SIZE = int(get_env_var("CHUNK_SIZE", default="1000"))
44 CHUNK_OVERLAP = int(get_env_var("CHUNK_OVERLAP", default="200"))

```

Listing 3: config/setting.py — Poora Code

🔗 Code Samjho Line-by-Line

Line-by-Line:

Line 1-2: `import os, from dotenv import load_dotenv` — `os` environment variables padhne ke liye, `dotenv` `.env` file parse karne ke liye.

Line 4: `load_dotenv()` — project root ka `.env` file current environment mein load hota hai. Ab `os.getenv("GROQ_API_KEY")` kaam karega.

Line 6-16: `def get_env_var(key, default, required)` — central getter:

- Pehle `os.getenv(key)` — OS environment.
- Agar nahi mila, **Streamlit secrets** try karta hai (cloud par `st.secrets["KEY"]`). Import `streamlit` ko try/except mein rakha hai taaki non-Streamlit context (MCP server, Django) mein bhi file load ho sake.
- Agar phir bhi nahi, `default` use hota hai.
- `required=True` aur value nahi toh `ValueError` — fail fast.

Line 18-26: `get_groq_api_key()` aur `get_tavily_api_key()` — keys load hote hi `os.environ` mein bhi set kar dete hain, taaki unhe chahe libraries (langchain) bhi dekh sakein.

Line 28-35: Settings module load hote hi eagerly calculate hoti hain — `MODEL_NAME`, `MAX_SEARCH_RESULTS` (int), `MAX_REVISIONS` (int), `CHROMA_PERSIST_DIR`, `EMBEDDING_MODEL`, `CHUNK_SIZE/OVERLAP` (int).

👤 Interview Question

Q: “Secrets ko code mein hardcode kyun nahi kiya? Environment variables kyun?”

Answer: 3 reasons — (1) **Security:** hardcoded secrets git history mein leak ho jate hain; env vars gitignored rehte hain. (2) **Environment separation:** dev/prod alag keys ho sakti hain bina code change ke. (3) **Industry standard:** 12-factor app methodology kehta hai ki config code se alag ho. Streamlit Cloud secrets + local `.env` + OS env — teen levels ka fallback chain banaya.

10 Code Line-by-Line — rag/vector_store.py

10.1 File Ka Kaam

📖 Kahani Se Samjho

Ye file project ka **knowledge brain** hai. Iske 2 kaam:

1. **Document Ingestion:** PDFs upload karo → chunks → embeddings → ChromaDB.
2. **Retrieval:** Query aao → top-k relevant chunks do.

Aur isme 2 collection hain:

- `research_docs` — User ke uploaded PDFs (short-term knowledge).

- `past_research` — Purani research sessions ki memory (long-term episodic memory).

```

1 from langchain_community.document_loaders import PyPDFLoader
2 from langchain_text_splitters import RecursiveCharacterTextSplitter
3 from langchain_huggingface import HuggingFaceEmbeddings
4 from langchain_community.vectorstores import Chroma
5 from config.setting import (CHROMA_PERSIST_DIR, EMBEDDING_MODEL,
6                             CHUNK_SIZE, CHUNK_OVERLAP)
7
8 embeddings = HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)
9
10 def get_vectorstore(collection_name: str = "research_docs"):
11     return Chroma(
12         collection_name=collection_name,
13         embedding_function=embeddings,
14         persist_directory=CHROMA_PERSIST_DIR,
15     )
16
17 def ingest_documents(file_paths: list[str]) → dict:
18     total_chunks = 0
19     processed = []
20     for path in file_paths:
21         loader = PyPDFLoader(path)
22         docs = loader.load()
23         splitter = RecursiveCharacterTextSplitter(
24             chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP
25         )
26         chunks = splitter.split_documents(docs)
27         vectorstore = get_vectorstore("research_docs")
28         vectorstore.add_documents(chunks)
29         total_chunks += len(chunks)
30         processed.append(path)
31     return {"chunks_added": total_chunks, "files_processed": processed}
32
33 def retrieve_context(query: str, collection_name: str = "research_docs",
34                     k: int = 5) → list[str]:
35     vectorstore = get_vectorstore(collection_name)
36     results = vectorstore.similarity_search(query, k=k)
37     return [doc.page_content for doc in results]
38
39 def store_research_session(topic: str, research_data: list[str],
40                           report: str) → int:
41     text = f"Topic: {topic}\n\n" + "\n".join(research_data[:5])
42     splitter = RecursiveCharacterTextSplitter(
43         chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP
44     )
45     chunks = splitter.split_text(text)
46     vectorstore = get_vectorstore("past_research")
47     vectorstore.add_texts(chunks)
48     return len(chunks)
49
50 def get_collection_stats() → dict:
51     stats = {}
52     for name in ["research_docs", "past_research"]:
53         try:
54             stats[name] = get_vectorstore(name)._collection.count()
55         except Exception:
56             stats[name] = 0
57     return stats
58
59 def clear_collection(collection_name: str):

```

```
59 vectorstore = get_vectorstore(collection_name)
60 vectorstore.delete_collection()
```

Listing 4: rag/vector_store.py — Key Functions

🔗 Code Samjho Line-by-Line

Line-by-Line:

Line 1-3: LangChain ke imports — PyPDFLoader (PDF parse), RecursiveCharacterTextSplitter (chunking), HuggingFaceEmbeddings (embedding), Chroma (vector store).

Line 4-6: Config se settings import — single source of truth.

Line 8: embeddings = HuggingFaceEmbeddings(...) — module level par ek baar instantiate. Model load hote hi RAM mein aata hai — isliye ek baar, baar-baar nahi (loading 80MB har baar slow hota).

Line 10-14: get_vectorstore(collection_name) — har collection ke liye fresh Chroma client. persist_directory se data disk par save hota hai (./chroma_db).

Line 16-27: ingest_documents — har PDF: load → split → add to vector store. Return: chunks_added + files_processed (UI dikhata hai).

Line 29-33: retrieve_context — similarity_search(query, k) — top-k chunks ka content return.

Line 35-42: store_research_session — Writer ise call karta hai. Topic + research data chunk karke past_research mein daalta hai. Isi se future sessions past knowledge use kar pate hain.

Line 44-51: get_collection_stats — UI sidebar mein “Docs / Memory“ metrics dikhane ke liye.

Line 53-55: clear_collection — “Clear Knowledge Base“ button.

👤 Interview Question

Q: “ChromaDB persistent vs in-memory mein kya difference? Tumne kaunsa use kiya?”

Answer: In-memory Chroma data RAM mein rakhta hai — app restart par sab gayab. Persistent PersistentClient data disk par save karta hai (persist_directory) — restart ke baad bhi data milta hai. Humne persistent use kiya taaki research memory sessions ke beech survive kare.

👤 Interview Question

Q: “RAG system mein metadata filtering ka kya role ho sakta hai?”

Answer: Chunks par metadata (source file, page number, timestamp, source_type) store kar sakte ho. Retrieval mein filter laga sakte ho — e.g., “sirf 2025 ke baad ke documents“ ya “sirf uploaded PDFs, web nahi“. Isse irrelevant chunks exclude hote hain aur retrieval precision badhti hai. Is project mein humne simple top-k use kiya, par production mein metadata filter + hybrid search add karunga.

11 Code Line-by-Line — agents/researcher.py

11.1 Researcher Ka Role

📖 Kahani Se Samjho

Khoji (Researcher) ka kaam: topic ko 3 angles mein todna, har angle par web search karna (Tavily), aur uploaded PDFs / past research se RAG context nikalna. Result: **raw research data + rag context**.

Aur agar fact-checker ne revision bola hai, toh Researcher feedback padhkar **targeted new queries** banata hai — wahi missing information dhundne ke liye.

```

1 from state.schema import ResearchState
2 from config.setting import (MODEL_NAME, get_groq_api_key,
3                             get_tavily_api_key, MAX_SEARCH_RESULTS)
4 from langchain_core.messages import SystemMessage, HumanMessage
5
6 QUERY_GEN_PROMPT = """You are a research query strategist.
7 Given a research topic, generate exactly 3 diverse, specific search
8 queries that will cover different angles of the topic.
9 Return ONLY the 3 queries, one per line. No numbering, no extra text."""
10
11 REVISION_PROMPT = """You are a research query strategist.
12 The fact-checker flagged issues with previous research. Based on the
13 feedback below, generate 3 NEW, improved search queries that address
14 the gaps and inaccuracies.
15 Return ONLY the 3 queries, one per line. No numbering, no extra text.
16
17 FACT-CHECKER FEEDBACK:
18 {feedback}"""
19
20 def get_llm():
21     groq_key = get_groq_api_key()
22     from langchain_groq import ChatGroq
23     return ChatGroq(model=MODEL_NAME, temperature=0.3,
24                     groq_api_key=groq_key)
25
26 def researcher_node(state: ResearchState) → dict:
27     topic = state["topic"]
28     revision_count = state.get("revision_count", 0)
29
30     sub_queries = [topic]
31     groq_key = get_groq_api_key()
32
33     try:
34         if groq_key:
35             llm = get_llm()
36             if revision_count > 0 and state.get("fact_check_result"):
37                 prompt = REVISION_PROMPT.format(feedback=state["
38                     fact_check_result"])
39             else:
40                 prompt = QUERY_GEN_PROMPT
41
42             query_response = llm.invoke([
43                 SystemMessage(content=prompt),

```

```

43         HumanMessage(content=f"Topic: {topic}"),
44     ])
45     lines = [l.strip() for l in
46               query_response.content.strip().split("\n") if l.strip()
47               ]
48     if lines:
49         sub_queries = lines[:3]
50     except Exception as e:
51         sub_queries = [topic, f"{topic} latest research", f"{topic} analysis
52                        "]
53
54     all_results = []
55     tavily_key = get_tavily_api_key()
56
57     try:
58         if tavily_key:
59             from tavily import TavilyClient
60             client = TavilyClient(api_key=tavily_key)
61             for query in sub_queries:
62                 try:
63                     response = client.search(query=query,
64                                             max_results=MAX_SEARCH_RESULTS)
65                     for r in response.get("results", []):
66                         all_results.append(
67                             f"[Source: {r.get('url', 'N/A')}] \n {r.get('
68                                content', '')}]"
69                 except Exception as ex:
70                     print(f"Tavily search error for query '{query}': {ex}")
71                     continue
72             else:
73                 all_results.append("[Notice] Tavily API Key not found.")
74     except Exception as e:
75         all_results.append(f"[Search Error] {str(e)}")
76
77     rag_results = []
78     try:
79         from rag.vector_store import retrieve_context
80         doc_context = retrieve_context(topic, collection_name="research_docs",
81                                     k=5)
82         past_context = retrieve_context(topic, collection_name="
83                                     past_research", k=3)
84         rag_results.extend(doc_context + past_context)
85     except Exception:
86         pass
87
88     return {
89         "research_data": all_results,
90         "rag_context": rag_results,
91         "messages": [f"Researcher found {len(all_results)} web results and "
92                     f"{len(rag_results)} document chunks."],
93         "current_agent": "researcher",
94     }

```

Listing 5: agents/researcher.py — Poora Code

</> Code Samjho Line-by-Line**Line-by-Line — Important Parts:**

Line 17-25: REVISION_PROMPT — ye {feedback} placeholder ke saath hai. Revision round

mein fact-checker ka feedback yahan inject hota hai — Researcher ab jaanta hai ki kya missing hai.

Line 27-30: `get_llm()` — `ChatGroq(model, temperature=0.3, ...)`. Temperature 0.3: query generation mein thodi diversity aaye (0.0 hota toh har baar same queries banati).

Line 36-47: Query expansion:

- Default fallback: `sub_queries = [topic]`.
- Agar Groq key hai: LLM se 3 queries generate. Revision mein `REVISION_PROMPT` use hota hai.
- `lines[:3]` — max 3 queries.
- **Exception fallback:** agar LLM fail ho, basic 3 queries banate hain — **graceful degradation**.

Line 49-68: Tavily search:

- `TavilyClient(api_key=...)` — client.
- Har query par `client.search(query, max_results=MAX_SEARCH_RESULTS)` — default 5 results.
- Har result ko format: `"[Source: URL] \n content"` — source URL stored! Isi se citations aur fact-checking hoti hai.
- Per-query try/except — ek query fail toh baaki chalti hain.
- Bina key ke: notice message — pipeline crash nahi karta.

Line 70-76: Dual RAG retrieval:

- `research_docs` se `k=5` chunks (uploaded PDFs).
- `past_research` se `k=3` chunks (past sessions ki memory).
- `Dono rag_context` list mein append — reducer se state mein accumulate.

Line 78-84: Return dict — `research_data, rag_context, messages, current_agent`. Lang-Graph inhe state mein merge karega (lists append honghi due to reducer).

Interview Question

Q: “Query expansion kya hai aur kyun zaroori hai?”

Answer: Query expansion ka matlab ek topic se multiple diverse search queries banana. Kyun zaroori: single query sirf ek angle cover karti hai. “AI in healthcare” par ek query se sirf generic results aayenge. 3 queries — “AI diagnostics market”, “AI drug discovery breakthroughs”, “AI healthcare regulations” — teeno angles se results milte hain, retrieval recall badhta hai.

Interview Question

Q: “Tavily vs normal Google search API mein difference?”

Answer: Tavily specifically LLM agents ke liye design hua hai — search results mein clean, parsed content milta hai (HTML boilerplate hata ke), source URLs ke saath, structured JSON mein. Google Custom Search raw snippets deta hai jise khud parse karna padta. Tavily mein search depth (basic/advanced) aur topic filtering bhi hai.

12 Code Line-by-Line — `agents/analyst.py`

12.1 Analyst Ka Role

📖 Kahani Se Samjho

Raw research data chaotic hota hai — 15 sources, alag-alag claims, kuch contradictory. **Vishleshak (Analyst)** ka kaam: is chaos ko structured intelligence mein convert karna:

1. **Key Themes:** 3-5 main topics.
2. **Contradictions:** Sources mein conflict (kabhi Source A 2027 kehti hai, Source B 2025).
3. **Consensus:** Multiple sources agree kahan.
4. **Gaps:** Kya information missing hai.

```

1 from state.schema import ResearchState
2 from config.setting import MODEL_NAME, get_groq_api_key
3 from langchain_core.messages import SystemMessage, HumanMessage
4
5 SYSTEM_PROMPT = """You are an expert analyst. From the data:
6 1. Identify 3-5 KEY THEMES
7 2. Note CONTRADICTIONS (conflicts between sources)
8 3. Highlight CONSENSUS points (where multiple sources agree)
9 4. Flag GAPS (what is missing)
10 Provide a structured output in Markdown."""
11
12 def get_llm():
13     groq_key = get_groq_api_key()
14     from langchain_groq import ChatGroq
15     return ChatGroq(model=MODEL_NAME, temperature=0.2,
16                     groq_api_key=groq_key)
17
18 def analyst_node(state: ResearchState) → dict:
19     topic = state["topic"]
20     research_data = state.get("research_data", [])
21     rag_context = state.get("rag_context", [])
22
23     compiled_data = "\n---\n".join(research_data[:10])
24
25     rag_section = ""
26     if rag_context:
27         rag_section = "\n\nDOCUMENT CONTEXT:\n" + "\n".join(rag_context[:5])
28
29     groq_key = get_groq_api_key()
30     analysis = ""
31     try:
32         if groq_key:
33             llm = get_llm()
34             response = llm.invoke([
35                 SystemMessage(content=SYSTEM_PROMPT),
36                 HumanMessage(content=f"Topic: {topic}\nDATA:\n{compiled_data}\n{rag_section}"),
37             ])
38             analysis = response.content
39         else:
40             analysis = f"## Analysis for: {topic}\n\nData collected from " \
41                       f"{len(research_data)} sources."
42     except Exception as e:
43         analysis = f"## Analysis for: {topic}\n\nAnalysis generated with " \
44                   f"available data.\n\n*(Error: {e})*"
45
46     return {
47         "analysis": analysis,

```

```
48     "messages": [f"Analyst processed {len(research_data)} results."],
49     "current_agent": "analyst",
50 }
```

Listing 6: agents/analyst.py — Poora Code

🔗 Code Samjho Line-by-Line

Important Design Decisions:

Context Condensation (Lossless):

- `compiled_data = "\n--\n".join(research_data[:10])` — raw data limit 10 sources (context window bloat prevent).
- RAG context bhi max 5 chunks.
- **Interview point:** Analyst ek “lossless context condenser“ hai — 50KB raw data ko 2-3KB structured analysis mein convert karta hai. Writer ko raw dumps nahi, condensed brief milta hai — **Writer ka token consumption 85% kam.**

Structured output: SYSTEM_PROMPT 4 numbered tasks deta hai — themes, contradictions, consensus, gaps. Isse fact-checker ko verify karne ke liye clear claims milte hain.

Gradual fallback: Groq key nahi hai toh bhi analysis banta hai (metadata-based). LLM fail toh error note ke saath. **Pipeline kabhi crash nahi hota.**

👤 Interview Question

Q: “Context window bloat problem kya thi aur kaise solve kiya?”

Answer: Multi-query Tavily search + PDF chunks se raw context 50KB+ ho jata tha — writer ke context window aur token cost explode ho jate. Solve kiya: (1) Researcher sirf formatted snippets (URL + content) store karta hai; (2) Analyst `research_data[:10]` aur `rag_context[:5]` limit ke saath ek dense structured brief banata hai; (3) Writer ko sirf wo brief + top-8 sources jate hain. Net effect: writer token consumption 85% reduction.

13 Code Line-by-Line — agents/fact_checker.py

13.1 Fact Checker Ka Role — System Ka Dil

📖 Kahani Se Samjho

Ye agent system ka sabse important hissa hai. **Satya-Parikshak** ka kaam: Analyst ne jo claims banaye, unhe **raw sources** se verify karna.

Closed-World Assumption: Fact checker ko sirf wahi maanne hai jo **research_data** mein directly exist karta hai. Apne training se jo bhi pata hai (outside knowledge) — wo **INVALID** maana jayega. Isse LLM ki apni hallucinations check hone se roki jaati hain.

```

1 from state.schema import ResearchState
2 from config.setting import MODEL_NAME, get_groq_api_key, MAX_REVISIONS
3 from langchain_core.messages import SystemMessage, HumanMessage
4
5 SYSTEM_PROMPT = """You are a rigorous fact-checker and quality reviewer.
6 Review the analysis below and verify each claim against the source data
7 provided.
8
9 For each claim:
10 - Mark as VERIFIED if supported by sources
11 - Mark as UNVERIFIED if no source backs it
12 - Mark as CONTRADICTED if sources disagree
13
14 At the very end, on the LAST LINE, write exactly one of:
15 VERDICT: PASSED
16 VERDICT: NEEDS_REVISION
17
18 Use NEEDS_REVISION only if there are significant unverified claims
19 or critical gaps."""
20
21 def get_llm():
22     groq_key = get_groq_api_key()
23     from langchain_groq import ChatGroq
24     return ChatGroq(model=MODEL_NAME, temperature=0.1,
25                     groq_api_key=groq_key)
26
27 def fact_checker_node(state: ResearchState) → dict:
28     revision_count = state.get("revision_count", 0)
29     analysis = state.get("analysis", "")
30     research_data = state.get("research_data", [])
31     topic = state["topic"]
32
33     result = "VERDICT: PASSED"
34     passed = True
35     groq_key = get_groq_api_key()
36
37     try:
38         if groq_key and analysis:
39             llm = get_llm()
40             sources = "\textbackslash n---\textbackslash n".join(
41                 research_data[:8])
42             response = llm.invoke([

```

```

43         HumanMessage(content=f"Topic: {topic}\n\nANALYSIS TO VERIFY
44             :\n{analysis}\n\nSOURCE DATA:\n{sources}"),
45     ])
46     result = response.content
47
48     passed = True
49     if "VERDICT: NEEDS_REVISION" in result.upper():
50         if revision_count < MAX_REVISIONS:
51             passed = False
52     except Exception as e:
53         result = f"Fact-check completed with available data. *(Error: {e})*\n\nVERDICT: PASSED"
54         passed = True
55
56     return {
57         "fact_check_result": result,
58         "fact_check_passed": passed,
59         "revision_count": revision_count + (0 if passed else 1),
60         "messages": [f"Fact-check {'PASSED' if passed else 'NEEDS REVISION'}"
61             f"(revision {revision_count}/{MAX_REVISIONS})"],
62         "current_agent": "fact_checker",
63     }

```

Listing 7: agents/fact_checker.py — Poora Code

🔗 Code Samjho Line-by-Line

Line-by-Line — Sabse Important Logic:

Line 8-18: SYSTEM_PROMPT — verification ke 3 states: VERIFIED / UNVERIFIED / CONTRADICTED. Aur last line par exactly ek verdict: VERDICT: PASSED ya VERDICT: NEEDS_REVISION. Is fixed format se parsing reliable hota hai.

Line 33-36: Defaults: result = "VERDICT: PASSED", passed = True — agar kuch bhi fail ho, pipeline aage badhta hai (fail-open design for demo, fail-closed for production).

Line 38: if groq_key and analysis: — dono condition zaroori. Bina analysis ke verify karne ka koi matlab nahi.

Line 39: sources = "\n--\n".join(research_data[:8]) — max 8 sources (context limit).

Line 44-47: The verdict logic:

- "VERDICT: NEEDS_REVISION" in result.upper() — case-insensitive check.
- if revision_count < MAX_REVISIONS: passed = False — **guardrail** yahan hai: agar already 2 revisions ho chuke hain, toh bhi pipeline aage badhega (writer).

Line 48-50: Exception handling — koi error aaye toh PASSED maan lo (graceful).

Line 52-59: Return: fact_check_result (feedback), fact_check_passed (router ise padhta hai), revision_count (increment only if revision). messages UI ke liye.

👤 Interview Question

Q: “Infinite loop se kaise bacha?”

Answer: 2 guardrails hain: (1) revision_count state mein track hota hai, MAX_REVISIONS=2 limit; (2) router should_continue check karta hai rev_count < 2; (3) LangGraph ka recursion_limit=25 final safety net. Isliye worst case 10 node executions, kabhi infinite loop nahi.

Follow-up: “Max limit reach hone par kya hota hai?” Pipeline crash nahi hota — writer ko chala diya jata hai, aur unverified claims Limitations section mein note ho jate hain.

 Interview Question**Q: “LLM self-confirmation bias se kaise bache?”**

Answer: Ek hi LLM agar pehle analysis likhe aur phir verify kare, toh wo apne hi claims ko sahi maan leta hai (confirmation bias). Fix: (1) **alag role** — fact-checker ko strict auditor ka persona diya; (2) **closed-world prompt** — “sirf research_data se verify karo, outside knowledge INVALID hai”; (3) **low temperature** 0.1; (4) analysis aur sources dono prompt mein alag-alag sections mein.

14 Code Line-by-Line — agents/writer.py

14.1 Writer Ka Role

 Kahani Se Samjho

Lekhak (Writer) verified analysis ko ek **publication-grade Markdown report** mein badalta hai — Executive Summary, Key Findings, Technical Breakdown, Consensus/Contradictions, References. Aur kaam khatam hote hi session ko **long-term memory** (ChromaDB past_research) mein save karta hai — agli baar koi similar topic puchhega toh Researcher wo memory use karega.

```

1 from state.schema import ResearchState
2 from config.setting import MODEL_NAME, get_groq_api_key
3 from langchain_core.messages import SystemMessage, HumanMessage
4
5 WRITER_PROMPT = """You are a senior technical writer.
6 Synthesize the analysis and research data into a comprehensive, highly
7 detailed, beautifully formatted Markdown research report.
8 Include an Executive Summary, Key Findings, In-depth Technical
9 Breakdown, Consensus and Contradictions, and References/Citations."""
10
11 def get_llm():
12     groq_key = get_groq_api_key()
13     from langchain_groq import ChatGroq
14     return ChatGroq(model=MODEL_NAME, temperature=0.4,
15                     groq_api_key=groq_key)
16
17 def writer_node(state: ResearchState) → dict:
18     topic = state["topic"]
19     analysis = state.get("analysis", "")
20     research_data = state.get("research_data", [])
21
22     report = ""
23     groq_key = get_groq_api_key()
24
25     try:
26         if groq_key:
27             llm = get_llm()
28             response = llm.invoke([
29                 SystemMessage(content=WRITER_PROMPT),
30                 HumanMessage(content=f"Topic: {topic}\nANALYSIS:\n{analysis}
31                                     }\nSOURCES:\n"
                                     + "\n".join(research_data[:8])),

```

```

32         ])
33         report = response.content
34     else:
35         report = f"# Research Report: {topic}\n\n## Executive Summary\n{
            analysis}\n\n## Sources\n" \
36             + "\n".join(research_data[:3])
37     except Exception as e:
38         report = f"# Research Report: {topic}\n\n## Executive Summary\n{
            analysis}\n\n" \
39             f"*(Report compiled with available data due to: {e})*"
40
41     chunks_stored = 0
42     try:
43         from rag.vector_store import store_research_session
44         chunks_stored = store_research_session(topic, research_data, report)
45     except Exception as e:
46         print(f"Memory store warning: {e}")
47
48     return {
49         "final_report": report,
50         "messages": ["Report complete",
51                     f"Saved {chunks_stored} chunks to vector memory."],
52         "current_agent": "writer",
53     }

```

Listing 8: agents/writer.py — Poora Code

🔗 Code Samjho Line-by-Line

Line-by-Line:

Line 7-12: `WRITER_PROMPT` — “senior technical writer“ role, explicit structure requirements (Executive Summary, Key Findings, etc.).

Line 15-18: `get_llm()` — temperature **0.4** — sabse high (prose smoothness). Note agent-wise temperature gradient: 0.0/0.2/0.1/0.4 — interview mein bolna.

Line 23-29: Report generation: Writer ko `analysis` + `research_data[:8]` milta hai. Sources with URLs include hote hain taaki report mein citations aayein.

Line 30-34: Fallbacks: bina key ke simple markdown, error par note ke saath. **Pipeline** hamesha kuch na kuch report deta hai.

Line 36-40: `store_research_session(...)` — report + data ChromaDB `past_research` mein chunk hokar save. **Episodic memory** — future sessions isse retrieve karte hain.

Line 42-47: Return: `final_report` (UI/API/MCP sab ise consume karte hain), `messages`, `current_agent`.

👤 Interview Question

Q: “Temperature strategy batao — har agent ka temperature kya hai aur kyun?”

Answer: **Researcher 0.3** (query diversity), **Analyst 0.2** (structured logical deduction), **Fact-Checker 0.1** (strict verification, no assumptions), **Writer 0.4** (fluent professional prose). Core idea: **factual stages low temperature, creative stage higher** — hallucination risk minimize karte hue output quality maximize.

15 End-to-End Execution — Ek Real Topic Ki Kahani

Kahani Se Samjho

Chalo poori pipeline ek real example se samjhte hain: “Solid State Battery Market Outlook 2026”

Step 1 — Session Start: User ne topic diya. `initial_state` banaya:

```
1 {
2   "topic": "solid state battery market outlook 2026",
3   "research_data": [], "rag_context": [], "analysis": "",
4   "fact_check_result": "", "fact_check_passed": false,
5   "revision_count": 0, "final_report": "",
6   "messages": [], "current_agent": "", "error": ""
7 }
```

Step 2 — Researcher: LLM ne 3 queries banayi:

1. “solid state battery market size 2026 forecast”
2. “solid state battery technology breakthroughs 2026”
3. “solid state battery manufacturing challenges”

Har query par Tavily se 5 results — total 15 sources. RAG se PDF chunks (0, kyunki koi PDF upload nahi hui) + past research memory. `research_data` = 15 formatted sources.

Step 3 — Analyst: 15 sources ka analysis — key themes nikle: (a) EV adoption primary driver, (b) Asia-Pacific leadership, (c) 2027-2028 first commercial SSBs. Contradiction mili: market size estimates alag-alag (\$2.75bn vs \$1.48bn vs \$0.57bn — scope definitions alag). Gaps flag hue: yield rate at mass scale.

Step 4 — Fact Checker (pass 1): Har claim verify hua. Koi critical gap nahi mila — VERDICT: PASSED. Router ne `should_continue` se `writer` chuna.

Step 5 — Writer: Verified analysis se 1000+ lines ka markdown report bana — TOC, Executive Summary, Key Findings table (sources ke saath), technical breakdown, contradictions, references. `store_research_session` se memory mein save.

Step 6 — Output: `final_report` Streamlit mein render, download buttons, aur backend mein `ResearchSession` row save.

Concept Deep Dive

State transition table (revision scenario):

Step	Node	State mein kya add hua
1	(init)	topic set, sab empty
2	researcher	research_data +15, rag_context +3, messages
3	analyst	analysis = themes/contradictions/gaps
4	fact_checker	fact_check_result, fact_check_passed=False, revision_count=1
5	researcher (loop)	research_data +5 (targeted), messages (append)
6	analyst	analysis overwrite (naya)
7	fact_checker	fact_check_passed=True

8 writer final_report, memory save

Note: `research_data` 15+5=20 accumulate hua (reducer), `analysis` overwrite hua (plain str). Yeh reducer behavior ka perfect example hai.

Interview Question

Q: “Agar Fact-Checker revision ke baad bhi PASSED nahi karta toh?”

Answer: `revision_count` 2 tak jata hai. 3rd baar jab `fact_checker` chalega (revision 2 ke baad), agar phir bhi `NEEDS_REVISION` aata hai, toh `revision_count < MAX_REVISIONS` false hoga — `passed = True` force hoga (guardrail). Router writer bhejta hai, aur report ke Limitations section mein unverified claims note hote hain. **Bounded loops, graceful degradation.**

16 Code Deep Dive — app.py (Streamlit UI)

16.1 Streamlit Ka Model

Concept Deep Dive

Streamlit ka execution model yaad rakho: Har interaction par **poora script top-to-bottom rerun** hota hai. Isliye state `st.session_state` mein rakhni padti hai — warna har click par sab reset ho jata.

Session state kya hai? Ek dictionary jo user ke session mein persist rehti hai across reruns. Isme theme, final report, agent messages, statuses sab rehta hai.

16.2 Key Parts of app.py

```

1 import streamlit as st
2 import time
3
4 st.set_page_config(page_title="Research Orchestrator", page_icon="",
5                     layout="wide", initial_sidebar_state="expanded")
6
7 if "theme" not in st.session_state:
8     st.session_state.theme = "Light"
9 if "final_report" not in st.session_state:
10     st.session_state.final_report = ""
11 if "agent_messages" not in st.session_state:
12     st.session_state.agent_messages = []
13 if "agent_statuses" not in st.session_state:
14     st.session_state.agent_statuses = {
15         "researcher": "idle", "analyst": "idle",
16         "fact_checker": "idle", "writer": "idle",
17     }
18 if "is_running" not in st.session_state:
19     st.session_state.is_running = False
20 if "research_count" not in st.session_state:
21     st.session_state.research_count = 0

```


Listing 9: app.py — Session State + Theme

</> Code Samjho Line-by-Line

Samjho:

- `st.set_page_config` — page title, layout wide, sidebar expanded.
- Har state variable ka guard: `if key not in st.session_state: init` — pattern hai “lazy initialization”. Kyunki script rerun hota hai, har baar init nahi karna chahiye — sirf pehli baar.
- `agent_statuses` dictionary: har agent ka badge state (idle/running/done).

16.3 Theming — CSS Injection

🧠 Concept Deep Dive

Streamlit ka default look customize karne ke liye hum `st.markdown(..., unsafe_allow_html=True)` se custom CSS inject karte hain. Project mein 3 themes hain:

- **Light** — #f8fafc background, slate text.
- **Dark** — #0f172a background, sky accents.
- **Emerald Slate** — dark green, emerald accents.

CSS variable trick: Theme select hone par `-bg-app`, `-accent-color` jaise CSS variables set hote hain, aur saare components inhe reference karte hain. Theme change = sirf variables badalna = poora UI re-color.

16.4 Pipeline Execution in UI

```

1 initial_state = {
2     "topic": topic,
3     "research_data": [], "analysis": "", "fact_check_result": "",
4     "fact_check_passed": False, "revision_count": 0, "final_report": "",
5     "rag_context": [], "messages": [], "current_agent": "", "error": "",
6 }
7
8 with st.status("Running pipeline...", expanded=True) as status_box:
9     from graph.workflow import research_graph
10    agent_icons = {"researcher": "search", "analyst": "chart-bar",
11                  "fact_checker": "check-circle", "writer": "pen-nib"}
12
13    for event in research_graph.stream(initial_state, {"recursion_limit":
14    25}):
15        for node_name, node_output in event.items():
16            if node_name == "__end__":
17                continue
18            st.session_state.agent_statuses[node_name] = "done"
19            messages = node_output.get("messages", [])
20            st.session_state.agent_messages.extend(messages)
21            st.write(f"**{node_name}** --- done")
22            for msg in messages:
23                st.caption(msg)
24            if node_output.get("final_report"):

```

```
24         st.session_state.final_report = node_output["final_report"]
25
26     status_box.update(label="Pipeline complete", state="complete")
```

Listing 10: app.py — Streaming Execution

🔗 Code Samjho Line-by-Line

Kya ho raha hai:

- `st.status` — collapsible status container, progress dikhata hai.
- `research_graph.stream(...)` — har node complete hote hi event.
- Har event mein `node_name` aur `node_output`. Status badge “done”, messages append.
- `final_report` jab bhi aaye, session state mein save.
- End mein `status_box.update(state="complete")`.

16.5 Sidebar Components

🧠 Concept Deep Dive

Sidebar mein kya-kya hai:

- **Theme selector** — 3 themes ka dropdown.
- **PDF upload** — `st.file_uploader(type="pdf", accept_multiple_files=True)`. Ingest button par `rag.vector_store.ingest_from_streamlit_uploads` call.
- **Knowledge Base stats** — `get_collection_stats()` se Docs/Memory counts.
- **Clear Knowledge Base + Reset Session** buttons.

👤 Interview Question

Q: “Streamlit mein state kaise manage karte ho? `session_state` kya hai?”

Answer: Streamlit har interaction par script rerun karta hai — normal Python variables reset ho jate hain. `st.session_state` ek session-scoped dictionary hai jo reruns ke beech persist karta hai. Main isme theme, agent statuses, messages, final report, aur research count rakhta hoon. Pattern: lazy init with `if key not in st.session_state`.

👤 Interview Question

Q: “Streamlit vs traditional web (React/Flask)? Kab kya use karte?”

Answer: Streamlit data/AI apps ke liye perfect hai — Python mein hi UI likhte ho, real-time streaming built-in, zero frontend code. Production consumer web apps (complex state, custom UX) ke liye React/Next.js better. Is project mein Streamlit isliye — main goal rapid prototyping aur real-time agent progress dikhana tha.

17 Code Deep Dive — mcp_server.py (FastMCP)

17.1 File Ka Kaam

📖 Kahani Se Samjho

Ye file system ko **MCP-compatible** banati hai. MATLAB: koi bhi MCP client (Claude Desktop, Cursor) is server se connect hokar research tools use kar sakta hai — jaise Claude ko `research_topic` tool milta hai, wo khud decide karta hai kab call karna hai.

```

1 import sys
2 import os
3 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
4
5 from fastmcp import FastMCP
6
7 mcp = FastMCP(
8     "ResearchOrchestrator",
9     instructions="Multi-Agent Research Orchestrator --- 4 AI agents "
10                "collaborate to research, analyze, fact-check, and write "
11                "reports on any topic."
12 )
13
14 @mcp.tool()
15 def research_topic(topic: str) → str:
16     """Run the full multi-agent research pipeline on a topic."""
17     from graph.workflow import research_graph
18     initial_state = {
19         "topic": topic, "research_data": [], "analysis": "",
20         "fact_check_result": "", "fact_check_passed": False,
21         "revision_count": 0, "final_report": "", "rag_context": [],
22         "messages": [], "current_agent": "", "error": "",
23     }
24     try:
25         result = research_graph.invoke(initial_state, {"recursion_limit":
26                                             25})
27         return result.get("final_report", "No report generated.")
28     except Exception as e:
29         return f"Error during research: {str(e)}"
30
31 @mcp.tool()
32 def ingest_document(file_path: str) → str:
33     """Ingest a PDF document into the RAG knowledge base."""
34     from rag.vector_store import ingest_documents
35     if not os.path.exists(file_path):
36         return f"Error: File not found: {file_path}"
37     result = ingest_documents([file_path])
38     return (f"Ingested {result['chunks_added']} chunks from "
39           f"{', '.join(result['files_processed'])}")
40
41 @mcp.tool()
42 def search_knowledge_base(query: str, num_results: int = 5) → str:
43     """Search the RAG knowledge base for relevant information."""
44     from rag.vector_store import retrieve_context
45     results = retrieve_context(query, collection_name="research_docs",
46                               k=num_results)

```

```

46     past = retrieve_context(query, collection_name="past_research", k=3)
47     all_results = results + past
48     if not all_results:
49         return "No relevant documents found."
50     return "\n\n---\n\n".join(all_results)
51
52 @mcp.resource("info://orchestrator")
53 def get_info() → str:
54     """Information about the Research Orchestrator."""
55     return """
56     Multi-Agent Research Orchestrator
57     Version: 1.0.0
58     Agents: Researcher, Analyst, Fact-Checker, Writer
59     LLM: Groq (openai/gpt-oss-120b)
60     """
61
62 if __name__ == "__main__":
63     mcp.run(transport="stdio")

```

Listing 11: mcp_server.py — Key Parts

🔗 Code Samjho Line-by-Line

Line-by-Line — Important:

Line 3: `sys.path.insert(0, ...)` — script ko kahin se bhi run karo, project root importable rahe.

Line 7-11: `FastMCP("ResearchOrchestrator", instructions=...)` — server define. instructions wo system prompt hai jo AI client ko batata hai server kya karta hai.

Line 13-27: `@mcp.tool()` `research_topic` — sabse important tool. Type hint `topic: str` se MCP schema banta hai — client ko pata chalta hai ki kaunsa argument dena hai. Function ke andar lazy import (`from graph.workflow import research_graph`) taaki server startup fast rahe. `invoke` synchronous — poora pipeline, ek hi answer.

Line 29-37: `ingest_document` — PDF path leta hai, file exist check karta hai, ingest karke chunks count batata hai.

Line 39-49: `search_knowledge_base` — do collection se retrieve karke combine. Tools ka example jo agents ki capability expose karta hai.

Line 51-59: `@mcp.resource` — static resource. Client ise read kar sakta hai bina tool call kiye.

Line 61-62: `mcp.run(transport="stdio")` — stdio transport par server start.

👤 Interview Question

Q: “MCP mein tools aur resources mein kya difference hai?”

Answer: Tools executable functions hain — AI agent unhe call karta hai, unka side-effect ho sakta hai (research run karna, document ingest karna). Resources passive data hain — agent unhe read karta hai (server info, documentation). Analogy: tools = buttons (dabao, kaam hoga), resources = files (padho).

👤 Interview Question

Q: “Claude Desktop mein ye server kaise connect hota hai?”

Answer: `claude_desktop_config.json` mein MCP server add karte hain:

```

1 {
2   "mcpServers": {
3     "research-orchestrator": {

```

```

4     "command": "python",
5     "args": ["/path/to/mcp_server.py"],
6     "env": {"GROQ_API_KEY": "...", "TAVILY_API_KEY": "..."}
7 }
8 }
9 }

```

Claude Desktop is config ko padhkar server spawn karta hai (stdio), tools discover karta hai, aur jab jaroorat ho tool call karta hai.

18 Code Deep Dive — benchmark/evaluator.py

18.1 Kyun Benchmark?

Kahani Se Samjho

Ek interview mein sabse powerful question: *“Ye multi-agent system actually single LLM se better hai? Prove karo.”* **Benchmark** isi ka jawab hai — ek hi topic par dono systems chalate hain aur depth + verifiability score karte hain.

- **Single-agent baseline:** Ek LLM call — research + report ek saath (na verification, na revision loop).
- **Multi-agent pipeline:** Poora 4-agent system.
- **Judge:** LLM dono reports ko 0-10 par score karta hai (depth, verifiability) aur verdict deta hai.

```

1 SINGLE_AGENT_PROMPT = """You are an expert research analyst.
2 Write a comprehensive, detailed Markdown research report about the topic
3 below. Cover market overview, technical details, key players, challenges,
4 and future outlook. Include section headings and cite sources inline
5 wherever possible.
6 TOPIC: {topic}"""
7
8 def run_single_agent_baseline(topic: str) → dict:
9     """One LLM call --- no fact-checking, no revision loop, no RAG."""
10    groq_key = get_groq_api_key()
11    report = f"# Research Report: {topic}\n\n*Single-agent baseline*"
12    try:
13        if groq_key:
14            from langchain_groq import ChatGroq
15            from langchain_core.messages import SystemMessage, HumanMessage
16            llm = ChatGroq(model=MODEL_NAME, temperature=0.5,
17                           groq_api_key=groq_key)
18            response = llm.invoke([
19                SystemMessage(content=SINGLE_AGENT_PROMPT.format(topic=topic)
20                               ),
21                HumanMessage(content=f"Write the report on: {topic}"),
22            ])
23            report = response.content
24        except Exception as e:
25            print(f"[benchmark] Single-agent baseline fallback: {e}")
26            return {"text": report, "topic": topic}
27
28 def _depth_score(text: str) → int:

```

```

28     if not text:
29         return 0
30     headings = len(re.findall(r"^\#{1,4}\s.+$", text, flags=re.MULTILINE))
31     bullets = len(re.findall(r"^\s*[-*+]\s", text, flags=re.MULTILINE))
32     words = len(text.split())
33     score = min(headings, 8) + min(bullets // 4, 4) + min(words // 150, 3)
34     return max(0, min(10, int(score)))
35
36 def _verifiability_score(text: str) → int:
37     urls = len(re.findall(r"https?:\/\/\S+", text))
38     citations = len(re.findall(r"\[\d+\]\|\[Source[: ]\|(\[Source[: ]",
39                             text, flags=re.IGNORECASE))
40     score = min(urls, 5) * 1.5 + min(citations, 5)
41     return max(0, min(10, int(score)))
42
43 def evaluate_outputs(topic, single_agent_text, multi_agent_text) → dict:
44     metrics = _heuristic_metrics(single_agent_text, multi_agent_text)
45     # Optional: LLM-as-a-judge with Groq; fallback to heuristics
46     if get_groq_api_key():
47         try:
48             # LLM judge JSON parse; on any error use heuristic verdict
49             ...
50         except Exception:
51             verdict = _heuristic_verdict(metrics)
52     else:
53         verdict = _heuristic_verdict(metrics)
54     metrics["verdict"] = verdict
55     return metrics

```

Listing 12: benchmark/evaluator.py — Key Functions

🔗 Code Samjho Line-by-Line

Design Points:

- **Heuristic scores** (depth: headings/bullets/words; verifiability: URLs/citations) — hamesha available, zero cost.
- **LLM judge** — jab Groq key ho toh JSON scores + verdict. Agar JSON parse fail ho, heuristics par fallback. **Double-layer: deterministic + LLM.**
- `baseline["text"]` dict return — Django view ise `baseline["text"]` se read karta hai.

👤 Interview Question

Q: “RAGAS framework se evaluate karte toh kaise?”

Answer: RAGAS metrics: **Faithfulness** (report ke claims retrieved context par grounded hain?), **Answer Relevancy** (answer query ke intent se match karta hai?), **Context Recall/Precision** (retrieved chunks ground truth ke relevant the?). Humara evaluator lightweight version hai — depth/verifiability heuristics + LLM judge. Production mein RAGAS add kar sakte hain.

19 Code Deep Dive — Django Backend (backend/)

19.1 Models — Database Tables

```

1 from django.db import models
2 from django.contrib.auth.models import User
3
4 class ResearchSession(models.Model):
5     user = models.ForeignKey(User, on_delete=models.CASCADE,
6                             null=True, blank=True)
7     topic = models.CharField(max_length=512)
8     final_report = models.TextField()
9     web_sources_count = models.IntegerField(default=0)
10    rag_chunks_count = models.IntegerField(default=0)
11    revision_count = models.IntegerField(default=0)
12    created_at = models.DateTimeField(auto_now_add=True)
13
14    def __str__(self):
15        return f"{self.topic} ({self.created_at.strftime('%Y-%m-%d %H:%M')})"
16
17 class BenchmarkEvaluation(models.Model):
18     topic = models.CharField(max_length=512)
19     single_agent_depth = models.IntegerField(default=0)
20     multi_agent_depth = models.IntegerField(default=0)
21     single_agent_verifiability = models.IntegerField(default=0)
22     multi_agent_verifiability = models.IntegerField(default=0)
23     verdict = models.TextField()
24     created_at = models.DateTimeField(auto_now_add=True)

```

Listing 13: backend/api/models.py — Poora Code

🔗 Code Samjho Line-by-Line

Model Design Samjho:

- **ResearchSession** — har research run ka record: user (optional FK), topic, report, counts, timestamp. Ye history/audit trail hai.
- **BenchmarkEvaluation** — har benchmark run ke scores + verdict. Ye Streamlit Benchmark Lab page ke charts ka data source hai.
- **on_delete=models.CASCADE** — user delete hua toh uske sessions bhi (referential integrity).
- **auto_now_add** — **created_at** automatically set, kabhi modify nahi hota.
- **__str__** — admin panel mein readable representation.

19.2 Serializers — JSON Conversion

```

1 from rest_framework import serializers
2 from django.contrib.auth.models import User
3 from .models import ResearchSession, BenchmarkEvaluation
4
5 class UserSerializer(serializers.ModelSerializer):
6     class Meta:
7         model = User

```

```

8         fields = ["id", "username", "email"]
9
10    class RegisterSerializer(serializers.ModelSerializer):
11        password = serializers.CharField(write_only=True)
12        class Meta:
13            model = User
14            fields = ["username", "email", "password"]
15        def create(self, validated_data):
16            user = User.objects.create_user(
17                username=validated_data["username"],
18                email=validated_data.get("email", ""),
19                password=validated_data["password"],
20            )
21            return user
22
23    class ResearchSessionSerializer(serializers.ModelSerializer):
24        class Meta:
25            model = ResearchSession
26            fields = "__all__"
27
28    class BenchmarkEvaluationSerializer(serializers.ModelSerializer):
29        class Meta:
30            model = BenchmarkEvaluation
31            fields = "__all__"

```

Listing 14: backend/api/serializers.py — Key Parts

🔗 Code Samjho Line-by-Line

Samjho:

- `ModelSerializer` — model se automatically fields detect karta hai.
- `password = serializers.CharField(write_only=True)` — password response mein kabhi nahi aata (security).
- `create` override — `create_user` use hota hai (password hash hota hai), direct `create` nahi (jo raw password rakhta).
- `fields = "__all__"` — saare model fields serialize.

19.3 Views — API Logic

```

1    class ExecuteResearchAPIView(APIView):
2        permission_classes = [permissions.AllowAny]
3
4        def post(self, request):
5            topic = request.data.get("topic")
6            if not topic:
7                return Response({"error": "Topic is required"},
8                                status=status.HTTP_400_BAD_REQUEST)
9
10           initial_state = { ... } # same as Streamlit/MCP
11
12           try:
13               from graph.workflow import research_graph
14               result = research_graph.invoke(initial_state,
15                                             {"recursion_limit": 25})
16
17               user = request.user if request.user.is_authenticated else None
18               session = ResearchSession.objects.create(

```



```

19         user=user, topic=topic,
20         final_report=result.get("final_report", ""),
21         web_sources_count=len(result.get("research_data", [])),
22         rag_chunks_count=len(result.get("rag_context", [])),
23         revision_count=result.get("revision_count", 0),
24     )
25
26     return Response({
27         "session_id": session.id, "topic": topic,
28         "final_report": result.get("final_report", ""),
29         "messages": result.get("messages", []),
30         "revision_count": result.get("revision_count", 0),
31         "sources_count": len(result.get("research_data", [])),
32     })
33 except Exception as e:
34     return Response({"error": f"Execution failed: {e}"},
35                     status=status.HTTP_500_INTERNAL_SERVER_ERROR)

```

Listing 15: backend/api/views.py — ExecuteResearchAPIView

🔗 Code Samjho Line-by-Line

Important Design:

- `permission_classes = [AllowAny]` — demo ke liye open; production mein token required.
- **Validation first:** topic missing toh 400 — fail fast.
- **Lazy import** graph — server startup fast.
- **DB persistence:** `ResearchSession.objects.create(...)` — har run save hota hai (audit). Counts `len()` se calculate.
- **Authenticated user linkage:** `request.user.is_authenticated` check — anonymous runs par `user=None`.
- **Error handling:** `except Exception` — 500 with message, client ko pata chalega.

19.4 URLs + Auth Flow

Listing 16: API Endpoint Map

```

1 GET  /api/health/           -> online status
2 POST /api/auth/register/    -> username+email+password -> user + token
3 POST /api/auth/login/       -> username+password -> user + token
4 POST /api/research/execute/ -> {topic} -> report + session_id
5 POST /api/research/benchmark/-> {topic} -> comparison + metrics
6 GET  /api/benchmark/history/ -> list of BenchmarkEvaluation (newest first)

```

👤 Interview Question

Q: “Django mein migrations kya hain? Is project mein kaise use hue?”

Answer: Migrations database schema ka version control hain — model mein change karo, python `manage.py makemigrations` se migration file banti hai, `migrate` se apply. Is project mein `0001_initial` dono tables banata hai (`ResearchSession`, `BenchmarkEvaluation`). Interviewer ke liye bonus: migrations team collaboration mein critical hain — har developer same schema par aata hai.

 Interview Question

Q: “Ek hi pipeline ko 3 jagah use kiya hai — Streamlit, MCP, Django. Code duplication kaise avoid kiya?”

Answer: Compiled graph ek **singleton** hai — `graph/workflow.py` mein `research_graph = build_graph()` module level par. Teeno interfaces sirf from `graph.workflow` import `research_graph` karte hain. Initial state bhi same structure ka hai. Sirf invocation method alag hai: Streamlit `stream()` (real-time), MCP aur Django `invoke()` (final result).

20 Code Deep Dive — Benchmark Lab (pages/) + pytest

20.1 Streamlit Multipage

 Concept Deep Dive

Streamlit **multipage** convention: `pages/` directory mein har `.py` file ek page hai. Is project mein:

- `app.py` — main research page.
- `pages/1_Benchmark.py` — Benchmark Lab.

Sidebar mein automatic navigation dikhta hai. Page order number prefix se control hota hai.

```

1 def _get_api_url() → str:
2     url = os.getenv("BACKEND_API_URL", "")
3     if not url:
4         try:
5             url = str(st.secrets.get("BACKEND_API_URL", ""))
6         except Exception:
7             pass
8     return url.rstrip("/") or "http://localhost:8000/api"
9
10 def run_benchmark(topic: str) → dict:
11     r = requests.post(BENCHMARK_URL, json={"topic": topic}, timeout=900)
12     r.raise_for_status()
13     return r.json()
14
15 def load_history() → list[dict]:
16     r = requests.get(HISTORY_URL, timeout=15)
17     r.raise_for_status()
18     return r.json()
19
20 def history_frame(records: list[dict]) → pd.DataFrame:
21     rows = []
22     for item in records:
23         rows.append({
24             "Run": f"#{item['id']}", "created_at": item["created_at"],
25             "topic": item["topic"],
26             "single_agent_depth": item["single_agent_depth"],
27             "multi_agent_depth": item["multi_agent_depth"],
28             "single_agent_verifiability": item["single_agent_verifiability"],
29             "multi_agent_verifiability": item["multi_agent_verifiability"],

```

```

30         "verdict": item["verdict"],
31     })
32     df = pd.DataFrame(rows)
33     if not df.empty:
34         df["created_at"] = pd.to_datetime(df["created_at"], errors="coerce")
35         df = df.sort_values("created_at")
36     return df

```

Listing 17: pages/1_Benchmark.py — API Client + Charts

🔗 Code Samjho Line-by-Line

Samjho:

- BACKEND_API_URL — env var -> st.secrets -> default. **Environment-agnostic config.**
- requests.post(..., timeout=900) — benchmark 1-3 minute leta hai, isliye bada timeout.
- history_frame — API JSON ko pandas DataFrame mein convert (charts ke liye).
- Charts: st.line_chart (scores over time), st.bar_chart (per-run comparison), st.dataframe (history table).

20.2 Testing with pytest

Listing 18: pytest.ini

```

[pytest]
DJANGO_SETTINGS_MODULE = orchestrator_backend.settings
pythonpath = backend
testpaths = backend
addopts = -q

```

```

1  @pytest.fixture
2  def mock_research_graph(monkeypatch, research_state):
3      """Compiled LangGraph ko stub se replace karo --- tests fast aur free."""
4      import graph.workflow
5      class FakeGraph:
6          def invoke(self, state, config=None):
7              return dict(research_state)
8      monkeypatch.setattr(graph.workflow, "research_graph", FakeGraph())
9      return FakeGraph()
10
11 @pytest.fixture
12 def mock_benchmark(monkeypatch):
13     """LLM judge ko stub karo --- koi API call nahi."""
14     import benchmark.evaluator
15     metrics = {
16         "single_agent": {"depth_score": 5, "verifiability_score": 4},
17         "multi_agent": {"depth_score": 9, "verifiability_score": 8},
18         "verdict": "MULTI_AGENT_SUPERIOR",
19     }
20     monkeypatch.setattr(benchmark.evaluator,
21         "evaluate_outputs", lambda *a, **k: metrics)
22     return metrics

```

Listing 19: backend/conftest.py — Mocking Pipeline

🔗 Code Samjho Line-by-Line

Testing Strategy Samjho (Interview Gold):

- **Unit tests:** Har endpoint ka behavior — bina LLM/network ke. Pipeline ko stub/mock karte hain.
- **Isolation:** monkeypatch fixtures graph aur evaluator ko fake se replace karte hain — tests milliseconds mein chalti hain, zero API cost, deterministic.
- **Coverage:** auth (register/login), research execution (validation, persistence, error 500), benchmark (comparison, persistence, history, limit).
- **Database:** pytest-django test database banata hai — real db.sqlite3 kabhi touch nahi hota.
- **Result:** 16 tests, 8 second.

👤 Interview Question

Q: “Testing mein mocking kyun? Integration vs unit tests?”

Answer: Mocking external dependencies (LLM, search API) se tests fast, deterministic, aur free hote hain — har run mein same result. Unit tests endpoints ki logic verify karte hain (validation, persistence, response shape). Integration tests (live APIs ke saath) alag se chala sakte hain — slow, costly, isliye CI mein unit tests chalti hain. Best practice: **pyramid** — bahut saare fast unit tests, kuch integration, few end-to-end.

Part I

Deep Dives — Andar Ki Duniya

21 LangGraph Internals — Graph Engine Ki Andar Ki Baat

21.1 StateGraph Kya Hai, Asal Mein?

 Kahani Se Samjho

Ab tak humne `graph/workflow.py` ko line-by-line dekha. Ab is section mein hum **andar jayenge** — LangGraph ka engine kaise kaam karta hai, node kab chalta hai, state kaise flow hoti hai, aur conditional edge ka jadoo kya hai. Ye concepts interview mein **gold** hain — kyunki har koi bolta hai “maine LangGraph use kiya”, par koi nahi batata ki uske andar kya ho raha hai.

 Concept Deep Dive**Graph Abstraction ka Core Idea:**

Ek LangGraph app = **state** + **nodes** + **edges**.

- **State:** Ek TypedDict (`ResearchState`) jo pura pipeline ki memory hai — topic, research_data, analysis, report, sab kuch.
- **Nodes:** Functions jo state lete hain aur ek partial update return karte hain.
- **Edges:** Bataate hain ki kaun sa node kis ke baad chalega.
- **Conditional Edges:** Runtime par decide karte hain ki agla node kaun sa hai.

Iska MATLAB: pipeline ka flow code mein hardcoded nahi hai — wo **graph definition** hai jo runtime par execute hota hai.

21.2 Node Execution Ka Mechanism

 Code Samjho Line-by-Line**Node function ki signature (hamesha):**

```
1 def researcher_node(state: ResearchState) → dict:
2     # state = ab tak ka pura data (read-only view)
3     # kuch kaam karo
4     return {
5         "research_data": all_results,
6         "messages": [...],
7     }
```

Listing 20: Ek node ka contract

Contract:

- Input: poora **state** dictionary (read karo, mutate mat karo).
- Output: sirf **jo fields update karne hain** unka dict.

- LangGraph output dict ko purani state par **merge** karta hai.

√* Maths Behind the Scene

State update ka math:

Agar purani state S hai aur node δ update return karta hai, to:

$$S' = S \oplus \delta$$

Jahan $\oplus$ default merge = shallow dictionary merge. Special fields (jaise `messages` with `add_messages` reducer) ka apna merge rule hota hai.

Shallow merge MATLAB: nested values replace ho jaati hain, top-level keys add/update hoti hain. Isliye har node sirf apne fields return karta hai — baaki state safe rehti hai.

21.3 add_messages Reducer — Messages Ka Jadoo

🧠 Concept Deep Dive

`messages` field par `Annotated[list, add_messages]` lagaya hai. Ye LangGraph ko batata hai: **“is field ko simple replace mat karo, balki append/merge karo.”**

Agar har node `messages`: `["naya message"]` return kare, to:

- Normal merge: pichla message **erase** ho jata (sirf last wala bachega).
- `add_messages` merge: naya message **append** hota hai.

Is project mein actually `messages` ko sirf logging ke liye use kiya hai — par architecture mein ye pattern important hai kyunki RAG/chat agents mein message history accumulate karni hoti hai.

21.4 Compile() Ke Andar Kya Hota Hai

🔗 Code Samjho Line-by-Line

```

1 graph = StateGraph(ResearchState)
2 graph.add_node("researcher", researcher_node)
3 graph.add_node("analyst", analyst_node)
4 graph.add_node("fact_checker", fact_checker_node)
5 graph.add_node("writer", writer_node)
6 graph.add_edge(START, "researcher")
7 graph.add_edge("researcher", "analyst")
8 graph.add_edge("analyst", "fact_checker")
9 graph.add_conditional_edges(
10     "fact_checker",
11     should_continue,
12     {"continue": "researcher", "end": "writer"}
13 )
14 graph.add_edge("writer", END)
15 research_graph = graph.compile()
```

compile() kya karta hai:

- Nodes aur edges ko ek internal **execution plan** mein convert karta hai.
- Har node ke aage/peeche validation layers add karta hai.

- Ek **CompiledStateGraph** object deta hai jisme `.invoke()`, `.stream()`, `.get_graph()` jaise methods hain.
- Compile par hi LangGraph ko pata chalta hai ki graph **connected** hai ya nahi (har node reachable hai?).

21.5 invoke() vs stream() — Do Tarike

💡 Yaad Rakhne Ka Tip

invoke(): Poora graph end-to-end chalao, final state lao. Blocking call.

stream(): Har node ke baad intermediate results yield karo. Real-time progress dikhane ke liye (Streamlit mein use hota hai).

- `invoke` returns dict — final state.
- `stream` returns generator of (node_name, output) tuples.

🔗 Code Samjho Line-by-Line

Streamlit app mein dono ka use:

```
1 # Streamlit: real-time streaming
2 for event in research_graph.stream(state, config=config):
3     for node_name, node_output in event.items():
4         st.session_state.agent_statuses[node_name] = "done"
5
6 # Django / MCP: final result chahiye
7 final_state = research_graph.invoke(state)
8 report = final_state["final_report"]
```

Listing 21: stream vs invoke in practice

21.6 should_continue — Graph Ka Traffic Cop

🔗 Code Samjho Line-by-Line

```
1 def should_continue(state: ResearchState) → str:
2     if state.get("fact_check_passed", False):
3         return "end"
4     if state.get("revision_count", 0) >= MAX_REVISIONS:
5         return "end"
6     return "continue"
```

Listing 22: Conditional edge logic

Traffic cop ka kaam:

- `fact_check_passed == True` → "end" → writer chalao.
- Revision count limit cross ho gaya → "end" (infinite loop se bachao).
- Warna "continue" → researcher se naye queries ke saath dubara research.

Why this matters in interviews: Ye ek **feedback loop** hai — agent apne output ko khud verify karta hai aur improve karta hai. Single-prompt LLM mein ye loop nahi hota.

⚠ Dhyān Do

Infinite loop ka danger: Agar MAX_REVISIONS check nahi hota aur fact_checker hamesha FAIL deta, to graph researcher → analyst → fact_checker mein kabhi na khatam hone wala chakkar laga deta. Isliye:

1. revision_count har loop mein increment hota hai.
2. MAX_REVISIONS (default 3) par hard stop.
3. Stop hone par bhi report generate hoti hai (best-effort).

21.7 LangGraph Bina — Manual Loop Kaisa Dikhta

🔗 Code Samjho Line-by-Line

Agar LangGraph na hota, to pipeline aise likhni padti:

```

1 def run_pipeline(topic):
2     state = initial_state(topic)
3     for revision in range(MAX_REVISIONS + 1):
4         state.update(researcher_node(state))
5         state.update(analyst_node(state))
6         result = fact_checker_node(state)
7         state.update(result)
8         if result["fact_check_passed"]:
9             break
10    state.update(writer_node(state))
11    return state

```

Listing 23: Manual pipeline (no framework)

Problems manual loop mein:

- Flow logic (kaun kis ke baad) code mein khoya hai — ek for loop ke andar.
- Naya node add karna = manual loop ko todna.
- Streaming / checkpointing / parallel nodes khud banana padega.
- Testing hard — pura pipeline ek monolithic function.

LangGraph gives: declarative definition, built-in streaming, checkpointing, visualization, parallelism, aur clean testing.

👤 Interview Question

Q: “StateGraph vs simple function chain — overhead kya hai?”

Answer: Honest answer — is project ke scale par (4 nodes, linear flow) framework ka overhead minimal hai, aur benefits practical hain: (1) Conditional loop cleanly express hota hai; (2) stream() se UI progress milta hai bina alag threading ke; (3) Agar future mein parallel nodes (jaise 2 researchers alag topics par) chahiye, graph extend karna trivial hai. Overhead sirf ek extra abstraction layer hai jo code readability ko improve karta hai.

21.8 LangGraph vs CrewAI vs AutoGen — Comparison

Feature	LangGraph	CrewAI	AutoGen
Core model	Graph/state chine	ma- Role-based crews	Conversational agents

Control flow	Explicit edges, conditions	Sequential/parallel tasks	Agent-to-agent chat
State management	TypedDict + reducers	Task outputs	Conversation history
Streaming	Built-in (.stream)	Limited	Built-in
Debugging	Graph visualization, tracing	Medium	Medium
Learning curve	Steep (graph thinking)	Gentle	Medium
Is project mein?	YES — full control	No	No

Concept Deep Dive

Is project mein LangGraph kyun? Kyunki is pipeline ka **heart** ek **conditional feedback loop** hai (fact-check → revise). Graph model isko naturally express karta hai. CrewAI role-based approach mein aisa loop force karna awkward hota. AutoGen conversational hai — research pipelines ke liye less structured.

22 Embeddings Deep Dive — Vector Banane Ka Science

22.1 Embedding Kya Hai — Ek Sentence Se 384 Numbers Tak

Kahani Se Samjho

Humne `rag/vector_store.py` mein `HuggingFaceEmbeddings` use kiya. Ab samjho: jab hum PDF ka ek chunk lete hain, `all-MiniLM-L6-v2` model us chunk ko **384 numbers** ki ek list mein convert karta hai. Ye numbers = us text ka **semantic fingerprint**.

Key insight: Do similar sentences ke embeddings **near** hote hain (vector space mein distance kam), dissimilar ke far. Isi par similarity search chalti hai.

Maths Behind the Scene

Cosine Similarity — Retrieval Ka Formula:

Do vectors $\vec{a}$ aur $\vec{b}$ ke beech similarity:

$$\text{similarity}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|} = \frac{\sum_{i=1}^n a_i b_i}{\sqrt{\sum_{i=1}^n a_i^2} \cdot \sqrt{\sum_{i=1}^n b_i^2}}$$

- Result $\in [-1, 1]$. 1 = same direction, 0 = orthogonal, -1 = opposite.
- Sentence embeddings normalize hote hain, to cosine similarity = dot product.
- ChromaDB isi score par top-k results laati hai.

22.2 all-MiniLM-L6-v2 Model — Chhota Par Zabardast

 Concept Deep Dive**Model specs:**

- Name: `sentence-transformers/all-MiniLM-L6-v2`
- Architecture: MiniLM (distilled BERT) — 6 transformer layers.
- Output dimensions: **384** (BERT-base 768 hota hai, isne compress kiya).
- Size: ~80MB — RAM mein load hota hai, GPU ki zaroorat nahi.
- Speed: CPU par bhi fast (isliye Streamlit app start hote hi load ho jata hai).
- Quality: 384-dim par bhi semantic search ke liye solid.

 Memory Trick**Why 384 not 768 or 1536?**

Trade-off: **quality vs speed/memory**.

- Higher dims = better nuance, par zyada RAM, slower search.
- `all-MiniLM-L6-v2` (384) is project ke scale ke liye perfect: 1000s documents, CPU-only environment, real-time search chahiye.
- OpenAI `text-embedding-3-small` (1536 dims) zyada powerful par paid API + network call.
- Interview answer: “Mainne quality-speed trade-off socha — local, free, fast chahiye tha, isliye 384-dim MiniLM.”

22.3 Chunking — PDF Ko Katna Kaise

 Code Samjho Line-by-Line

```

1 CHUNK_SIZE = 1000          # har chunk ~1000 characters
2 CHUNK_OVERLAP = 200        # 200 chars overlap pichle chunk se
3
4 splitter = RecursiveCharacterTextSplitter(
5     chunk_size=CHUNK_SIZE, chunk_overlap=CHUNK_OVERLAP
6 )
7 chunks = splitter.split_documents(docs)

```

Listing 24: Chunking settings

RecursiveCharacterTextSplitter ka kaam:

- Pehle **paragraph breaks** par split karta hai (`\n\n`).
- Phir **sentences** (`\n, .`), phir words, phir characters.
- Har level par `chunk_size` limit check karta hai.
- Overlap ensure karta hai ki context chunk boundary par na kate.

 Dhyan Do**Chunking ke 3 common galtiyan:**

1. **Too small** (100 chars): har chunk mein adhura context — retrieval par garbage answers.
 2. **Too big** (5000+): ek chunk mein bahut saari cheezein — embedding dilute, irrelevant chunks retrieve hote hain.
 3. **Zero overlap**: Sentence beech mein kat jata hai, meaning khota hai.
- Is project mein 1000/200 balanced choice hai — generic documents ke liye.

22.4 ChromaDB — Vector Store Ke Andar

Concept Deep Dive

ChromaDB ki key concepts:

- **Collection:** Vector ka folder (`research_docs`, `past_research`).
- **Embedding function:** Kisi bhi text ko vector mein convert karne wala callable.
- **Persistence:** `persist_directory` par disk mein save hota hai (SQLite + HNSW index files).
- **Similarity search:** Query embedding → collection ke saare vectors se cosine compare → top-k.

Maths Behind the Scene

HNSW — Approximate Nearest Neighbor:

10000 chunks ho to har query par saare 10000 se compare karna slow hai. HNSW (Hierarchical Navigable Small World) ek **graph-based index** banata hai:

- Layers of proximity graphs (top layer = coarse, bottom = fine).
- Search upper layer se shuru, greedily neeche aata hai.
- **Recall** ~95% par **speed** 10-100x faster.

Interview line: “ChromaDB HNSW index use karti hai — approximate search se quality ka zyada nuksan nahi, par speed ka bada faida.”

22.5 Retrieval Quality — Kya Galt Ho Sakta Hai

Interview Question

Q: “Agar retrieval galat chunks laaye to kya hoga?”

Answer: Garbage in, garbage out — RAG system ka output utna hi acha jitna retrieved context. Is project mein 3 safeguards:

1. **Multi-source:** Tavily web results + `research_docs` + `past_research` teen alag sources se context milta hai.
2. **Analyst filter:** Analyst sirf `research_data[:10]` aur `rag_context[:5]` le kar dense brief banata hai — raw retrieval se pehle hi refine.
3. **Fact-checker:** Har claim sources se verify hota hai — hallucinated retrieval bhi pakda jata hai.

Memory Trick

Failure modes table:

Failure	Kya hota	Mitigation
Chunk context cut	Adhura answer	Overlap 200
Wrong collection	Galat docs retrieve	Collection names separate
Empty collection	Kuch retrieve nahi	try/except → 0 stats
Embedding mismatch	Garbage similarity	Same model everywhere
Query too specific	No good match	Multi-query expansion (researcher)

23 Prompt Engineering Masterclass — Har Prompt Ki Kahani

23.1 Prompt Engineering Kyun Zaroori Hai

Kahani Se Samjho

LLM ek genie hai — par genie se acha wish maangna ek skill hai. Is project mein **5 alag prompts** hain, har ek apne agent ke liye design kiya gaya. Interview mein jab aap in prompts ko **khol ke** samjhaoge — kyun aisa likha, kya problem hoti agar aisa na likha — wo interviewer ko dikhata hai ki aapne sirf code copy nahi kiya, **socha** hai.

Concept Deep Dive

Har prompt mein 3 cheezein hoti hain:

1. **Role:** “You are a ...” — LLM ko identity deta hai, tone/behavior set karta hai.
2. **Task:** Exactly kya karna hai — clear, measurable.
3. **Output format:** Structure jisme answer chahiye — parse karna aasan hota hai.

Iske alawa **constraints** (“Return ONLY”, “No extra text”) hallucinations/extra fluff rokne ke liye.

23.2 Prompt 1: Researcher ka Query Generator

Code Samjho Line-by-Line

Verbatim prompt (agents/researcher.py):

```
1 """You are a research query strategist.
2 Given a research topic, generate exactly 3 diverse, specific
3 search queries that will cover different angles of the topic.
4 Return ONLY the 3 queries, one per line. No numbering,
5 no extra text."""
```

Listing 25: QUERY_GEN_PROMPT — asli code se

Design decisions:

- **Role** (“research query strategist”): Model ko search-expert mode mein daalta hai.
- **“exactly 3”**: Fixed count — baad mein `lines[:3]` slice se parse hota hai.
- **“diverse, specific”**: Ek hi angle ke 3 similar queries nahi, alag angles chahiye.
- **“Return ONLY ... one per line. No numbering”**: Output parse karna trivial — `split("\n"), strip, [:3]`.

Interview Question

Q: “Prompt mein ‘one per line’ kyun? JSON kyun nahi mangaya?”

Answer: 3 cheezein: (1) **Simplicity** — line-separated output ko parse karna ek split hai, JSON parse mein overhead aur failure modes zyada. (2) **Token efficiency** — chhota prompt = kam tokens = kam cost/latency. (3) **Reliability** — LLM JSON format mein kabhi kabhi

braces bhool jata hai, line list galat nahi hoti. Is pattern ko “structured output with minimal structure” kehte hain.

23.3 Prompt 2: Revision Queries

Code Samjho Line-by-Line

Verbatim (agents/researcher.py):

```
1 """You are a research query strategist.
2 The fact-checker flagged issues with previous research.
3 Based on the feedback below, generate 3 NEW, improved search
4 queries that address the gaps and inaccuracies.
5 Return ONLY the 3 queries, one per line. No numbering,
6 no extra text.
7
8 FACT-CHECKER FEEDBACK:
9 {feedback}"""
```

Listing 26: REVISION_PROMPT

Key insight — feedback loop: Ye prompt tab chalta hai jab fact-checker **NEEDS_REVISION** deta hai. Researcher ko pichli baar ki galtiyan batayi jaati hain, aur wo **targeted** naye queries banata hai. Ye hi multi-agent ka power hai: **memory of failure** next attempt mein use hoti hai.

Note: {feedback} placeholder `.format()` se bharta hai — template injection ke liye careful rehna.

23.4 Prompt 3: Analyst — Structure Se Synthesis Tak

Code Samjho Line-by-Line

Verbatim (agents/analyst.py):

```
1 """You are an expert analyst. From the data:
2 1. Identify 3-5 KEY THEMES
3 2. Note CONTRADICTIONS (conflicts between sources)
4 3. Highlight CONSENSUS points (where multiple sources agree)
5 4. Flag GAPS (what is missing)
6 Provide a structured output in Markdown."""
```

Listing 27: Analyst SYSTEM_PROMPT

Why this structure:

- **THEMES:** 15 raw sources ko 3-5 themes mein compress — writer ke liye digestible.
- **CONTRADICTIONS:** Research mein disagreements utne hi important jitne agreements.
- **CONSENSUS:** Multiple sources agree → claim zyada reliable.
- **GAPS:** Jo data nahi mila — usse writer honest reporting karta hai, hallucinate nahi.
- **Markdown output:** Baad mein writer ke prompt mein seedha pass hota hai.

Memory Trick

Analyst temperature = 0.2 (sabse kam, fact-checker ke baad). Kyun? Analyst **summarize/compress** karta hai — factual aggregation, creativity nahi chahiye. Writer 0.4 (prose flair), researcher 0.3 (query diversity thodi chahiye). **Interview pearl:** “Har agent ka temperature uske task ke nature se decide hota hai — analyst factual hai isliye low, writer creative isliye higher.”

23.5 Prompt 4: Fact Checker — Closed-World Assumption

Code Samjho Line-by-Line

Verbatim (agents/fact_checker.py):

```
1 """You are a rigorous fact-checker and quality reviewer.
2 Review the analysis below and verify each claim against the
3 source data provided.
4
5 For each claim:
6 - Mark as VERIFIED if supported by sources
7 - Mark as UNVERIFIED if no source backs it
8 - Mark as CONTRADICTED if sources disagree
9
10 At the very end, on the LAST LINE, write exactly one of:
11 VERDICT: PASSED
12 VERDICT: NEEDS_REVISION
13
14 Use NEEDS_REVISION only if there are significant unverified
15 claims or critical gaps."""
```

Listing 28: Fact checker SYSTEM_PROMPT — system ka dil

Closed-world assumption (critical!):

- Model ko sirf **diye gaye sources** par verify karna hai.
- Apne training knowledge (outside data) se jo pata hai — wo **count nahi** hota.
- Isliye **UNVERIFIED** ka matlab: “source data mein nahi mila” — chaahe model ko pata ho.
- Ye hallucination ko rokta hai: analyst ka claim sirf tab VERIFIED jab source backup kare.

VERDICT parsing: `result.upper()` mein “VERDICT: NEEDS_REVISION” substring check — robust (case-insensitive), aur LLM extra text likh de to bhi kaam karta hai.

Dhyan Do

Closed-world ki limitation: Agar sources galat hain (internet par galat info), fact-checker unhe **VERIFIED** mark karega. Fact-checker source ki **truth** nahi, source ki **existence** verify karta hai. Ye limitation honest interview mein batana **impression** banata hai.

23.6 Prompt 5: Writer — Final Report Ka Architect

</> Code Samjho Line-by-Line

Verbatim (agents/writer.py):

```
1 """You are a senior technical writer.
2 Synthesize the analysis and research data into a comprehensive,
3 highly detailed, beautifully formatted Markdown research report.
4 Include an Executive Summary, Key Findings, In-depth Technical
5 Breakdown, Consensus & Contradictions, and References/Citations."""
```

Listing 29: WRITER_PROMPT

Structure requirements:

- **Executive Summary** — 30 second reader ke liye.
- **Key Findings** — bullet-proof quick facts.
- **In-depth Technical Breakdown** — detail section.
- **Consensus & Contradictions** — analyst output ka echo.
- **References/Citations** — verifiability (benchmark ke **verifiability** score ka basis!).

Input discipline: Writer ko sirf analysis + research_data[:8] (top 8 sources) milta hai — 50+ sources nahi. Context window control = cost control + focus.

23.7 Prompt 6: Benchmark ka LLM Judge

</> Code Samjho Line-by-Line

Verbatim (benchmark/evaluator.py mein judge prompt):

```
1 """You are an impartial research quality judge.
2 Score the following two research reports on:
3 1. DEPTH (0-10): How comprehensive, detailed, and technically
4   accurate is the analysis?
5 2. VERIFIABILITY (0-10): How well does the report cite sources
6   and distinguish facts from opinions?
7
8 Report A (single-agent baseline):
9 {report_a}
10 Report B (multi-agent pipeline):
11 {report_b}
12
13 Return ONLY JSON: {"depth_a":...,"depth_b":...,
14 "verifiability_a":...,"verifiability_b":...,"verdict":".."}"""
```

Listing 30: LLM-as-judge scoring

LLM-as-judge pattern:

- Do reports ko **side-by-side** ek hi judge ko do — relative scoring absolute se zyada stable.
- **Verifiability** dimension: citations check karta hai — multi-agent pipeline ka advantage yahin dikhna chahiye.
- JSON output parse: `json.loads` ke liye.
- Fallback: Agar LLM judge fail ho (API down), **heuristic** scoring chalti hai — regex se section count, citation count, markdown structure.

23.8 Prompt Design Principles — Project Se Nikale Gaye Lessons

 Concept Deep Dive

7 principles is project ke prompts se:

1. **Role first** — har prompt role se shuru.
2. **Counts fixed** — “exactly 3”, “3-5”, “top 8” — parse-friendly.
3. **Output format explicit** — “one per line”, “Markdown”, “JSON”, “LAST LINE”.
4. **Negative constraints** — “No numbering, no extra text”.
5. **Verbatim source** — fact-checker ko raw sources do, summary nahi.
6. **Context window discipline** — har agent ko sirf utna do jitna zaroori.
7. **Robust parsing** — substring check (NEEDS_REVISION), try/except, fallbacks.

 Interview Question

Q: “Agar aapko prompt improve karne ko bola jaye to kya badalte?”

Answer: 3 cheezein: (1) **Analyst ko few-shot examples** — ek example theme/contradiction format ka, taaki output quality stable ho. (2) **Writer ko citation format enforce** — “har key finding ke baad [source N]” — isse verifiability score bhi improve hoga. (3) **Fact-checker ko per-claim JSON** — structured output jisse revision feedback researcher ko machine-readable mile. Ye 3 changes benchmark scores par measurable effect denge.

24 Real Run Postmortems — Jo Asal Mein Hua

24.1 Yeh Section Kaisa Hai

Kahani Se Samjho

Theory kafi ho gayi. Ab baat karte hain **asli runs** ki — jo maine is project par execute kiye, asli API keys ke saath. Ye numbers aur observations **real** hain. Interview mein jab aap bologe “maine pipeline chalayi, ye aaya, ye problem mili, aise fix kiya” — wo **experience** ka proof hai jo koi textbook nahi deta.

24.2 Run 1: Solid State Battery Market — End-to-End Success

Code Samjho Line-by-Line

Run details (real):

- **Topic:** “solid state battery market outlook 2026”
- **Duration:** ~176 seconds (end-to-end, real API calls)
- **Sources:** 15 web sources (3 sub-queries × 5 results) via Tavily
- **Fact-check:** **PASSED on first pass** — revision_count = 0
- **Output:** Publication-grade Markdown report
- **Persistence:** ResearchSession #1 in Django DB + ChromaDB memory

Report structure (jo LLM ne banaya):

1. Table of Contents
2. Executive Summary
3. Evidence-backed Key Findings table
4. Technical breakdown (chemistry table + CAGR table)
5. Consensus & Contradictions section
6. Citations/References

Concept Deep Dive

Is run se 3 lessons:

1. **First-pass fact-check success** — Analyst ka structured output (themes/contradictions/gaps) researcher ke raw data se itna consistent tha ki fact-checker ko koi critical gap nahi mili. Ye prompt design ka payoff hai.
2. **176s ka time budget** — Tavily search (3 calls) + 4 LLM calls + embedding store. Sabse bada chunk: Tavily web calls aur ChromaDB embedding computation.
3. **Determinism nahi hai** — LLM outputs vary karte hain; isliye benchmark multiple runs ka average mangta hai.

24.3 Run 2: RAG Benchmark — Jab Multi-Agent Haar Gaya

🔗 Code Samjho Line-by-Line

Run details (real):

- **Topic:** “Best practices for RAG in 2026”
- **Duration:** ~74.4s (baseline + pipeline + judge)
- **Baseline:** Single LLM call (writer prompt, no research)
- **Pipeline:** Full multi-agent flow (researcher → analyst → fact_checker → writer)
- **Judge:** LLM-as-judge, depth + verifiability (0-10)

Result:

Metric (0-10)	Single-agent	Multi-agent
Depth	9	8
Verifiability	8	7

Verdict: SINGLE_AGENT_SUPERIOR — single prompt ne benchmark jeeta!

⚠️ Dhyan Do

Ye failure gold hai. Multi-agent pipeline haar gayi kyunki us run mein Writer ko Analyst ka raw (excerpt-heavy) output mila aur report source digest jaisi ban gayi — synthesized prose nahi. Interview mein ise **honestly** batana: “Maine benchmark chalaya, ek run mein multi-agent haar gaya, ye cause tha, ye fix propose kiya” — ye dikhata hai ki aap apne system ki **weaknesses** jaante ho aur measurement seriously lete ho.

24.4 Why Did Multi-Agent Lose? — Root Cause Analysis

🧠 Concept Deep Dive

Handoff quality hypothesis:

Multi-agent pipeline ki quality **chain ki weakest link** jitni hai:

1. Researcher 15 raw sources deta hai (URL + content snippets).
2. Analyst unhe `research_data[:10]` tak limit karke **brief** banata hai — par is run mein brief excerpt-heavy tha, synthesis kam.
3. Writer ko brief + top-8 sources milte hain. Agar brief already excerpt-heavy hai, to Writer ke paas **synthesize karne ki material** nahi hoti — wo source list copy kar deta hai.

Contrast: Solid-state run mein Analyst ne clean themes/contradictions structure banaya, isliye Writer ne behtareen report likhi. RAG run mein handoff weak tha.

Conclusion: Pipeline deterministic nahi hai; ek run ka verdict **statistically meaningless** hai. 3-5 topics ka average chahiye.

24.5 Fix Proposals — Agar Mein Aage Kaam Karun

🔗 Code Samjho Line-by-Line

3 concrete fixes (implementation-ready):

1. **Analyst few-shot:** Analyst prompt mein 1 example of “theme with synthesis” — excerpt copy nahi, balki 2-line synthesized insight. Isse handoff quality stabilize hoti.
2. **Writer instructions:** “Har section mein at least 3 synthesized sentences likho jo multiple sources ko combine karein; direct source excerpt mat copy karo.” Negative instruction = prevention.
3. **Multi-topic benchmark:** Benchmark runner ko 5 fixed topics par chalao, average scores report karo. Single run ka verdict basura hai.

👤 Interview Question

Q: “Agar aapko 1 month mile is project ko improve karne, top 3 cheezein kya karoge?”

Answer: (1) **Evaluation infrastructure** — 20-topic benchmark suite + regression tracking, kyunki bina measurement ke improvement nahi hota. (2) **Parallel researcher agents** — 3 sub-queries ko alag parallel nodes mein chalao, latency 176s se 60s tak. (3) **Production hardening** — rate limiting, async task queue (Celery), observability (LangSmith tracing), aur vector store versioning. Ye 3 system ko demo se product tak le jaate hain.

25 System Design — Poora Architecture Ek Nazar Mein

25.1 High-Level Architecture

⚙️ System Design Deep Dive

4-layer architecture:

- **Presentation layer:** Streamlit (human UI) + MCP server (AI clients) + Django REST API (programmatic).
- **Orchestration layer:** LangGraph StateGraph — researcher/analyst/fact_checker/writer nodes + conditional loop.
- **Knowledge layer:** ChromaDB (research_docs + past_research) + Tavily web search.
- **Model layer:** Groq API (GPT-OSS-120B) + sentence-transformers embeddings.

Key design principle: Har layer apni boundary maintain karta hai. Interface files (`app.py`, `mcp_server.py`, `backend/api/views.py`) sirf `research_graph` ko call karte hain — pipeline implementation se decoupled.

25.2 Data Flow — Request Se Report Tak

Code Samjho Line-by-Line

Full request journey (Streamlit example):

1. User topic type karta hai, **Run** dabata hai.
2. `app.py` state banata hai: `{'topic': topic, 'messages': []}`.
3. `research_graph.stream(state)` shuru — har node ka event UI mein dikhta hai.
4. **Researcher**: 3 sub-queries (LLM) → 3 Tavily searches → ~15 sources + RAG retrieval (5 docs + 3 past) → state update.
5. **Analyst**: top-10 sources + top-5 rag chunks → themes/contradictions/gaps brief (LLM).
6. **Fact_checker**: brief + top-8 raw sources → VERDICT (LLM). Fail → researcher loop (max 3).
7. **Writer**: brief + top-8 sources → Markdown report (LLM) → ChromaDB memory store.
8. UI report render karta hai; **Download** buttons markdown/text.

25.3 Module Dependency Graph

Concept Deep Dive

Dependency direction (acha design):

```

1 app.py / mcp_server.py / backend/api/views.py
2   |           |           |
3   v           v           v
4   graph/workflow.py <----- (compiled singleton: research_graph)
5   |
6   v
7   agents/*.py -----> state/schema.py
8   |
9   +-----> config/setting.py
10  |
11  +-----> rag/vector_store.py

```

Observations:

- `state/schema.py` par **koi dependency nahi** — pure data contract.
- `config/setting.py` leaf node — environment/config sab isse aata hai.
- `rag/vector_store.py` sirf config par depend karta hai — reusable module.
- Har agent sirf state + config + rag use karta hai — **dependency inversion** (interface through state).

25.4 Failure Modes and Resilience

Failure	Kya hota	Is project mein handling
Groq API down	LLM calls fail	try/except, fallback text with error message
Tavily key missing	No web search	Notice message in <code>research_data</code>
ChromaDB missing	No RAG context	try/except pass (retrieval empty)

LLM returns garbage	Bad report	Fact-checker catches, revision loop
Model 404 (bad name)	All LLM calls fail	Fallback paths; fixed via config
Port 8000 busy	Django unreachable	Configurable BACK-END_API_URL

⚠ Dhyani Do

Resilience principle: Har external call (LLM, search, vector store) try/except mein wrapped hai aur system **degraded mode** mein kaam karta rehta hai. Interview line: “Maine fail-fast nahi, fail-graceful kiya — user ko hamesha kuch na kuch output milta hai, error message ke saath.”

25.5 Scaling Story — Kya Tootega, Kya Fix Hoga

🧠 Concept Deep Dive

Scaling analysis (interview-ready):

- **10 users:** Sab kuch chalta hai — Streamlit single instance, SQLite, local ChromaDB.
- **100 users:** SQLite concurrency limit → PostgreSQL. ChromaDB local → managed vector DB (Pinecone/Qdrant). Streamlit → multi-instance behind load balancer.
- **10k documents:** Embedding batch jobs (offline), HNSW index tuning, collection sharding.
- **1M requests:** Task queue (Celery/Redis) for async pipeline execution; results cached; webhook/poll for completion.
- **Cost:** Groq cheap; embedding local free; Tavily paid — caching + dedup search results.

Core scalability insight: Pipeline synchronous hai (176s/request). Production mein ise **async** karna first priority hai — user request par 176s wait nahi karta.

25.6 Why This Architecture (Not Monolith)

👤 Interview Question

Q: “Ek hi Python app mein sab kuch kyun nahi? 3 interfaces kyun?”

Answer: 3 alag **consumption patterns** hain: (1) **Human** real-time progress + download chahta hai → Streamlit. (2) **AI assistant** (Claude Desktop) tools ke through call karna chahta hai → MCP. (3) **Programmatic/benchmark** users REST API chahte hain → Django. Teeno ek hi compiled graph ko call karte hain — logic ek jagah, **interfaces alag**. Ye “single core, multiple adapters” pattern hai — jaise REST, GraphQL, gRPC sab ek service ke upar.

Part II

Interview Prep — 60+ Questions, Full Answers

26 Q&A Bank Part 1 — Fundamentals & LLMs

Kahani Se Samjho

Ye section 60+ interview questions ka bank hai — har question ka **pura answer** is project ke context mein. Pattern: pehle **concept** ka clean answer, phir **project connection** (“is project mein kahan use hua”). Interviewer ko ye combination pasand aati hai — theory + practice.

26.1 LLM Fundamentals

Interview Question

Q1: “LLM kya hai aur kaise kaam karta hai? Simple words mein.”

Answer: LLM (Large Language Model) ek deep neural network hai jo **billions of parameters** ke saath text ka pattern seekhta hai. Training: internet par se trillions of words dekhe, har word ka next word predict karna seekha (self-supervised learning). Inference: aap text dete ho, model token-by-token next word generate karta hai. Is project mein **GPT-OSS-120B** (120 billion parameters) use kiya hai via Groq API — har agent ek prompt dekar model se output leta hai.

Interview Question

Q2: “Token kya hota hai?”

Answer: Token text ki smallest unit hai jo model process karta hai — word ka part. “Hello world” $\approx$ 2 tokens; ek page text $\approx$ 1000-1500 tokens. Model **token-level** par kaam karta hai, character-level par nahi. Cost aur context window dono tokens mein measure hote hain. Is project mein context budget: writer ko top-8 sources + analysis bhejte hain taaki tokens limit mein rahein.

Interview Question

Q3: “Temperature kya karta hai? 0 vs 1?”

Answer: Temperature model ki **randomness** control karta hai. Low (0-0.2): deterministic, factual output — model hamesha most-likely token leta hai. High (0.7-1): creative, varied output. Is project mein: fact-checker 0.1 (factual), analyst 0.2 (aggregation), researcher 0.3 (thodi diversity queries mein), writer 0.4 (prose thoda natural). Har agent ka temperature **task ke nature** se decide hota hai.

 Interview Question

Q4: “Context window kya hai? Agar zyada text do to?”

Answer: Context window = model ek baar mein kitna text dekh sakta hai (jaise GPT-OSS-120B ka 128K tokens). Zyada denge to ya to truncate hoga, ya error, ya cost explode. Isliye is project mein **context discipline**: researcher 15 sources store karta hai, analyst sirf top-10 leta hai aur dense brief banata hai, writer sirf brief + top-8 sources dekhta hai. Ye layered compression hai.

 Interview Question

Q5: “Hallucination kya hai aur is project mein kaise control hua?”

Answer: Hallucination = model ka aisa content jo facts se match nahi karta, par confident lagta hai. Control ke 3 layers is project mein: (1) **RAG** — model ko sources do taaki wo answer copy kare, invent na kare; (2) **Fact-checker** — closed-world assumption: har claim sirf diye gaye sources se VERIFIED hota hai; (3) **Revision loop** — unverified claims milne par naye search queries se research improve hoti hai.

26.2 Prompting

 Interview Question

Q6: “Zero-shot, few-shot, chain-of-thought — kya difference?”

Answer: Zero-shot: bina example ke task do (is project ke zyada prompts aise hain). Few-shot: 2-3 examples do — output format stabilize hota hai. Chain-of-thought: model ko step-by-step sochne ko kaho — complex reasoning improve hota hai. Is project mein fact-checker ko “VERDICT: PASSED/NEEDS_REVISION” last line par format diya hai — structured output jo parse aasan hai.

 Interview Question

Q7: “System prompt vs user prompt?”

Answer: System prompt: model ki **role/behavior** define karta hai (persistent instructions). User prompt: actual task/data. Is project mein har agent ka **SYSTEM_PROMPT** constant hai (role + output format), aur **HumanMessage** mein topic + data jata hai. Ye separation code ko clean rakhta hai aur prompts ko reusable.

 Interview Question

Q8: “Prompt injection kya hai? Is project vulnerable hai?”

Answer: Prompt injection = user ke data ke through model ko system instructions ignore karwana. Example: source content mein “ignore previous instructions” likha ho. Is project mein risk: Tavily sources aur PDFs untrusted hain — unka content analyst/writer prompts mein jata hai. Mitigation: (1) sources sirf **data section** mein jate hain, instructions se alag; (2) fact-checker closed-world hai — claims sirf sources se verify; (3) Production mein input sanitization + output filtering add karna chahiye.

26.3 Retrieval-Augmented Generation (RAG)

 Interview Question**Q9: “RAG kya hai? Bina RAG ke LLM ka problem kya?”**

Answer: RAG = Retrieval-Augmented Generation: pehle relevant documents **retrieve** karo, phir unke saath LLM se answer generate karo. Bina RAG ke LLM: (1) apne training data tak limited (jaise 2024 tak ka); (2) private documents nahi jaanta; (3) hallucinate karta hai. RAG 3 problem solve karta hai: **freshness** (naye docs), **privacy** (private docs), **accuracy** (source-backed answers). Is project mein: PDFs + purani research ChromaDB mein hain, query par top-k chunks retrieve hote hain.

 Interview Question**Q10: “RAG pipeline ke steps batao.”**

Answer: 5 steps: (1) **Ingestion** — PDF load (PyPDFLoader), chunk (RecursiveCharacterTextSplitter 1000/200), embed (all-MiniLM-L6-v2), store (ChromaDB). (2) **Query understanding** — user query ko embed karo. (3) **Retrieval** — cosine similarity, top-k (k=5 docs, k=3 past). (4) **Augmentation** — retrieved chunks + query ko prompt mein rakho. (5) **Generation** — LLM answer. Is project mein `rag/vector_store.py` ye sab karta hai.

 Interview Question**Q11: “Chunk size chhota/ bada hone se kya farak padta hai?”**

Answer: Chhota chunk (200 chars): precise retrieval par context adhura — answer incomplete. Bada chunk (5000): pura context par embedding dilute — irrelevant chunk bhi match ho jata hai. Is project mein 1000 chars + 200 overlap: balance. Overlap important hai kyunki sentence chunk boundary par na kate.

 Interview Question**Q12: “Embedding model kaise choose karte ho?”**

Answer: 5 criteria: (1) **Dimension** — 384 vs 1536: quality vs speed/memory; (2) **Language support** — Hinglish/English docs ke liye multilingual chahiye; (3) **License/cost** — open-source free vs paid API; (4) **Speed** — CPU par chalna chahiye; (5) **Benchmark** — MTEB scores. Is project mein all-MiniLM-L6-v2: 384-dim, 80MB, CPU-fast, free — is scale ke liye perfect.

 Interview Question**Q13: “Similarity search mein cosine similarity kyun, Euclidean kyun nahi?”**

Answer: Cosine **direction** measure karta hai, magnitude nahi. Sentence embeddings mein magnitude (length of text) quality nahi batata — do alag-length texts same meaning ke ho sakte hain. Cosine unhe similar dikhata hai, Euclidean distance unhe alag dikha sakta hai. Is project mein ChromaDB default cosine use karta hai.

 Interview Question**Q14: “Vector database vs normal database (PostgreSQL)?”**

Answer: Normal DB: exact match, range queries, transactions — efficient for structured data. Vector DB: **similarity search** over high-dimensional vectors — HNSW index, cosine distance. PostgreSQL mein pgvector extension se vectors bhi aate hain, par dedicated vector DB

(ChromaDB) lightweight + simple API deta hai. Is project mein SQLite (Django) structured data ke liye, ChromaDB vectors ke liye — **two databases, two jobs**.

Interview Question

Q15: “Retrieval fail ho jaye to kya hoga?”

Answer: Is project mein graceful degradation: (1) ChromaDB unavailable → `try/except` → `rag_context` empty; (2) Tavily key missing → notice message; (3) LLM fail → fallback text. System kabhi crash nahi karta — hamesha best-effort output deta hai. Ye production mindset dikhata hai.

26.4 Multi-Agent Systems

Interview Question

Q16: “Single agent vs multi-agent? Kab multi-agent use karna chahiye?”

Answer: Multi-agent tab jab task ke **alag specializations** hain aur **quality control** chahiye. Is project mein: researcher (search), analyst (synthesis), fact-checker (verification), writer (prose) — har ek apne prompt, temperature, data contract ke saath. Fayde: modular, testable, hallucination control. Nuksan: latency (4+ LLM calls), cost, handoff quality risk — jo is project ke benchmark mein dikha (ek run mein single-agent jeet gaya).

Interview Question

Q17: “Agents aapas mein kaise communicate karte hain?”

Answer: Shared state (blackboard pattern) — LangGraph ka `ResearchState` TypedDict. Har agent state padhta hai, apna output add karta hai, agla agent usse aage badhta hai. File/message-passing nahi — **in-memory shared state**. Communication contract = state schema (`state/schema.py`).

Interview Question

Q18: “Agar ek agent ka output kharab ho jaye?”

Answer: Is project mein 2 mechanisms: (1) **Fact-checker guard** — analyst ka output verify hota hai, fail par revision loop; (2) **try/except fallbacks** — har agent API fail hone par graceful output deta hai. Production mein aage: output validation schema + retry with backoff + human-in-the-loop.

Interview Question

Q19: “Is pipeline ko parallel karna possible hai?”

Answer: Haan — 3 jagah: (1) Researcher ke 3 sub-queries parallel Tavily calls (`asyncio/ThreadPool`); (2) Multiple topics ek saath; (3) Inference batching. LangGraph mein **fan-out/fan-in** edges se parallel nodes add hote hain. Is project mein abhi sequential hai (simplicity), par architecture parallel banane ke liye ready hai.

 Interview Question

Q20: “LangGraph bina use kiye bhi pipeline ban sakti thi. Framework kyun?”

Answer: Haan, manual loop se bhi chalti (maine dono dikhaye hain is document mein). Par LangGraph deta hai: (1) **conditional edges** — revision loop cleanly express hota hai; (2) **streaming** — real-time UI progress bina threading ke; (3) **checkpointing** — state persistence; (4) **visualization/testing** tools. Manual loop mein ye sab khud banana padta.

 Interview Question

Q21: “CrewAI/AutoGen se LangGraph kyon choose kiya?”

Answer: Compare kiya maine: CrewAI role-based hai — aisa loop (verify → revise) force karna awkward. AutoGen conversational hai — research pipeline ke liye less structured. LangGraph **graph-first** hai — mera flow (conditional loop) naturally express hota hai. Choice use-case driven thi, trend-driven nahi.

26.5 LLM APIs & Providers

 Interview Question

Q22: “Groq kya hai? OpenAI se kya difference?”

Answer: Groq ek inference provider hai jo **LPU (Language Processing Unit)** chips par LLMs chalata hai — generation speed bahut fast (hundreds of tokens/sec). OpenAI/ChatGPT API similar models deta hai par cloud GPUs par. Is project mein Groq: fast + cheap (free tier) + OpenAI-compatible API. Cost/latency sensitive demo ke liye acha.

 Interview Question

Q23: “API key ko code mein hardcode kyun nahi kiya?”

Answer: 3 reasons: (1) **Security** — git history mein leak ho jata; (2) **Environment separation** — dev/prod alag keys; (3) **12-factor app** — config code se alag. Is project mein 3-level fallback: OS env → Streamlit secrets → .env file (config/setting.py ka `get_env_var`).

 Interview Question

Q24: “Rate limiting / quota khatam ho jaye to?”

Answer: Is project mein: try/except + fallback outputs — system degraded mode mein chalta hai. Production: retry with exponential backoff, queue, caching, aur multiple provider fallback (Groq → OpenAI). Ye batao ki tum production readiness ke baare mein sochte ho.

27 Q&A Bank Part 2 — Project Specific & System Design

27.1 Streamlit

Interview Question

Q25: “Streamlit kya hai? Rerun model samjhao.”

Answer: Streamlit Python data/AI apps banane ka framework — script har interaction par **top se neeche rerun** hota hai. Isliye state reset na ho iske liye `st.session_state` chahiye — session-scoped dictionary jo reruns ke beech persist karti hai. Is project mein theme, agent statuses, messages, final report sab session state mein hain.

Interview Question

Q26: “`st.session_state` bina kya problem hoti?”

Answer: Har button click par script rerun hoti hai — normal Python variables reset. Agar report session state mein na ho, to har interaction par report gayab. Isliye pattern: lazy init `if key not in st.session_state: st.session_state[key] = default`. Ye Streamlit interviews ka favourite question hai.

Interview Question

Q27: “Streamlit vs Gradio vs React?”

Answer: Streamlit: rapid prototyping, Python-only, built-in widgets — data/AI apps ke liye. Gradio: ML demos ke liye focused (image/model demos). React: production web apps — full control par frontend development chahiye. Is project mein Streamlit isliye: real-time agent progress streaming + zero frontend code + Python ek hi language.

Interview Question

Q28: “Streamlit app production mein?”

Answer: Streamlit Cloud free hosting deta hai — secrets manager ke saath. Limitations: session-based (state server memory mein), single-process. Production scale par: multi-instance + sticky sessions, ya frontend alag. Is project ka architecture isliye interfaces alag rakhta hai — Streamlit sirf ek interface hai, backend logic independent.

27.2 Django REST Framework

Interview Question

Q29: “Django vs Flask vs FastAPI?”

Answer: Django: batteries-included — ORM, admin, auth, migrations built-in — bade structured apps. Flask: minimal, flexible. FastAPI: async, type-hint based, automatic OpenAPI docs — modern APIs. Is project mein Django isliye: (1) ORM + migrations — models quickly; (2) built-in auth (register/login tokens); (3) DRF serializers clean API layer. FastAPI bhi kaam

karta, par Django ka admin/ecosystem is scale ke liye solid.

Interview Question

Q30: “DRF mein serializer kya karta hai?”

Answer: Serializer = data conversion layer: (1) Python objects → JSON (serialization); (2) JSON → validated Python objects (deserialization); (3) **Validation** — required fields, types, custom validators. Is project mein `ResearchSessionSerializer`, `BenchmarkEvaluationSerializer` models ko API responses mein convert karte hain.

Interview Question

Q31: “View vs ViewSet? Is project mein kya use hua aur kyun?”

Answer: `APIView`: manual control — har HTTP method (get/post) khud handle karta hai. `ViewSet`: CRUD boilerplate auto-generated (list/create/retrieve). Is project mein `APIView` use kiya: endpoints custom logic ke hain (register, login, execute research, benchmark) — standard CRUD nahi, isliye explicit control better tha. Interviewer ko ye trade-off batana acha hai.

Interview Question

Q32: “Auth kaise implement kiya? JWT kyun nahi?”

Answer: Is project mein simple token auth: register/login par random token generate karke user se link — requests mein `Authorization: Token <token>` header. JWT: stateless (signature-verified), multi-service scaling ke liye. Simple token: stateful (DB lookup), logout turant, implementation simple. Is project ke scale par simple token sufficient tha — production multi-service par JWT better.

Interview Question

Q33: “Migrations kya hote hain?”

Answer: Migrations = DB schema changes ka version control. `models.py` mein changes → `makemigrations` (change file banata) → `migrate` (DB apply). Is project mein `0001_initial` migration hai — `ResearchSession` + `BenchmarkEvaluation` tables. Isse schema changes trackable aur reversible hain.

Interview Question

Q34: “SQLite vs PostgreSQL?”

Answer: SQLite: file-based, zero-config, single-user — dev/demo ke liye perfect. PostgreSQL: server-based, concurrent users, features (JSONB, full-text) — production. Is project mein SQLite (dev) + Django ORM ka matlab hai ki production par connection string badalna enough hai — **ORM abstraction** ka fayda.

27.3 Testing

 Interview Question

Q35: “Testing kyun zaroori? Kya test kiya is project mein?”

Answer: Regression protection + documentation + confidence. Is project mein 16 pytest tests: auth (register/login/failures), research execution (validation, persistence, 500 paths), benchmark endpoint + history. **Mocking** se real API calls test mein nahi hote — fast, free, deterministic.

 Interview Question

Q36: “Mocking kya hai? pytest-django se kya mila?”

Answer: Mocking = external dependencies ko fake se replace karna (LLM calls, search, vector store). pytest-django: test DB (real DB untouched), test client (API calls bina server), fixtures (setup/teardown). Is project mein `mock_research_graph` fixture LangGraph ko stub se replace karta hai — canned state return.

 Interview Question

Q37: “Test DB real data kyun nahi chhuta?”

Answer: pytest-django har test ke liye **throwaway test database** banata hai (transactions rollback) — real `db.sqlite3` untouched rehta hai. Tests isliye safe hain chhalao kabhi bhi. Ye “isolated test environment” principle hai.

27.4 Benchmarking & Evaluation

 Interview Question

Q38: “Benchmark kya measure karta hai is project mein?”

Answer: Do dimensions: **Depth** (comprehensiveness, detail, technical accuracy) aur **Verifiability** (citations, facts vs opinions). Single-agent baseline (1 LLM call) vs multi-agent pipeline (4 nodes) — same topic, same judge, side-by-side. LLM-as-judge + heuristic fallback dono.

 Interview Question

Q39: “LLM-as-judge ki problems?”

Answer: (1) **Bias** — order bias (pehla report favor), position bias, self-preference (same model judge apne jaise output pasand karta hai); (2) **Inconsistency** — same input alag scores; (3) **Expense**. Mitigation: side-by-side scoring, multiple judges, heuristic cross-check. Is project mein heuristic fallback hai — judge fail hone par regex-based scoring.

 Interview Question

Q40: “Is benchmark ka result kya nikla?”

Answer: Ek run mein **SINGLE_AGENT_SUPERIOR** aaya (depth 9 vs 8, verifiability 8 vs 7) — Writer ko analyst ka excerpt-heavy output mila tha. Par ek run statistically meaningless hai — pehla end-to-end run (solid-state) mein multi-agent ne behtareen report banayi. Ye hi evaluation ka point hai: **measure, don’t assume**.

27.5 System Design

Interview Question

Q41: “Is system ko 100 concurrent users ke liye kya change karna padega?”

Answer: (1) **SQLite** → **PostgreSQL** — concurrent writes; (2) **Sync pipeline** → **async** — 176s ka request thread block nahi karega; task queue (Celery/Redis) + polling/webhook; (3) **Streamlit** → **multi-instance** + load balancer + sticky sessions ya stateless frontend; (4) **Local ChromaDB** → **managed vector DB**; (5) Rate limiting + caching (same topics ka result cache).

Interview Question

Q42: “Is system mein bottleneck kya hai?”

Answer: 3 bottlenecks: (1) **Pipeline latency** — 4+ sequential LLM calls + Tavily + embedding = 176s end-to-end; (2) **Embedding** — sentence-transformers CPU par, ingestion slow; (3) **LLM rate limits** — concurrent users par Groq quota. Fixes: parallel sub-queries, batch embedding, async architecture.

Interview Question

Q43: “Data flow diagram banao is system ka.”

Answer: (Board par draw karo) User/Client → Interface (Streamlit/MCP/Django) → `research_graph.invoke/stream` → nodes: researcher (Tavily + RAG) → analyst → `fact_checker` → [loop] → writer → `final_report` → UI + ChromaDB store + Django DB. Har node state update karta hai, conditional edge fact-check par decide karta hai.

Interview Question

Q44: “Security concerns is project mein?”

Answer: (1) **API keys** — .env gitignored, 3-level fallback; (2) **Prompt injection** — untrusted sources; (3) **CORS** — Django mein `django-cors-headers`, allowlist needed; (4) **Input validation** — serializer required fields; (5) **Rate limiting** — production mein add karna. Security interview mein in points ko bolna hi enough hai.

27.6 Behavioral & Project Story

Interview Question

Q45: “Tell me about this project (30 sec elevator pitch).”

Answer: “Maine ek **multi-agent research orchestration system** banaya hai jo kisi bhi topic par deep, verified research reports generate karta hai. Architecture: LangGraph-based pipeline — researcher web search karta hai (Tavily), analyst synthesis, fact-checker claims verify karta hai aur revision loop chala sakta hai, writer final report likhta hai. Teen interfaces: Streamlit UI (real-time streaming), MCP server (AI assistants connect karein), aur Django REST API. RAG memory: PDFs + past research ChromaDB mein. Aur maine ek benchmark suite bhi banaya hai jo single vs multi-agent quality measure karta hai — depth aur verifiability par.” (30 sec mein ye perfect hai)

 Interview Question**Q46: “Sabse difficult problem kya thi?”**

Answer: (STAR format) **Situation:** Multi-agent pipeline benchmark mein haar gayi single-agent se. **Task:** Pata karo kyun aur fix karo. **Action:** Outputs compare kiye — writer ko analyst ka raw excerpt-heavy brief mila tha, synthesis nahi. Root cause: handoff quality. **Result:** Fix proposals — analyst few-shot + writer synthesis instructions + multi-topic benchmark. Ye dikhata hai ki main measure karta hoon aur root cause analysis karta hoon.

 Interview Question**Q47: “Agar dobara banate to kya alag karte?”**

Answer: (1) **Evaluation-first** — pehle benchmark infra, phir features; (2) **Async pipeline** — synchronous 176s se task queue; (3) **Structured agent outputs** — pydantic schemas instead of free text; (4) **Tracing** — LangSmith/Langfuse observability; (5) **Versioned prompts** — prompt registry + A/B testing.

 Interview Question**Q48: “Team mein kaise kaam karte? Conflicts kaise handle?”**

Answer: (Project context) Is project mein code review habits: modules alag (agents/, rag/, graph/) isliye merge conflicts kam. Technical decisions: benchmark se data lekar decision (model choice, architecture) — **data-driven disagreements**. Real team story bhi ready rakho: college projects/competitions se.

28 Whiteboard Problems — Is Project Se Nikale Gaye

 Kahani Se Samjho

Interviewers is project ko dekh ke inme se koi bhi whiteboard problem puch sakte hain. Har problem ka **approach** + **is project se connection** neech hai. Pattern yaad rakho: clarify → design → trade-offs → code.

28.1 Problem 1: Design a RAG System

 Interview Question

Problem: “Ek RAG system design karo jo 10,000 PDFs index kare aur real-time queries answer kare.”

Approach:

1. **Clarify:** Data type (PDFs), scale (10k), latency (real-time), languages.
2. **Ingestion pipeline:** PDF parser → chunking (recursive, 500-1000 chars, overlap) → embedding (batch) → vector store (Pinecone/Qdrant) + metadata store (PostgreSQL).
3. **Query pipeline:** Query → embedding → hybrid search (vector + keyword) → re-ranking (cross-encoder) → LLM with citations.
4. **Evaluation:** Retrieval quality (recall@k), answer quality (LLM judge), latency SLO.

Project connection: “Is project mein simplified version banaya — ChromaDB + cosine

search, $k=5/3$. Production version mein hybrid search + re-ranking add karta.”

28.2 Problem 2: Design a Multi-Agent Orchestrator

Interview Question

Problem: “Ek system design karo jisme 5 AI agents milkar complex task karein.”

Approach:

1. **State contract** — shared schema (topic, research_data, analysis, verdict, report).
2. **Graph topology** — nodes, edges, conditional edges; loop guard (max iterations).
3. **Handoff quality** — har agent ka output schema-validated.
4. **Failure handling** — retries, fallbacks, degraded mode.
5. **Observability** — tracing, per-node latency, cost tracking.

Project connection: “Exactly ye architecture maine LangGraph mein banaya — 4 nodes, conditional fact-check loop, MAX_REVISIONS guard. Failure handling try/except se.”

28.3 Problem 3: Rate Limiting Design

Interview Question

Problem: “LLM API calls ko rate limit karo — 1000 users, 10 calls/sec budget.”

Approach: Token bucket (smooth burst control) ya fixed window (simple) — Redis counter, per-user + global buckets, queue with retry/backoff, 429 responses with Retry-After. Priority: paid users vs free.

Project connection: “Is project mein abhi simple try/except fallback hai; production mein Redis-based token bucket add karta.”

Part III

Reference — API, Testing, Deployment

29 Django REST API Reference — Har Endpoint Ka Pura Detail

29.1 Endpoint Map

Method	Endpoint	Kaam
GET	/api/health/	Server + pipeline readiness check
POST	/api/auth/register/	Naya user + token
POST	/api/auth/login/	Existing user se token
POST	/api/research/execute/	Full research pipeline chalao
POST	/api/research/benchmark/	Single vs multi-agent benchmark
GET	/api/benchmark/history/	Past benchmark evaluations (limit param)

29.2 GET /api/health/

Code Samjho Line-by-Line

Request: No body. **Response 200:**

```
1 {  
2   "status": "ok",  
3   "pipeline": "ready",  
4   "model": "openai/gpt-oss-120b"  
5 }
```

Kya karta hai: Django server + pipeline imports check karta hai. Streamlit app isi se backend reachability decide karta hai (Benchmark Lab ka “Backend API reachable” banner).

29.3 POST /api/auth/register/

Code Samjho Line-by-Line

Request body:

```
1 {"username": "rahul", "password": "secret123"}
```

Response 201:

```
1 {"id": 1, "username": "rahul", "token": "abc123..."}
```

Failures:

- 400 — missing fields ya duplicate username (serializer validation).
- Password hashed hota hai (`set_password`) — plain text kabhi store nahi hota.

29.4 POST /api/auth/login/

Code Samjho Line-by-Line

Request body:

```
1 {"username": "rahul", "password": "secret123"}
```

Response 200:

```
1 {"token": "abc123...", "username": "rahul"}
```

Failures: 401 — galat password ya user exist nahi karta (`authenticate` check). Response mein 401 deliberately generic hai — user enumeration prevent karta hai.

29.5 POST /api/research/execute/

Code Samjho Line-by-Line

Request body:

```
1 {"topic": "solid state battery market outlook 2026"}
```

Response 200 (pipeline ke baad):

```
1 {
2   "id": 1,
3   "topic": "solid state battery market outlook 2026",
4   "report": "# Research Report: ... (full markdown)",
5   "messages": ["\U0001f50d Researcher found 15 web results...",
6               "\U0001f4ca Analyst processed 15 results.",
7               "\u2705 Fact-check PASSED (revision 0/3)",
8               "\u270d\ufe0f Report complete"],
9   "sources_count": 15,
10  "revision_count": 0,
11  "created_at": "2026-08-17T12:00:00Z"
12 }
```

Validation: Missing topic → 400. **Error path:** Pipeline exception → 500, no row persisted (transaction rollback) — half-saved sessions nahi.

29.6 POST /api/research/benchmark/

Code Samjho Line-by-Line

Request body:

```
1 {"topic": "Best practices for RAG in 2026"}
```

Response 200:

```
1 {
2   "id": 1,
3   "topic": "Best practices for RAG in 2026",
4   "single_agent_report": "...",
5   "multi_agent_report": "...",
6   "single_agent_depth": 9,
7   "multi_agent_depth": 8,
8   "single_agent_verifiability": 8,
9   "multi_agent_verifiability": 7,
10  "verdict": "SINGLE_AGENT_SUPERIOR",
11  "created_at": "2026-08-17T12:00:00Z"
12 }
```

Verdict logic: Multi-agent depth+verifiability sum > single-agent sum → MULTI_AGENT_SUPERIOR; close margin → TIE; warna SINGLE_AGENT_SUPERIOR.

29.7 GET /api/benchmark/history/

Code Samjho Line-by-Line

Query params: ?limit=10 (default 20, max 500).

Response 200:

```
1 [
2   {"id": 1, "topic": "...", "single_agent_depth": 9, "...": "..."},
3   {"id": 2, "...": "..."}
4 ]
```

Ordering: Newest first (-created_at, -id tiebreak — same-timestamp rows deterministic). Empty history → []. Benchmark Lab page isi se charts banata hai.

⚠ Dhyan Do

Interview note on API design: (1) Nested resources (/api/research/...) logical grouping ke liye; (2) /api/benchmark/history/ alag endpoint isliye kyunki history **read-only collection** hai — benchmark POST se alag concern; (3) Response fields consistent snake_case; (4) Errors: proper status codes (400/401/500) + DRF default error body. Ye REST best practices ke examples hain.

30 Testing Deep Dive — 16 Tests Ka Khoj

30.1 Test Infrastructure

🔗 Code Samjho Line-by-Line

pytest.ini:

```
[pytest]
DJANGO_SETTINGS_MODULE = orchestrator_backend.settings
pythonpath = backend
testpaths = backend
```

Kya karta hai: (1) Django settings point karta hai; (2) backend/ ko sys.path mein daalta hai taaki from api.models import ... chale; (3) sirf backend/ mein tests dhoondta hai.

30.2 Mocking Strategy — Real Calls Kabhi Nahin

🧠 Concept Deep Dive

Kyun mock? Real Tavily/Groq/ChromaDB calls tests ko: slow (seconds-minutes), paid (API credits), non-deterministic (LLM output varies), aur flaky (network) banate hain. Mock se: fast (~9s), free, deterministic, reliable.

Kaise mock kiya:

- `mock_research_graph` fixture: `graph.workflow.research_graph` ko ek stub object se replace — `invoke()` canned state return karta hai.
- `exploding_research_graph`: exception raise karta hai — 500 error paths test karne ke liye.
- `mock_benchmark`: `run_single_agent_baseline + evaluate_outputs` ko stub.

30.3 Test Case Analysis

File	Kya test karta hai	Key assertion
test_auth.py (6)	register success/fail, login success/fail	201+token, 400 dup, 401 wrong pass
test_research.py (4)	execute validation/success/error	400 no topic, session persisted, 500 no row
test_benchmark.py (6)	benchmark + history + limit	scores persisted, newest first, empty list

👤 Interview Question

Interesting bug jo tests se mila: History ordering test mein do rows ek hi microsecond mein bane (Django test env time freeze karta hai) — `created_at` tie ho gaya aur ordering nondeterministic. Fix: order by `-created_at`, `-id`. Ye asli robustness win tha — sirf test ke liye nahi, production ordering ke liye bhi.

30.4 How To Run

Code Samjho Line-by-Line

```
# from project root (venv active)
pytest                # full suite, ~9 seconds

# specific file
pytest backend/api/tests/test_auth.py

# specific test with output
pytest -v backend/api/tests/test_benchmark.py
```

Interview point: Test suite **fast** hone se developers baar baar chalte hain — 9s suite = daily use. Slow suite (minutes) = kabhi nahi chalta.

31 Deployment — Demo Se Production Tak

31.1 Local Setup (Interview Demo)

Code Samjho Line-by-Line

```
# 1. Environment
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt

# 2. Keys (.env ya Streamlit secrets)
GROQ_API_KEY=...
TAVILY_API_KEY=...

# 3. Django backend (port 8001)
cd backend
python manage.py migrate
python manage.py runserver 8001

# 4. Streamlit UI (alag terminal, port 8501)
streamlit run app.py

# 5. MCP server (Claude Desktop ke liye)
python mcp_server.py
```

Environment separation: Streamlit secrets (cloud) vs .env (local) — config/setting.py dono try karta hai.

31.2 Streamlit Cloud

Concept Deep Dive

Steps: (1) Repo GitHub par push; (2) Streamlit Cloud → New app → repo select; (3) **Secrets** mein GROQ_API_KEY, TAVILY_API_KEY daalo; (4) Deploy. **Caveat:** Streamlit Cloud ephemeral storage hai — ChromaDB aur SQLite har session par reset ho sakte hain. Production: managed vector DB + PostgreSQL.

31.3 Django Production

Concept Deep Dive

Production checklist: (1) DEBUG=False; (2) PostgreSQL; (3) Gunicorn/uvicorn + Nginx reverse proxy; (4) HTTPS (Let's Encrypt); (5) ALLOWED_HOSTS; (6) CORS allowlist; (7) Rate limiting; (8) Secrets via env, kabhi code mein nahi.

31.4 MCP Server for Claude Desktop

Code Samjho Line-by-Line

```
1 // claude_desktop_config.json
2 {
3   "mcpServers": {
4     "research-orchestrator": {
5       "command": "python",
6       "args": ["D:/path/to/mcp_server.py"]
7     }
8   }
9 }
```

Fix yaad rakho: fastmcp 3.x mein FastMCP(description=...) rename hua instructions=... — version mismatch par ye error aata hai. (Maine is project mein ye fix kiya.)

31.5 Docker (Optional)

Code Samjho Line-by-Line

```
# Dockerfile (streamlit service)
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
EXPOSE 8501
CMD ["streamlit", "run", "app.py"]
```

Compose mein 3 services: **streamlit** (8501), **backend** (8001), **chromadb**. Volumes: ChromaDB persist directory + SQLite.

Part IV

Playbook, Glossary & Cheat Sheets

32 Extension Playbook — Naya Feature Kaise Add Karein

32.1 Add a New Agent

Code Samjho Line-by-Line

Example: “Critic” agent jo final report ka review kare. 5 steps:

1. **State** (state/schema.py): `critic_review`: str field add karo.
2. **Agent** (agents/critic.py): node function + `SYSTEM_PROMPT` + `get_llm` — same pattern as writer.
3. **Graph** (graph/workflow.py): `add_node("critic", critic_node)` + edges: writer → critic → END.
4. **UI** (app.py): statuses dict mein critic add karo.
5. **Tests**: mock graph ke saath test.

Design principle: Agent add karna = 4 files touch karna (state, agent, graph, UI) — modules independent hain isliye tight coupling nahi.

32.2 Add a New API Endpoint

Code Samjho Line-by-Line

Example: GET `/api/research/history/` — past sessions list. Steps:

1. **Serializer** (serializers.py): `ResearchSessionSerializer` already hai.
2. **View** (views.py): `ResearchHistoryAPIView(APIView)` — GET mein `queryset` + `serializer`.
3. **URL** (urls.py): `path("research/history/", ...)`.
4. **Test** (tests/test_research.py): GET returns sessions newest-first.

32.3 Add a New Vector Collection

Code Samjho Line-by-Line

```
1 # rag/vector_store.py mein
2 def get_collection_stats() → dict:
3     for name in ["research_docs", "past_research", "new_collection"]:
4         ...
```

Listing 31: Naya collection example

Steps: (1) Collection name constant add; (2) stats loop mein add; (3) store/retrieve functions mein optional param. ChromaDB collections isolated hain — cross-contamination nahi.

33 Glossary — 60+ Terms Hinglish Mein

Term	Simple Explanation
LLM	Large Language Model — text generate karne wala neural network
Token	Text ki smallest processing unit (word ka part)
Temperature	Output randomness control (0=deterministic, 1=creative)
Context window	Model ek baar mein kitna text dekh sakta hai
Hallucination	Model ka factual galat par confident output
Prompt	Model ko diya gaya instruction/task
System prompt	Model ki role/behavior define karne wala prompt
Few-shot	Examples ke saath prompting
RAG	Retrieve-then-generate — sources se answer
Embedding	Text ka numerical vector (semantic fingerprint)
Chunking	Bade text ko chhote parts mein katna
Chunk overlap	Adjacent chunks ka shared content
Cosine similarity	Do vectors ke beech direction similarity
Vector database	Similarity search ke liye optimized store
HNSW	Graph-based approximate nearest neighbor index
Agent	LLM-powered unit with role + tools + memory
Multi-agent	Multiple specialized agents ka system
State	Pipeline ki shared memory (data contract)
Node	Graph mein ek processing step
Edge	Nodes ke beech connection
Conditional edge	Runtime par decide hone wala edge
Reducer	State update ka merge rule (add_messages)
LangGraph	Graph-based agent orchestration framework
TypedDict	Type-annotated dictionary (Python)
Tavily	Web search API for LLMs
Groq	Fast LLM inference provider (LPU chips)
ChromaDB	Open-source vector database
MCP	Model Context Protocol — AI tools ka universal port
FastMCP	MCP server banane ka Python framework
Tool (MCP)	AI assistant call kar sakta hai (function)
Resource (MCP)	AI assistant read kar sakta hai (data)
Streamlit	Python data/AI app framework
Session state	Streamlit reruns ke beech persist hone wala state
Django	Python web framework (batteries included)
DRF	Django REST Framework — API layer
Serializer	Data conversion + validation layer
View	HTTP request handler
Migration	DB schema change ka version control
ORM	Object-Relational Mapping — DB as Python objects
Token auth	Token-based authentication (stateless per request)
pytest	Python testing framework

Fixture	Test setup/teardown helper
Mock	External dependency ka fake replacement
pytest-django	Django testing integration for pytest
Benchmark	Standardized quality measurement
LLM-as-judge	LLM se outputs score karwana
Verdict	Benchmark ka outcome (superior/tie)
SQLite	File-based embedded database
PostgreSQL	Production-grade relational database
CORS	Cross-Origin Resource Sharing policy
APIView	DRF class-based view
Endpoint	API ka ek specific URL+method
JSON	Data exchange format
Markdown	Lightweight text formatting
Hinglish	Hindi+English mixed language
PyPDFLoader	PDF text extraction (langchain)
RecursiveCharacterTextSplitter	Smart text chunker
HuggingFaceEmbeddings	HuggingFace embedding model wrapper
Singleton	Ek hi instance poori app mein
Fan-out	Parallel node execution (graph)
Checkpointing	Graph state persistence
Graceful degradation	Partial failure par bhi system chalta rahe
Fail-fast	Pehle failure par turant rukna
12-factor app	Cloud app best practices
Rate limiting	Request frequency control
Latency	Request-response time
Throughput	Time unit mein processed requests

34 Comparison Tables — Ek Nazar Mein

34.1 Framework Comparisons

Criteria	LangGraph	CrewAI	AutoGen
Core model	State graph	Role crews	Conversations
Control flow	Explicit	Task-based	Chat-based
Loop support	Built-in	Manual	Manual
Streaming	Yes (.stream)	Limited	Yes
Visualization	Yes	No	No
Best for	Complex pipelines	Simple crews	Agent chats

Criteria	Streamlit	Gradio	React
Language	Python only	Python only	JS/TS
Speed to build	Very fast	Fast	Slow
Custom UI	Limited	Limited	Full
Real-time	Native	Native	Manual
Best for	Data/AI apps	ML demos	Production web

Criteria	Django	Flask	FastAPI
ORM + admin	Built-in	None	None
Auth	Built-in	Manual	Manual
Async	Manual	No	Native
Migrations	Built-in	None	None
Best for	Full apps	Micro services	High-perf APIs

34.2 Vector Store Comparisons

Criteria	ChromaDB	FAISS	Pinecone
Hosting	Local	Local	Managed
Setup	1 line	Medium	Account needed
Persistence	Built-in	Manual	Built-in
Scale	Small-medium	Large	Large
Cost	Free	Free	Paid
Best for	Prototypes	Research	Production

34.3 Model Comparisons (Groq Account)

Model	Params	Notes	Used
openai/gpt-oss-120b	120B	Clean output, fast	YES (current)
openai/gpt-oss-20b	20B	Faster, less depth	Fallback option
qwen/qwen3.6-27b	27B	Reasoning model, think blocks	Rejected (prose pollution)
meta-llama/llama-prompt-guard	Small	Safety classifier only	Not usable for reports

35 Cheat Sheets — Quick Reference

35.1 LangGraph Cheat Sheet

Code Samjho Line-by-Line

```

1 from langgraph.graph import StateGraph, START, END
2 from typing import TypedDict, Annotated
3 from langgraph.graph.message import add_messages
4
5 class State(TypedDict):
6     topic: str
7     messages: Annotated[list, add_messages]
8
9 g = StateGraph(State)
10 g.add_node("node_a", func_a)

```

```

11 g.add_node("node_b", func_b)
12 g.add_edge(START, "node_a")
13 g.add_edge("node_a", "node_b")
14 g.add_edge("node_b", END)
15 # conditional
16 g.add_conditional_edges("node_b", route_fn,
17     {"go_back": "node_a", "finish": END})
18 app = g.compile()
19 result = app.invoke({"topic": "x"})
20 for event in app.stream(state):
21     ...

```

Listing 32: LangGraph essentials

35.2 DRF Cheat Sheet

Code Samjho Line-by-Line

```

1 from rest_framework.views import APIView
2 from rest_framework.response import Response
3 from rest_framework import status
4 from rest_framework.permissions import IsAuthenticated
5
6 class MyView(APIView):
7     permission_classes = [IsAuthenticated]
8     def get(self, request):
9         data = {"ok": True}
10        return Response(data, status=status.HTTP_200_OK)
11    def post(self, request):
12        serializer = MySerializer(data=request.data)
13        if serializer.is_valid():
14            obj = serializer.save()
15            return Response(serializer.data, status=status.
16                HTTP_201_CREATED)
17        return Response(serializer.errors, status=status.
18            HTTP_400_BAD_REQUEST)

```

Listing 33: DRF essentials

35.3 Streamlit Cheat Sheet

Code Samjho Line-by-Line

```

1 import streamlit as st
2
3 # session state
4 if "theme" not in st.session_state:
5     st.session_state.theme = "light"
6
7 # layout
8 st.set_page_config(page_title="App", layout="wide")

```

```

9  col1, col2 = st.columns(2)
10 with col1:
11     st.metric("Depth", 9)
12 with col2:
13     st.bar_chart(df)
14
15 # widgets
16 topic = st.text_input("Topic")
17 if st.button("Run"):
18     with st.spinner("Working..."):
19         result = pipeline(topic)
20 st.download_button("Download", result, file_name="report.md")

```

Listing 34: Streamlit essentials

35.4 pytest Cheat Sheet

🔗 Code Samjho Line-by-Line

```

1  import pytest
2  from unittest.mock import patch
3
4  @pytest.fixture
5  def mock_graph():
6      with patch("graph.workflow.research_graph") as m:
7          m.invoke.return_value = {"final_report": "..."}
8          yield m
9
10 def test_success(mock_graph):
11     resp = client.post("/api/research/execute/",
12                       {"topic": "AI"}, format="json")
13     assert resp.status_code == 200
14
15 def test_error():
16     with pytest.raises(ValueError):
17         risky_function()

```

Listing 35: pytest essentials

36 Final Summary — Poora Project 10 Points Mein

⚙️ System Design Deep Dive

10 key takeaways (interview closing ke liye):

1. **Multi-agent pipeline** — LangGraph: researcher → analyst → fact_checker → writer, conditional revision loop.
2. **Fact-checking** — closed-world verification + MAX_REVISIONS guard.
3. **RAG memory** — PDFs + past research ChromaDB mein, cosine retrieval.
4. **3 interfaces** — Streamlit (human), MCP (AI), Django REST (programmatic) — ek

compiled graph, multiple adapters.

5. **Config discipline** — 3-level secret fallback, gitignored .env, config/setting.py central.
6. **Testing** — 16 pytest tests, full mocking, ~9s suite.
7. **Benchmarking** — single vs multi-agent, depth + verifiability, LLM judge + heuristic fallback.
8. **Real runs** — end-to-end execution verified: HTTP 200, 15 sources, first-pass fact-check pass.
9. **Honest evaluation** — benchmark mein multi-agent haar bhi gayi ek run mein — root cause analyzed, fixes proposed.
10. **Production thinking** — graceful degradation, scaling analysis, deployment plan.

Kahani Se Samjho

Last line (interviewers ko yaad rehti hai): “Is project mein maine sirf code nahi likha — maine ek **system** banaya: agents jo verify karte hain, memory jo yaad rakhti hai, interfaces jo har client ke liye khule hain, aur benchmarks jo measure karte hain ki system actually acha hai ya nahi. Aur sabse important: maine apne system ko **fail** hote dekha, root cause nikala, aur fix propose kiya. Ye hi engineering hai.”

All the best! Ye document aapka companion hai — concepts, code, aur conversations teeno ready hain. Ab jaake confidently bolo. **Jai Hind!** 👍

Part V

Theory Deep Dive — Interview Gold

37 Transformer Architecture — LLM Ki Jaan

 Kahani Se Samjho

Interviewer puchta hai: “LLM ke andar kya hai?” — is section mein transformer ka pura breakdown hai: attention, embeddings, layers, training. **Bina transformer samjhe LLM interview answer adhura hai.** Is project ke models (GPT-OSS-120B) transformer-based hain — isliye ye theory directly relevant hai.

37.1 Transformer Kya Hai — 2017 Ka Revolution

 Concept Deep Dive

Transformer: “**Attention Is All You Need**” paper (Vaswani et al., 2017) se aaya architecture. Pehle RNNs/LSTMs text process karte the sequentially (word-by-word) — slow, aur long-range dependencies bhool jate the. Transformer ka breakthrough: **saare words ko parallel process karo**, aur **attention** se words ke beech relationships capture karo.

2 core ideas:

1. **Self-attention** — har word ko baaki sab words se context milta hai.
2. **Parallelization** — training bahut fast (isliye scale possible hua).

37.2 Embedding Layer — Words Se Numbers

 Maths Behind the Scene

Har token ko ek vector mile: $w_i \in \mathbb{R}^d$ jahan d = embedding dimension (GPT-OSS class models mein 4096+).

Positional encoding: Transformer ko word ka order pata hona chahiye (“dog bites man” vs “man bites dog”). Position pos ke liye:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Ye sine/cosine pattern model ko relative positions samajhne deta hai.

37.3 Self-Attention — The Heart

√* Maths Behind the Scene

Scaled Dot-Product Attention:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Step-by-step:

1. Har token ke liye 3 vectors: **Query** (main poochta hoon), **Key** (tum kya ho), **Value** (tum kya dete ho).
2. $Q \cdot K^T$: har token ka har token se **similarity score**.
3. $\sqrt{d_k}$ se divide: scores stable (large dims mein softmax saturation se bachao).
4. Softmax: scores $\rightarrow$ probabilities (kitna attention kis par).
5. Probabilities $\times$ Values: weighted sum = token ka context-aware representation.

Intuition: Sentence “The cat sat on the mat because it was tired” mein **it** ko **cat** se connect karne ke liye attention hi mechanism hai.

37.4 Multi-Head Attention

🧠 Concept Deep Dive

Ek attention head ek **relationship type** capture karta hai (syntax, co-reference, sentiment). Multi-head: **8-64 heads parallel** — har head alag relationship family. Outputs concatenate hote hain.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

Is project ke GPT-OSS-120B mein ye multi-head mechanism hi har agent prompt ka context samajhta hai.

37.5 Transformer Block Stack

🧠 Concept Deep Dive

Har block (layer) mein:

1. Multi-head self-attention
2. Residual connection (skip) + LayerNorm
3. Feed-forward network (2 linear layers + ReLU/GELU)
4. Residual connection + LayerNorm

GPT-OSS-120B mein **dozens of such blocks stacked** hain. Depth = capability: deeper model complex patterns seekhta hai. Output layer: next-token probability distribution over vocabulary (~100K tokens).

👤 Interview Question

Q: “GPT ka ‘Generative Pre-trained Transformer’ kya matlab hai?”

Answer: (1) **Generative** — next token predict karta hai; (2) **Pre-trained** — pehle massive

text par self-supervised training; (3) **Transformer** — architecture. Training 2-stage: pre-training (next-word prediction, trillions tokens) → fine-tuning (instruction following, RLHF). Is project ka model inference par chalata hai — trained model se sirf generate karte hain.

37.6 Training vs Inference

Aspect	Training	Inference
Goal	Learn weights	Generate output
Data	Trillions tokens	User prompt
Compute	GPUs, weeks-months	Single forward pass
Cost	Millions USD	Cents per request
Updates	Backpropagation	No updates
Is project mein?	Provider side (Groq)	Yes — every API call

Memory Trick

Interview pearl: “Aapka project LLM **train** nahi karta — trained model ko **use** karta hai (inference via API). Model chunna, prompt design karna, aur evaluation karna — ye humara kaam hai. Fine-tuning tab chahiye jab domain-specific style/format chahiye jo prompting se nahi aata.”

38 Advanced RAG — Naive Se Production Tak

38.1 Naive RAG ki Problems

Concept Deep Dive

Naive RAG (retrieve → stuff → generate) ki 4 problems:

1. **Retrieval quality** — top-k chunks irrelevant ho sakte hain (query phrasing par sensitive).
2. **Lost in the middle** — LLM context ke middle ko ignore karta hai (position bias).
3. **Chunk boundaries** — context beech mein kat sakta hai.
4. **No evaluation** — retrieval/answer quality measured nahi.

38.2 Advanced RAG Techniques

Technique	Kya karta hai
Query rewriting	LLM se query ko expand/compress karna (is project: re-searcher ke 3 sub-queries)
Hybrid search	Vector + BM25 keyword search combine (dense + sparse)
Re-ranking	Cross-encoder se top-50 → top-5 refine
Context compression	LLM se retrieved chunks ko compress (LLMLingua)

Parent-child chunks	Chhote chunks retrieve, bade parent chunks LLM ko do
Metadata filtering	Date/source/type filters pehle se
RAPTOR	Hierarchical summaries build karke multi-level retrieve
Self-RAG	Model khud decide kare: retrieve karna hai ya nahi

Interview Question

Q: “Is project mein kaunsa advanced technique use hua?”

Answer: Query expansion (researcher 3 diverse sub-queries banata hai — naive single-query ka improved version) aur **multi-source retrieval** (Tavily web + research_docs + past_research — three diverse corpora). Production version mein main re-ranking (cross-encoder) aur hybrid search add karta. Ye progression batana strong answer deta hai.

38.3 RAG Evaluation

Concept Deep Dive

2-layer evaluation:

1. **Retrieval quality:** Recall@k, MRR — “kya relevant chunks top-k mein aaye?”
2. **Generation quality:** Faithfulness (answer sources se hai?), Relevance, Hallucination rate.

Is project ka benchmark **verifiability** dimension exactly generation-layer check karta hai — citations + facts-vs-opinions. Interview mein ye connection batao: “benchmark ki verifiability = RAG faithfulness ka proxy.”

39 Debugging Session — Ek Real Kahani

Kahani Se Samjho

Ye section ek **real debugging session** ki kahani hai jo maine is project par ki — sab kuch sach hai jo is document mein likha hai. Interview mein story-telling format mein answer dena (STAR) powerful hota hai — ye ek ready-made example hai.

39.1 Bug 1: MCP Server Crash on Startup

Interview Question

Symptom: `mcp_server.py` import karte hi crash: `FastMCP() got an unexpected keyword argument 'description'`.

Diagnosis: `fastmcp` library version 3.4.6 mein `description` parameter rename hua `instructions`. Code purane version ke liye likha tha — dependency update ke baad toot gaya.

Fix: `FastMCP(description=...) → FastMCP(instructions=...)`. Verified: sab 5 tools + resource register hote hain.

Lesson: Pin dependencies (`requirements.txt` mein versions) + upgrade ke baad smoke test. Ye

“dependency drift” problem har production system mein aati hai.

39.2 Bug 2: Model 404 on Groq

Interview Question

Symptom: Pipeline mein har LLM call 404 error — `llama-3.3-70b-versatile` not found on account.

Diagnosis: Groq account par wo model available hi nahi tha. Mainne `GET /v1/models` call karke asli available models dekhe — sirf prompt-guard classifiers + openai/gpt-oss + qwen models mile.

Fix: Live tests kiye teen models par — `gpt-oss-120b` ne clean output diya (qwen reasoning model `<think>` blocks deta tha jo report prose kharab karta). `MODEL_NAME=openai/gpt-oss-120b` set kiya config + `.env` + `README` + notes mein.

Lesson: “Documented model” ka matlab available model nahi — hamesha live API se verify karo. Agents ki graceful fallback ne system ko crash hone se bachaya.

39.3 Bug 3: History Ordering Nondeterministic

Interview Question

Symptom: Test fail — do benchmark rows ek hi timestamp ke saath, ordering flaky.

Diagnosis: Django test env **time freezes** — dono rows ka `created_at` identical, `order_by("-created_at")` tie par nondeterministic.

Fix: `order_by("-created_at", "-id")` — deterministic tiebreak.

Lesson: Production mein bhi same-timestamp rows possible hain (bulk inserts) — **deterministic ordering** hamesha secondary key rakho.

39.4 Bug 4: venv Silent Fallback

Interview Question

Symptom: Django No module named `'django'` jabki `manage.py` pehle chalta tha.

Diagnosis: Git Bash mein source `.venv/bin/activate` (Linux-style path) Windows venv par **silently fail** karta hai — Python global se chalta tha. Naye environment mein `django` missing tha.

Fix: `.venv/Scripts/python.exe` use karo + `full requirements.txt` install.

Lesson: “Aur environment setup documentation likho” — ye exact issue `run.md` mein documented hai. Cross-platform tooling ki path differences real problems deti hain.

39.5 Bug 5: LaTeX Caption Underscores

 Interview Question

Symptom: Ye notes document compile karte waqt `Missing $ inserted` errors — sirf kuch listings par.

Diagnosis: Captions mein `rag/vector_store.py` jaise filenames — text mode mein `_ =` math subscript. Isliye sirf underscore wale captions fail hote the (`vector_store`, `fact_checker`, `mcp_server`, `1_Benchmark`) — baaki (`schema.py`, `workflow.py`) clean the.

Fix: Captions mein `_` escape kiya.

Lesson: Reproduce minimally — maine failing listing ko alag file mein isolate kiya aur byte-level diff kiya. Debugging ka systematic approach: hypothesis → minimal repro → root cause → fix.

Part VI

Extended Q&A, Troubleshooting & Tutorial

40 30 More Interview Questions — Advanced & Career

40.1 Advanced LLM Topics

 Interview Question**Q49: “Fine-tuning vs RAG — kab kya?”**

Answer: RAG: **knowledge injection** — facts/docs jo badalte rehte hain ya private hain. Fine-tuning: **behavior/style** — format, tone, domain-specific response pattern. Rule of thumb: RAG pehle try karo (cheap, updateable, no training), fine-tuning tab jab style/behavior problem ho jo prompting se na solve ho. Is project mein RAG use kiya (knowledge) — fine-tuning zaroori nahi thi.

 Interview Question**Q50: “RLHF kya hai?”**

Answer: Reinforcement Learning from Human Feedback: (1) Model ko responses generate karwao; (2) Humans rank karein; (3) Reward model train karo; (4) Model ko RL se align karo. Ye LLM ko **helpful, harmless, honest** banata hai. Pre-trained model (next-word) vs aligned model (useful assistant) — API models aligned hote hain.

 Interview Question**Q51: “Quantization kya hai?”**

Answer: Model weights ki precision kam karna: FP32 (4 bytes/weight) → FP16/INT8/INT4. 70B model FP16 = 140GB, INT4 = 35GB — RAM requirement 4x kam. Trade-off: thodi accuracy loss, bada memory/speed gain. Is project mein models provider side hain (Groq), par RAG embeddings local chalti hain — quantization wahan relevant hota.

 Interview Question**Q52: “Context window ko extend kaise karein?”**

Answer: 4 techniques: (1) **Positional encoding** improvements (RoPE, ALiBi) — extrapolate karta hai; (2) **Sliding window** attention; (3) **Retrieval** — pura context dena hi mat, relevant parts do (is project ki strategy); (4) **Hierarchical** summaries. “Is project mein context compression: analyst dense brief banata hai, writer ko sirf wo + top-8 sources jate hain.”

 Interview Question**Q53: “Embedding model vs LLM mein difference?”**

Answer: Embedding model: text → vector (retrieval ke liye, encoder-only, fast). LLM: text → text (generation, decoder-only, heavy). Is project mein dono: all-MiniLM-L6-v2 embeddings

(ChromaDB ke liye) + GPT-OSS-120B generation.

Interview Question

Q54: “Streaming responses kyun important?”

Answer: User perception: 176s ka request bina feedback = user sochta hai system hang hai. Streaming token-by-token output dikhata hai — perceived latency kam. Is project mein Streamlit agent-status updates real-time dikhata hai (graph.stream events), aur MCP/API invoke() mein final result hi milta hai.

40.2 MLOps & Production

Interview Question

Q55: “Prompt versioning kya hai? Kya version control karte ho?”

Answer: Prompts bhi code hain — git mein version karo, change log rakho, aur A/B test karo (same prompt ke 2 versions benchmark par). Is project mein prompts constants hain (agents/*.py) — production mein separate prompt registry + config-driven banata.

Interview Question

Q56: “Observability: kya track karte ho?”

Answer: (1) **Per-node latency** — researcher kitna, writer kitna; (2) **Token/cost** per run; (3) **Retry/failure rates** — API errors; (4) **Quality metrics** — benchmark scores over time; (5) **Tracing** — LangSmith/Langfuse se pura run graph. Is project mein Django DB runs store karta hai + benchmark history.

Interview Question

Q57: “CI/CD is project mein kaise lagate?”

Answer: GitHub Actions pipeline: (1) **Lint/format** (ruff/black); (2) **pytest** — 16 tests; (3) **Build** — Docker images; (4) **Deploy** — Streamlit Cloud/Django server. Quality gate: tests fail = deploy nahi. Ye standard practice hai jo project par seedha apply hoti hai.

Interview Question

Q58: “Feature flag kya hota hai?”

Answer: Code deploy without enabling feature — runtime config se on/off. Use: gradual rollout, A/B testing, instant rollback. Is project mein example: naya analyst prompt flag ke peeche — 10% traffic par enable karke benchmark compare karo, phir full rollout.

Interview Question

Q59: “Cost optimization kaise karte?”

Answer: (1) **Caching** — same topic ka result cache; (2) **Model tiering** — simple tasks chhota model, complex bada; (3) **Context discipline** — kam tokens; (4) **Batching** — multiple requests ek call mein; (5) **Local models** — embeddings free. Is project mein: embeddings local free, Groq cheap, sirf Tavily paid — cache search results.

 Interview Question

Q60: “A/B testing LLM features kaise?”

Answer: (1) Traffic split (50/50); (2) Har version ke outputs **LLM judge** se score; (3) Metrics: depth, verifiability, user feedback; (4) **Statistical significance** — p-value, sample size (ek run ka verdict meaningless, jaise is project ke benchmark mein dikha). Ye hi is project ke benchmark infra ka production use case hai.

40.3 Career & Team

 Interview Question

Q61: “Apni strength/weakness batao?”

Answer: Strength: Problem solving with measurement — main benchmark/data se decisions leta hoon (is project ke examples se prove karo). **Weakness:** Kabhi kabhi detail mein itna chala jata hoon ki high-level deadline miss hoti — maine time-boxing seekhi (sprint planning, priorities). Weakness ko **improvement plan** ke saath bolna zaroori hai.

 Interview Question

Q62: “5 saal baad khud ko kahan dekhte ho?”

Answer: (Honest + growth-oriented) “AI engineering mein deep expertise — complex production systems (agents, RAG, evaluation infra) design karna. Pehle 2 saal: strong foundation (system design, ML fundamentals, production engineering). Aage: team lead / architect role jahan main systems design karun aur juniors ko mentor karun. Ye project usi direction ka pehla step hai.”

 Interview Question

Q63: “Salary expectation?”

Answer: Research karke bolo (Glassdoor/levels.fyi, role+location basis). Range do: “Maine is role ke liye 12-15 LPA ka range dekha hai, experience basis par. Main flexible hoon agar growth + learning acha hai.” Kisi number se pehle interviewer ko bolne do (“aapke budget ke hisaab se kya range hai?”) — negotiation basics.

 Interview Question

Q64: “Do you have any questions for us?”

Answer: 3 acha sawaal: (1) “Team mein AI features kaise ship hote hain — evaluation/benchmarking process kya hai?” (2) “Is role mein pehle 90 din mein kya achieve karna expected hai?” (3) “Team kaise kaam karti hai — code review, on-call, sprint structure?” Ye questions aapki **engineering maturity** dikhate hain.

 Interview Question

Q65: “Why NIT Hamirpur / dual degree CSE?”

Answer: (Personal + honest) “NIT Hamirpur ka CSE program strong theoretical foundation deta hai — DSA, OS, DBMS, ML core courses. Dual degree (B.Tech + M.Tech) ne research exposure diya — thesis/project work mein deep-dive ka habit bana. Is project mein wo hi habit dikhti hai — end-to-end system, benchmark se validate.”

 Interview Question**Q66: “College projects / internships batao?”**

Answer: (STAR format, 2-3 projects ready rakho) Is project ko main story banao + 1-2 aur (web dev, ML, hackathon). Har project mein: problem → approach → tech stack → result (numbers ke saath) → lessons.

 Interview Question**Q67: “Coding round ke liye DSA preparation kaise?”**

Answer: (Interview context) “Arrays/strings/hashmaps → two pointers, sliding window, binary search → trees/graphs → DP. 300+ problems, pattern-based (not count-based). Weekly contests + mock interviews. Ye project DSA se alag skill hai — dono parallel prepare karna zaroori.”

 Interview Question**Q68: “Aapne ye project kaise time management kiya?”**

Answer: (Honest) “Modules alag the (backend, agents, UI, benchmark) — har weekend ek module. Pehle working end-to-end (thoda raw), phir polish. Testing last mein nahi, har module ke saath. Failures (benchmark loss) ko feature ke roop mein treat kiya — measurement infra banaya. Realistic: 4-6 weeks, evenings + weekends.”

41 Troubleshooting Guide — Common Problems & Fixes

Symptom	Cause	Fix
No module named django	venv activate fail / deps missing	.venv/Scripts/python.exe + pip install -r
FastMCP unexpected keyword	fastmcp version mismatch	instructions= instead of description=
Model 404 / bad response	MODEL_NAME unavailable	un- GET /v1/models, live test, update config
Port 8000 already in use	Another service	runserver 8001 + BACK-END_API_URL
Emoji print error (Windows)	Console encoding	PYTHONIOENCODING=utf-8
Test time frozen / ties	Django test env	order by -created_at, -id
LaTeX Missing \$	Underscore in caption/text	Escape _
Streamlit secrets missing	Cloud secrets not set	Add keys in Streamlit settings
ChromaDB empty results	Collection empty / model mismatch	Check stats endpoint, same embedding model
Slow first run	Model download /	Preload embeddings, cache warmup

42 30-Minute Tutorial — Clone Se Running Tak

🔗 Code Samjho Line-by-Line

Minute 0-5: Setup

```
git clone <repo-url>
cd <project>
python -m venv .venv
# Windows: .venv\Scripts\activate | Linux/Mac: source .venv/bin/
# activate
pip install -r requirements.txt
cp .env.example .env # keys daalo
```

Minute 5-10: Backend

```
cd backend
python manage.py migrate
python manage.py runserver 8001
# verify: curl http://127.0.0.1:8001/api/health/
```

Minute 10-15: UI

```
# naya terminal
streamlit run app.py
# browser: http://localhost:8501
```

Minute 15-20: First Research

- Topic type karo (jaise “AI in healthcare 2026”)
- Run dabao — agent statuses live dikhenge
- Report + download + PDF upload try karo

Minute 20-25: Benchmark Lab

- Sidebar mein “Benchmark” page
- Topic do, Run — single vs multi-agent comparison dekho
- History charts dekho

Minute 25-30: MCP + Tests

```
pytest # 16 tests, ~9s
python mcp_server.py # MCP server
```

Done! Interview demo ke liye ye script yaad rakho — 30 min mein poora system dikhaya.

43 Project File Map — Har File Ka Kaam

File	Kaam
app.py	Streamlit main page: UI, session state, streaming pipeline, download
pages/1_Benchmark.py	Benchmark Lab: run benchmark + history charts
mcp_server.py	MCP server: 5 tools + resource for AI clients
config/setting.py	Central config: env vars, model, chunk settings, API keys
state/schema.py	ResearchState TypedDict — pipeline data contract
graph/workflow.py	LangGraph: nodes, edges, conditional loop, compiled singleton

agents/researcher.py	Search: sub-queries, Tavily, RAG retrieval
agents/analyst.py	Synthesis: themes, contradictions, gaps
agents/fact_checker.py	Verification: closed-world claims check, VERDICT
agents/writer.py	Report generation + memory store
rag/vector_store.py	ChromaDB: ingest, retrieve, stats, clear
benchmark/evaluator.py	Baseline + evaluation (LLM judge + heuristics)
backend/manage.py	Django entry point
backend/orchestrator_backend/	Django settings, urls, wsgi/asgi
backend/api/models.py	ResearchSession + BenchmarkEvaluation
backend/api/serializers.py	DRF serializers + validation
backend/api/views.py	6 API endpoints
backend/api/urls.py	URL routing
backend/api/tests/	16 pytest tests (auth, research, benchmark)
backend/conftest.py	Mock fixtures for tests
requirements.txt	Python dependencies
pytest.ini	pytest + Django config
.env.example	Environment template
README.md	Project docs + API reference
Multi_Agent_Master_Notes.tex	Ye document (source)

Memory Trick

Final revision tip: Interview se pehle: (1) 30-min tutorial 2 baar practice karo; (2) 10-15 Q&A khud bol ke practice karo (voice mein); (3) Project ke 3 numbers yaad rakho: 16 tests, 4 agents, 3 interfaces; (4) Ye document khol ke rakho — koi bhi question, kahin bhi jump karo. **All the best!**

Part VII

Core CS Revision — Coding Round Ke Liye

 Kahani Se Samjho

NIT CSE interview mein project ke saath **coding round** bhi hota hai — DSA + core CS (OS, DBMS, CN). Ye section 30-min quick revision hai — important concepts Hinglish mein, interview-ready form mein. Is project ke concepts se connect karke padho — double fayda.

44 DSA Patterns — 15 Patterns, 90% Problems

 Concept Deep Dive

LeetCode style coding round mein 15 patterns kaafi hain:

1. **Sliding Window** — subarray/substring problems (fixed + variable window)
2. **Two Pointers** — sorted arrays, palindrome, 3-sum
3. **Fast & Slow Pointers** — cycle detection, middle of linked list
4. **Merge Intervals** — sort + merge overlapping
5. **Cyclic Sort** — 1..n numbers missing/duplicate
6. **In-place Linked List Reversal**
7. **Tree BFS/DFS** — level order, max depth, paths
8. **Tree Traversals** — inorder/preorder/postorder + iterative versions
9. **Heap** — top-k, k-way merge, median stream
10. **Binary Search** — search space reduction (rotated array, peak)
11. **Backtracking** — permutations, combinations, subsets, board problems
12. **Dynamic Programming** — 0/1 knapsack, LCS, LIS, coin change
13. **Graph BFS/DFS** — connected components, shortest path (unweighted)
14. **Topological Sort** — dependency ordering, course schedule
15. **Trie** — prefix problems, autocomplete, word search

44.1 Sliding Window — Code Template

 Code Samjho Line-by-Line

```
1 def sliding_window(nums, k):
2     left = 0
3     window_sum = 0
4     best = 0
5     for right in range(len(nums)):
6         window_sum += nums[right]           # expand
7         while window_sum > k:               # shrink condition
8             window_sum -= nums[left]
9             left += 1
10        best = max(best, right - left + 1)
11    return best
```

Listing 36: Variable window template

Key insight: Har right index par window **expand** karo, condition tootne par **shrink** — har element max 2 baar touch hota hai, $O(n)$.

44.2 Binary Search — Template

Code Samjho Line-by-Line

```
1 def binary_search(arr, target):
2     lo, hi = 0, len(arr) - 1
3     while lo <= hi:
4         mid = (lo + hi) // 2
5         if arr[mid] == target:
6             return mid
7         elif arr[mid] < target:
8             lo = mid + 1
9         else:
10            hi = mid - 1
11    return -1 # not found
```

Listing 37: Binary search template

Variations: lower bound (first position $\geq$ target), upper bound, rotated array (check `arr[lo] <= arr[mid]` for sorted half), peak element.

44.3 DP — 0/1 Knapsack Pattern

Code Samjho Line-by-Line

```
1 def knapsack(weights, values, capacity):
2     n = len(weights)
3     dp = [0] * (capacity + 1)
4     for i in range(n):
5         for w in range(capacity, weights[i] - 1, -1):
6             dp[w] = max(dp[w], dp[w - weights[i]] + values[i])
7     return dp[capacity]
```

Listing 38: Knapsack DP

Pattern recognition: “Pick or skip” problems (subset sum, partition equal, coin change) — 1D DP with reverse loop. LCS/LIS: 2D DP grid.

Interview Question

Q: “DSA project se kaise relate karta hai?”

Answer: (Connect karo!) (1) **Graph traversal** = LangGraph ke nodes/edges; (2) **HashMap** = session state / ChromaDB metadata lookups; (3) **Sliding window** = context window management (top-k sources); (4) **Binary search** = sorted history queries; (5) **Trie** = autocomplete in search. Ye connections interviewer ko impress karte hain — “CS fundamentals apply karta hoon, sirf padhta nahi.”

45 OS + DBMS Quick Revision

45.1 Operating Systems — Top 10

Concept	1-line interview answer
Process vs Thread	Process: alag memory space; Thread: process ke andar, shared memory
Context switch	CPU ek process se dusre par switch — state save/restore
Deadlock	Circular wait of resources — 4 conditions: mutual exclusion, hold&wait, no preemption, circular wait
Deadlock prevention	Ek condition tod do (allocation strategy)
Scheduling	FCFS, SJF, Round Robin (time-slice), Priority — trade-offs: fairness vs throughput
Page fault	Requested page RAM mein nahi — disk se load
LRU cache	Least Recently Used eviction — Python dict preserves insertion, OrderedDict for LRU
Semaphore/Mutex	Mutex: binary lock; Semaphore: counting lock (producer-consumer)
Virtual memory	Disk ko RAM extension — paging, TLB
Race condition	Concurrent access without sync — locks needed

Interview Question

Project connection: “Streamlit app **thread-safety**: session state per-session isolated hai — concurrent users ka data mix nahi hota. Django production mein Gunicorn workers = multiple processes (isolated memory). Thread-safety aur isolation real systems mein practical problem hai.”

45.2 DBMS — Top 12

Concept	1-line interview answer
Primary key	Unique row identifier (NOT NULL + UNIQUE)
Foreign key	Dusri table ka reference — referential integrity
Index	Search speed up (B-tree / hash) — write slow down
Normalization	Redundancy kam — 1NF/2NF/3NF (no transitive dependency)
ACID	Atomicity, Consistency, Isolation, Durability
Transaction	Multiple ops as one unit — commit/rollback
JOIN	Inner: common rows; Left: all left + matches; Cross: cartesian
GROUP BY	Rows group karke aggregate (COUNT, SUM, AVG)
SQL injection	Untrusted input SQL mein — parameterized queries se bacho
CAP theorem	Consistency vs Availability vs Partition tolerance — 2 choose karo
Sharding	Data horizontally split across servers

Replication

Data copy across servers — read scaling + failover

 Interview Question

Project connection: “Django ORM migrations = schema version control. Is project mein ResearchSession + BenchmarkEvaluation FK/reasons. `order_by("-created_at", "-id")` — index-friendly deterministic ordering. SQLite single-writer limitation production mein PostgreSQL shift ka reason hai. Ye DBMS concepts is project par directly apply hote hain.”

45.3 SQL Practice — 5 Must-Know Queries

 Code Samjho Line-by-Line

```

1 -- 1. Join + aggregate: har user ke sessions count
2 SELECT u.username, COUNT(s.id) AS session_count
3 FROM auth_user u
4 LEFT JOIN api_researchsession s ON s.user_id = u.id
5 GROUP BY u.id, u.username
6 ORDER BY session_count DESC;
7
8 -- 2. Latest benchmark per topic (window function)
9 SELECT topic, single_agent_depth, created_at
10 FROM (
11     SELECT *, ROW_NUMBER() OVER (
12         PARTITION BY topic ORDER BY created_at DESC) rn
13     FROM api_benchmarkevaluation
14 ) t WHERE rn = 1;
15
16 -- 3. Index
17 CREATE INDEX idx_session_created ON api_researchsession (created_at
18     DESC);
19
20 -- 4. Update
21 UPDATE api_benchmarkevaluation SET verdict = 'TIE'
22 WHERE created_at < '2026-01-01';
23
24 -- 5. Delete duplicates
25 DELETE FROM api_benchmarkevaluation
26 WHERE id NOT IN (
27     SELECT MIN(id) FROM api_benchmarkevaluation GROUP BY topic);

```

Listing 39: SQL essentials

46 Computer Networks Quick Revision

Concept	1-line interview answer
OSI model	7 layers: Physical, Data Link, Network, Transport, Session, Presentation, Application

TCP vs UDP	TCP: reliable, ordered, connection (handshake); UDP: fast, no guarantee
TCP handshake	3-way: SYN → SYN-ACK → ACK
HTTP methods	GET (read), POST (create), PUT (update), DELETE, PATCH
HTTP status	200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404, 500
HTTPS	HTTP + TLS encryption — certificate based trust
DNS	Domain → IP resolution (hierarchy: root → TLD → authoritative)
Load balancer	Traffic distribute across servers (round-robin, least-connections)
CDN	Content cached geographically near users
WebSocket	Full-duplex persistent connection (real-time)
CORS	Browser security: cross-origin policy — server headers control
Rate limiting	Request frequency cap — token bucket, fixed window

Interview Question

Project connection: “(1) Django API: HTTP methods/status codes exact REST semantics follow karte hain; (2) CORS: django-cors-headers — Streamlit (8501) se Django (8001) cross-origin calls; (3) MCP stdio transport: local process pipes; SSE/HTTP remote; (4) Real-time: Streamlit WebSocket-based; (5) Security: HTTPS + rate limiting production checklist.”

46.1 System Design Quick Patterns

Concept Deep Dive

4 design patterns interview mein:

1. **API Gateway** — single entry, auth, rate limiting, routing.
2. **Message Queue** — decoupling + async (SQS/RabbitMQ/Kafka) — pipeline jobs ke liye.
3. **Cache** — Redis: hot data, session, rate limit counters.
4. **Database per service** — microservices isolation (ya monolith first).

Is project ke design principles: single core (research_graph) + multiple adapters (Streamlit/MCP/Django) — adapter pattern ka real example. Ye bolo to interviewer ko pata chalega ki tum patterns ko naam se jaante ho.

Part VIII

Demo Scripts, Mock Interview & Case Studies

47 Demo Scripts — 30 Sec, 2 Min, 10 Min

47.1 30-Second Version (Elevator Pitch)

 Interview Question**Script (word-by-word):**

“Maine ek **multi-agent AI research system** banaya hai. User koi bhi topic deta hai, system **web search** karta hai (Tavily), data ko **analyze** karta hai, **facts verify** karta hai ek dedicated fact-checker agent se, aur **verified report** generate karta hai — with revision loop agar claims weak milein. Teen interfaces: **Streamlit UI**, **MCP server** (AI assistants ke liye), **Django REST API**. RAG memory: PDFs + past research **ChromaDB** mein. Aur ek **benchmark suite** hai jo single vs multi-agent quality measure karta hai.”

47.2 2-Minute Version (With Architecture)

 Interview Question**Script:**

“Pehle 30-sec wala part, phir:”

Architecture: LangGraph StateGraph — shared state (TypedDict) jisme har node apna output add karta hai. 4 nodes: **researcher** (LLM se 3 diverse search queries, Tavily se 15 sources, RAG retrieval), **analyst** (top-10 sources se themes/contradictions/gaps ka dense brief), **fact-checker** (closed-world verification — claims sirf sources se VERIFIED; fail par revision loop max 3 baar), **writer** (brief + top-8 sources se final Markdown report, ChromaDB memory store).

Interfaces: Ek compiled graph, teen adapters — Streamlit real-time streaming (`graph.stream`), MCP tools (Claude Desktop connect), Django REST (programmatic + benchmark).

Engineering: 16 pytest tests with full mocking (~9s), config 3-level secret fallback, graceful degradation har external call par, aur benchmark (depth + verifiability, LLM judge + heuristic fallback). Real runs verified: end-to-end 176s, 15 sources, first-pass fact-check pass.”

47.3 10-Minute Version (Live Demo Walkthrough)

Interview Question

Structure:

1. 0-1 min: Elevator pitch + ek line problem statement.
2. 1-3 min: Architecture draw karo (whiteboard): 4 layers + data flow.
3. 3-5 min: **Live demo** — Streamlit: topic run karo, agent statuses dikho, report + download.
4. 5-6 min: PDF upload + RAG retrieval dikhao (“ab ye knowledge base use hoga”).
5. 6-7 min: **Benchmark Lab** — run benchmark, verdict dikhao, history charts.
6. 7-8 min: Django API — `curl /api/health/`, register, execute (ya Postman screenshot).
7. 8-9 min: Tests — `pytest` chalo (9s), output dikhao.
8. 9-10 min: Code tour — `graph/workflow.py` ke 40 lines, ek agent file.

Rule: Kabhi bhi 30 sec se zyada ek cheez par mat ruko — interviewer ko breadth + depth dono dikho.

48 Mock Interview Dialogue — Poora Session

Interview Question

Interviewer: “So, tell me about your project.”

You: (30-sec pitch bol do) “Maine multi-agent research orchestration system banaya hai...” (section 47 ka script)

Interviewer: “Why multi-agent? Couldn’t one LLM call do the same?”

You: “Single call se **verification** nahi hoti. Research quality 3 cheezon par depend karti hai: **coverage** (sahi sources mile), **synthesis** (data se insight bani), aur **verification** (claims true hain). Ek call mein teeno ek saath nahi hote — agents specialize karte hain. Aur maine isko **benchmark** se prove karne ki koshish ki — depth aur verifiability measure karke. Result interesting tha: ek run mein single-agent jeet bhi gaya — isliye evaluation zaroori hai, assumptions nahi.”

Interviewer: “What’s the most interesting technical challenge?”

You: (Bug 1 ya Bug 5 ki story — section 39) “FastMCP version mismatch se MCP server crash — root cause nikala, fix kiya. Aur benchmark mein multi-agent haar gayi — root cause: writer-analyst handoff. Ye debugging + root-cause analysis wali mindset dikhati hai.”

Interviewer: “How would you scale this?”

You: (Section 25 scaling story) “Synchronous 176s pipeline ko async karna (task queue), SQLite to PostgreSQL, local ChromaDB to managed vector DB, Streamlit multi-instance, rate limiting + caching. Pehla step hamesha bottleneck measure karna.”

Interviewer: “Do you have questions for us?”

You: (Section 40 Q64 ke 3 questions)

49 Case Studies — 5 Topics Ka Deep Analysis

49.1 Case Study 1: Finance Research

Concept Deep Dive

Topic: “Indian EV market 2026” **Pipeline behaviour:**

- Researcher: sub-queries — market size, policy incentives, players, supply chain.
- Analyst: themes (growth drivers, policy, competition), contradictions (market size estimates alag sources mein), gaps (battery import data).
- Fact-checker: market numbers sources se verify.
- Writer: Executive summary + investment outlook + policy table.

Interview value: Finance domain mein **verifiability** critical hai — citations + numbers cross-check. Is project ki fact-checker yahin shine karta hai.

49.2 Case Study 2: Academic Research

Concept Deep Dive

Topic: “Attention mechanisms survey” **Pipeline behaviour:**

- RAG: uploaded papers (PDFs) se context — research_docs collection.
- Analyst: 3-5 themes (architectures, benchmarks, limitations).
- Fact-checker: claims papers se verify (closed-world = sirf uploaded papers).
- Writer: survey-style report with citations.

Interview value: Ye dikhata hai ki RAG (PDFs) + agents ka combination academic literature review mein kaam karta hai — thesis/project work se connect karo.

49.3 Case Study 3: Tech Comparison

Concept Deep Dive

Topic: “Rust vs Go for backend 2026” **Pipeline behaviour:**

- Contradictions strong honge (community opinions alag).
- Fact-checker: performance claims benchmarks se verify.
- Writer: comparison tables + use-case recommendations.

Interview value: Contradiction handling ka perfect example — system **disagreement ko hide nahi karta**, report mein highlight karta hai (“Consensus & Contradictions” section). Ye honest AI hai.

49.4 Case Study 4: Health Research

Concept Deep Dive

Topic: “AI in medical diagnosis 2026” **Pipeline behaviour:**

- High stakes — hallucination dangerous. Fact-checker critical: har claim source-backed.
- Gaps flag: “clinical trial data missing” — writer honest reporting.

Interview value: Ye case study batao jab interviewer puchhe “is system ke limitations kya hain?” — medical domain mein human-in-the-loop + regulatory compliance zaroori hota. Aap

limitations jaante ho — ye maturity dikhata hai.

49.5 Case Study 5: How to Answer “Improve This System”



Interview Question

Question: “Is system ko production-ready banane ke liye kya karenge?”

Answer (prioritized roadmap):

1. **Async pipeline** — Celery/Redis, results stored, polling/webhook (176s sync request khatam).
2. **Evaluation infra** — 20-topic benchmark suite + CI mein regression check.
3. **Structured outputs** — Pydantic schemas har agent ke (free text → typed).
4. **Observability** — LangSmith tracing, per-node latency, cost dashboards.
5. **Security** — rate limiting, input sanitization, CORS allowlist, HTTPS.
6. **Data layer** — PostgreSQL + managed vector DB + backups.
7. **Multi-model fallback** — Groq down → OpenAI.
8. **Caching** — same-topic results + search result dedup.

Ordering logic: Pehle reliability (async), phir quality measurement, phir scale — har point justify karo.



Memory Trick

Document ka closing thought:

Ye 150-page document aapke saath interview tak rahega. **Jo sabse important hai:** concepts itne samjho ki bina notes ke bolo — ye document notes nahi, **understanding ka tool** hai. Har section khud bol ke practice karo. Numbers yaad rakho (16 tests, 4 agents, 3 interfaces, 176s, 15 sources). Stories ready rakho (bugs, benchmark loss, fixes). Aur sabse badi baat — **genuinely interested** raho: ye project tumhara hai, tumne banaya hai, aur isse pyar karte ho. Wo energy interview mein dikhti hai.

All the best, bhai! 🎓👍

Part IX

Python Deep Dive — Project Ke Code Se Seekho

Kahani Se Samjho

Is project ka saara code Python hai — aur isme modern Python features kaafi use hue hain. Ye section har feature ko **project ke asli code se** samjhata hai — interview mein “Python deep knowledge” ka proof.

50 Python Features — Project Ke Code Se

50.1 TypedDict — Dictionary Ka Type Contract

Code Samjho Line-by-Line

```
1 from typing import TypedDict, Annotated
2
3 class ResearchState(TypedDict):
4     topic: str
5     research_data: list[str]
6     analysis: str
7     fact_check_result: str
8     fact_check_passed: bool
9     revision_count: int
10    final_report: str
11    messages: Annotated[list, add_messages]
12    rag_context: list[str]
13    current_agent: str
```

Listing 40: state/schema.py se

Kya hai: TypedDict runtime par sirf ek normal dict hai — par **type checker** (mypy/pyright) ko batata hai ki kis key par kaunsa type hoga. Isse: (1) autocomplete milta hai; (2) typo keys compile-time pakda jaata hai; (3) pipeline ka contract document hota hai.

Interview line: “TypedDict se state schema **self-documenting** hai — naya developer dekh ke samajh jata hai ki graph kya data carry karta hai.”

50.2 Annotated + Reducer — LangGraph Ka Contract

Code Samjho Line-by-Line

```
1 from typing import Annotated
2 from langgraph.graph.message import add_messages
3
4 messages: Annotated[list, add_messages]
```

Annotated kya hai: Type + metadata ko combine karta hai. Type: `list`; metadata: `add_messages` (ek function). LangGraph is metadata ko padh kar decide karta hai ki is field ko kaise merge karna hai.

Why useful: Normal typing type checker ko sirf type batata hai; **Annotated behaviour** bhi batata hai. Ye “metadata-driven design” ka example hai — framework configuration type system mein hi rehta hai.

50.3 Generators — `stream()` Ke Peeche

Code Samjho Line-by-Line

```
1 def countdown(n):
2     while n > 0:
3         yield n           # pause here, value do, phir resume
4         n -= 1
5
6 for x in countdown(3):    # 3, 2, 1
7     print(x)
```

Listing 41: Generator concept

Project mein: `research_graph.stream(state)` ek generator return karta hai — har node ke baad event yield hota hai. Streamlit har event par UI update karta hai. Agar poora result ek saath aata (invoke), to real-time progress dikhana possible nahi hota.

50.4 Context Managers — `with` Statement

Code Samjho Line-by-Line

```
1 # Django test client ke andar
2 with patch("graph.workflow.research_graph") as mock_graph:
3     mock_graph.invoke.return_value = {"final_report": "..."}
4     resp = client.post("/api/research/execute/", ...)
```

Listing 42: with pattern

Kya hai: `with` block enter/exit lifecycle control karta hai — setup/teardown guarantee. `unittest.mock.patch` isi se kaam karta hai: enter par mock install, exit par restore. Django test client bhi transaction lifecycle manage karta hai.

50.5 Exception Handling — Graceful Degradation

Code Samjho Line-by-Line

```
1 try:
2     if groq_key:
3         llm = get_llm()
4         query_response = llm.invoke([...])
```

```

5     lines = [l.strip() for l in ... if l.strip()]
6     if lines:
7         sub_queries = lines[:3]
8 except Exception as e:
9     sub_queries = [topic, f"{topic} latest research", f"{topic}
    analysis"]

```

Listing 43: agents/researcher.py se

Pattern: try → best path; except → **graceful fallback** (default values). Ye har external call ke saath repeat hota hai. Interview point: “Broad except Exception intentional hai — main fail-graceful chahata hoon, fail-fast nahi. Production mein specific exceptions + logging add karunga.”

50.6 Dictionary Operations — Python Ka Dil

🔗 Code Samjho Line-by-Line

```

1 state.get("revision_count", 0)      # safe access with default
2 state.get("fact_check_result")      # None if missing
3 "VERDICT: NEEDS_REVISION" in result.upper() # substring check
4 {"chunks_added": n, "files_processed": p} # dict literal return
5 f"Topic: {topic}\n\n" + "\n".join(...) # f-string + join

```

Listing 44: Dict patterns used

Interview line: “state.get(key, default) se missing keys crash nahi karti — optional fields safe. Isliye revision_count pehli baar 0 default leta hai.”

50.7 List Slicing — Context Window Control

🔗 Code Samjho Line-by-Line

```

1 research_data[:10]      # analyst: top 10 sources
2 research_data[:8]       # writer: top 8 sources
3 rag_context[:5]         # top 5 rag chunks
4 lines[:3]               # exactly 3 queries

```

Interview line: “Slicing se context window discipline milta hai — har agent ko fixed-size input. Ye token cost + focus control karta hai. Isliye hi har agent ki ‘context diet’ alag hai.”

51 Full Code Walkthroughs — Jo Pehle Nahi Dekha

51.1 benchmark/evaluator.py — Line-by-Line

</> Code Samjho Line-by-Line

```

1 def run_single_agent_baseline(topic: str) → str:
2     # 1 LLM call: writer prompt seedha, bina research ke
3     llm = get_llm()
4     resp = llm.invoke([
5         SystemMessage(content=WRITER_PROMPT),
6         HumanMessage(content=f"Topic: {topic}"),
7     ])
8     return resp.content
9
10 def evaluate_outputs(topic, report_a, report_b) → dict:
11     # Step 1: LLM judge try karo
12     try:
13         scores = llm_judge(topic, report_a, report_b)
14         if scores:
15             return scores
16     except Exception:
17         pass
18     # Step 2: fallback --- heuristics (hamesha kaam karta hai)
19     return heuristic_scores(report_a, report_b)
20
21 def heuristic_scores(report_a, report_b) → dict:
22     # depth: section count + list/bullet count + word count
23     # verifiability: citation count + URL mentions
24     ...

```

Listing 45: evaluator.py — poora flow

Design decisions:

- **LLM judge first, heuristics fallback** — quality jab ho, reliability hamesha.
- **Heuristics deterministic** — tests mein predictable.
- **Both reports same judge, same call** — relative scoring, bias kam.

51.2 pages/1_Benchmark.py — Charts Ka Logic**</> Code Samjho Line-by-Line**

```

1 def _get_api_url() → str:
2     url = os.getenv("BACKEND_API_URL", "")
3     if not url:
4         try:
5             url = str(st.secrets.get("BACKEND_API_URL", ""))
6         except Exception:
7             pass
8     return url.rstrip("/") or "http://localhost:8000/api"
9
10 def load_history() → list[dict]:
11     r = requests.get(HISTORY_URL, timeout=15)
12     r.raise_for_status()
13     return r.json()
14
15 def history_frame(records: list[dict]) → pd.DataFrame:

```

```

16 rows = [{...} for item in records]
17 df = pd.DataFrame(rows)
18 if not df.empty:
19     df["created_at"] = pd.to_datetime(df["created_at"], errors="
        coerce")
20     df = df.sort_values("created_at")
21 return df

```

Listing 46: Benchmark page API client

Design: (1) Configurable API URL (env → secrets → default); (2) `r.raise_for_status()` fail-fast client side; (3) pandas DataFrame — `st.line_chart` seedha consume karta hai.

51.3 backend/api/views.py — Execute Research View

Code Samjho Line-by-Line

```

1 class ExecuteResearchAPIView(APIView):
2     def post(self, request):
3         topic = request.data.get("topic", "").strip()
4         if not topic:
5             return Response({"error": "Topic is required"},
6                               status=status.HTTP_400_BAD_REQUEST)
7         try:
8             state = initial_state(topic)
9             final = research_graph.invoke(state)
10            session = ResearchSession.objects.create(
11                topic=topic,
12                report=final["final_report"],
13                messages=final.get("messages", []),
14                user=request.user if request.user.is_authenticated else
                    None,
15            )
16            return Response(ResearchSessionSerializer(session).data,
17                            status=status.HTTP_201_CREATED)
18        except Exception as e:
19            return Response({"error": str(e)},
20                            status=status.
                    HTTP_500_INTERNAL_SERVER_ERROR)

```

Listing 47: ExecuteResearchAPIView

Flow: Validate → invoke pipeline → persist → serialize. **Transaction note:** Agar invoke fail hota hai, create kabhi nahi hota — half-saved session impossible. Anonymous user → `user=None`.

51.4 mcp_server.py — Tools Deep Dive

 Code Samjho Line-by-Line

```

1 from mcp.server.fastmcp import FastMCP
2
3 mcp = FastMCP("Research Orchestrator",
4               instructions="Research + knowledge base tools")
5
6 @mcp.tool()
7 def research_topic(topic: str) → dict:
8     """Research karo aur report do."""
9     state = initial_state(topic)
10    final = research_graph.invoke(state)
11    return {"report": final["final_report"], "messages": final.get("
        messages")}
12
13 @mcp.tool()
14 def ingest_document(file_path: str) → dict:
15     """PDF ko knowledge base mein daalo."""
16     return ingest_documents([file_path])
17
18 @mcp.tool()
19 def search_knowledge_base(query: str, k: int = 5) → list[str]:
20     """Knowledge base search karo."""
21     return retrieve_context(query, k=k)
22
23 @mcp.tool()
24 def get_kb_stats() → dict:
25     return get_collection_stats()
26
27 @mcp.tool()
28 def clear_knowledge_base(collection: str = "research_docs") → dict:
29     clear_collection(collection)
30     return {"cleared": collection}
31
32 @mcp.resource("info://orchestrator")
33 def info() → str:
34     """Project info resource."""
35     return "Multi-agent research orchestrator..."

```

Listing 48: MCP tool pattern

Interview points: (1) `@mcp.tool()` decorator — function signature se tool schema auto-generate; (2) Type hints se input validation; (3) docstring = tool description (LLM ko batata hai kab call karna hai); (4) **fix:** fastmcp 3.x mein `description=` → `instructions=`.

52 Python Interview Questions — Project-Based

Interview Question

Q69: “Mutable vs immutable Python mein?”

Answer: Mutable (changeable): list, dict, set. Immutable: tuple, str, int, frozenset. Is project mein: state dict mutable hai (LangGraph update karta hai); `research_data` list append hota

hai; TypedDict dict hai — isliye shallow copy semantics careful. Interviewer ko id/equality ka difference bhi batao.

Interview Question

Q70: “Shallow vs deep copy?”

Answer: Shallow copy: top-level copy, nested objects **shared**. Deep copy: sab kuch naya. Is project mein state update karte waqt **shallow merge** hota hai (LangGraph default) — nested fields replace hote hain. Isliye har node apne top-level keys hi return karta hai — shared nested mutation se bachta hai.

Interview Question

Q71: “GIL kya hai? Python multi-threading?”

Answer: GIL (Global Interpreter Lock): ek time par ek thread hi Python bytecode chala sakta hai — CPU-bound threading limited. I/O-bound (network calls) threading se acha scale hota hai kyunki lock I/O wait par release hota hai. Is project mein: Tavily/LLM calls I/O-bound hain — 3 sub-queries parallel ThreadPool se 3x fast ho sakte hain. CPU-bound (embeddings) ke liye multiprocessing.

Interview Question

Q72: “Decorator kya hai? Project mein kahan use hua?”

Answer: Decorator = function ko modify karne wala wrapper. Is project mein: `@mcp.tool()` — function ko MCP tool schema mein convert; Django `@api_view`; pytest fixtures (decorator-like). Custom decorator example: `@timer` jo function latency log kare — production observability ke liye.

Interview Question

Q73: “Generator vs list? Kab kya?”

Answer: Generator: lazy — ek-ek value, memory efficient, ek baar iterate. List: eager — saari values memory mein, multiple iteration. Is project mein: `graph.stream()` generator hai — 176s ke run mein har event real-time process hota hai, sab kuch memory mein nahi. Report (final result) list/str hai — multiple baar access chahiye (render, download, store).

Interview Question

Q74: “args and kwargs?”

Answer: args: positional args ka tuple. kwargs: keyword args ka dict. Is project mein: `tool_type` patterns, fixture helpers, aur function wrappers mein. Django/DRF bhi internally kwargs use karta hai (request context).

Interview Question

Q75: “Type hints kyun? Runtime mein kya hota hai?”

Answer: Type hints runtime par **ignore** hote hain (Python duck-typed) — sirf documentation + type checker ke liye. Is project mein TypedDict/Annotated type checkers ko contract batate hain — bugs development mein hi pakde jaate hain, production mein nahi. Optional: pydantic runtime validation ke liye (production upgrade).

Part X

Extra Q&A + Project Reference

53 20 More Interview Questions — Mixed Topics

53.1 AI/ML Concepts

 Interview Question**Q76: “Overfitting kya hai? LLM mein hota hai?”**

Answer: Overfitting: model training data memorize karta hai, generalize nahi. LLM fine-tuning mein ho sakta hai (data memorize). Is project mein overfitting relevant nahi (hum fine-tune nahi karte) — par **prompt overfitting** hota hai: prompts specific examples par tune ho jate hain aur real-world fail. Isliye benchmark on unseen topics zaroori.

 Interview Question**Q77: “Precision vs recall? Is project mein kaunsa metric?”**

Answer: Precision: jo bole wo kitna sahi (false positives kam). Recall: jo sahi tha wo kitna pakda (false negatives kam). RAG retrieval: precision = retrieved chunks relevant hain? recall = saare relevant chunks aaye? Is project ke benchmark mein depth/verifiability quality metrics hain — retrieval metrics (recall@k) production mein add karne ki plan hai.

 Interview Question**Q78: “Gradient descent intuition?”**

Answer: Loss function ko minimize karte hue weights update: $w = w - \alpha \nabla L$. Learning rate α : bada = fast par overshoot; chhota = slow par stable. Is project mein hum trained model use karte hain — gradient descent training ka core hai, inference forward pass only.

 Interview Question**Q79: “Batch vs online learning?”**

Answer: Batch: pura dataset ek saath train. Online/incremental: data aate hi update. Is project mein RAG ek tarah ka “online knowledge” hai — naye PDFs embed hokar turant available, bina model retrain kiye. Ye hi RAG vs fine-tuning ka core difference hai.

 Interview Question**Q80: “Confusion matrix?”**

Answer: TP/FP/TN/FN ka table. Is project mein: fact-checker ke VERIFIED/UNVERIFIED decisions ko confusion matrix mein rakha ja sakta hai — VERIFIED par source support nahi = false positive (hallucination slipped). Ye evaluation maturity dikhata hai.

53.2 Product & Design Thinking

 Interview Question

Q81: “Is system ka target user kaun hai?”

Answer: Teen user types: (1) **Knowledge worker** — deep research reports chahiye (Streamlit); (2) **AI assistant user** — Claude/Cursor se research tool call (MCP); (3) **Developer** — programmatic access + benchmark (Django API). Har interface apne user ke consumption pattern ke liye designed hai.

 Interview Question

Q82: “Is system ki moat kya hai?”

Answer: (Honest) Core moat nahi hai — LLMs + Tavily + ChromaDB sab commodity hai. Moat: (1) **Workflow design** — fact-check + revision loop ka specific orchestration; (2) **Evaluation data** — benchmark results jo accumulate hote hain; (3) **UX** — 3 interfaces ka integration. Ye bolo to product thinking dikhti hai.

 Interview Question

Q83: “Ek feature drop karna ho to kaunsa?”

Answer: (Prioritization) MCP server drop kar sakte hain — niche use case (AI assistants ko tools chahiye), audience chhota. Core (pipeline + UI + benchmark) rakho. Ya PDF upload drop karo agar koi use nahi karta — data se decide karo (analytics dekho), assumptions se nahi.

 Interview Question

Q84: “User feedback kaise collect karoge?”

Answer: (1) **Quantitative** — benchmark scores, latency, usage analytics; (2) **Qualitative** — report quality rating (thumbs up/down), open feedback; (3) **Interviews** — 5 users se 30-min sessions. Is project mein benchmark hi quantitative feedback loop hai.

 Interview Question

Q85: “Is project ko open source karna chahiye?”

Answer: Trade-offs: **Pro:** credibility, community contributions, portfolio visibility. **Con:** maintenance burden, secrets leak risk (env gitignored par careful), competition. Personal project ke liye: haan, open source + good README — ye interview portfolio ke liye strong signal hai.

53.3 Miscellaneous Technical

 Interview Question

Q86: “JSON vs XML vs YAML?”

Answer: JSON: lightweight, universal, no comments — API standard. XML: verbose, schema validation (XSD) — legacy/enterprise. YAML: human-friendly, comments, nested — config files. Is project mein: API responses JSON (DRF default), config .env (key-value), MCP schemas JSON, aur ye document LaTeX.

 Interview Question**Q87: “REST vs GraphQL?”**

Answer: REST: resource-based endpoints, simple, cacheable — is project mein. GraphQL: client-controlled queries, over-fetching/under-fetching solve — par complexity + caching hard. Is scale par REST right choice; GraphQL tab jab client data needs vary karein (mobile + web + dashboard).

 Interview Question**Q88: “Synchronous vs asynchronous programming?”**

Answer: Sync: ek kaam khatam, phir agla — simple, blocking. Async: I/O wait mein doosra kaam — efficient. Is project mein pipeline sync hai (176s block) — production mein async (task queue) because users wait nahi karte. Python async: asyncio/await + event loop; concurrency (not parallelism) for I/O.

 Interview Question**Q89: “Docker kya hai? Container vs VM?”**

Answer: Container: app + dependencies packaged, OS kernel shared — light, fast start. VM: full OS per instance — heavy, isolated. Is project ke liye Docker: `python:3.11-slim` base, requirements install, streamlit/django CMD — reproducible environments (“works on my machine” khatam).

 Interview Question**Q90: “Git branching strategy?”**

Answer: Simple: main (stable) + feature branches + PR review + merge. Production: main protected, release branches, tags. Is project mein main par direct work hua (solo) — team mein feature branches + PRs. Interview mein trunk-based vs git-flow dono batao.

 Interview Question**Q91: “Logging vs print?”**

Answer: Logging: levels (DEBUG/INFO/WARNING/ERROR), configurable, timestamps, structured. Print: ad-hoc, no control. Is project mein agents `print` karte hain (sabse basic) — production mein `logging` module + structured JSON logs + log aggregation. Interview line: “print demo ke liye, logging production ke liye.”

 Interview Question**Q92: “Environment variables vs config files?”**

Answer: Env vars: secrets + environment-specific (keys, URLs) — gitignored, 12-factor. Config files: non-secret defaults (chunk size, model name) — committed, versioned. Is project mein: secrets env/secrets; defaults config/setting.py mein with env override. Ye separation security + clarity dono deta hai.

 Interview Question**Q93: “API key leak ho jaye to kya karoge?”**

Answer: (Incident response) (1) **Revoke immediately** — provider dashboard se key delete; (2) Naya key generate + rotate; (3) **Source hunt** — git history se remove (git filter-repo),

secrets scanning; (4) Damage assessment — usage logs check karo unauthorized calls; (5) Prevention — .env gitignore, secret managers, pre-commit hooks. Ye structured response impress karta hai.

Interview Question

Q94: “Documentation kyu important?”

Answer: (1) Onboarding — naya developer 10 min mein setup (README + run.md); (2) **Knowledge preservation** — 6 mahine baad khud bhi bhool jaoge; (3) **Quality signal** — interviewers README dekhte hain; (4) **API docs** — consumers ke liye contract. Is project mein: README (setup + API table), run.md (preview ops), ye 150-page document (concepts + code + interview).

Interview Question

Q95: “Koi feature ship karte waqt testing strategy?”

Answer: (1) **Unit tests** — logic (evaluator heuristics); (2) **Integration** — API + mocked pipeline; (3) **E2E** — real run (manual, paid); (4) **Benchmark** — quality regression; (5) **Monitoring** — production metrics post-ship. Is project mein 16 tests + benchmark + real-run verification — multi-layer strategy.

54 Dependency Reference — Har Package Kya Karta Hai

Package	Role	Project mein kahan
langchain-core	Messages, prompts	agents/*.py
langchain-groq	ChatGroq LLM client	agents ka get_llm
langchain-community	PyPDFLoader, Chroma	rag/vector_store.py
langchain-text-splitters	RecursiveCharacterTextSplitter	rag ingestion
langchain-huggingface	HuggingFaceEmbeddings	rag embeddings
langgraph	StateGraph orchestration	graph/workflow.py
tavily-python	Web search client	researcher
sentence-transformers	MiniLM model	embeddings (transitive)
chromadb	Vector store	rag/vector_store.py
streamlit	Web UI	app.py, pages/
mcp / fastmcp	MCP server	mcp_server.py
django	Web framework	backend/
djangorestframework	REST API layer	backend/api/
django-cors-headers	CORS policy	backend settings
python-dotenv	.env loading	config/setting.py
requests	HTTP client	pages/1_Benchmark.py
pandas	DataFrames + charts	pages/1_Benchmark.py
pytest	Testing	backend/api/tests/
pytest-django	Django test integration	pytest.ini

 Memory Trick

Interview line: “Dependencies ka har package ka role pata hai — langchain ecosystem orchestration/retrieval ke liye, Django API ke liye, Streamlit UI ke liye, pytest quality ke liye. Koi dependency bina kaam ke nahi hai.”

55 Project Metrics — Numbers Jo Yaad Rakho

Metric	Value	Context
Agents	4	researcher, analyst, fact-checker, writer
Interfaces	3	Streamlit, MCP, Django REST
Tests	16	auth(6) + research(4) + benchmark(6)
Test time	~9s	Fast suite = daily use
Pipeline time	~176s	End-to-end real run
Sources per run	~15	3 sub-queries x 5 results
Fact-check passes	1st pass	Solid-state run, revision 0
Max revisions	3	Loop guard
Chunk size / overlap	1000 / 200	Chars
Embedding dims	384	all-MiniLM-L6-v2
Model	GPT-OSS-120B	Via Groq
Endpoints	6	health, register, login, execute, benchmark, history
Benchmark dims	2	Depth + verifiability (0-10)
Benchmark verdicts	3	single/multi superior + tie
DB tables	2 + auth	ResearchSession, BenchmarkEvaluation

 Kahani Se Samjho**Final message:**

Ye document ab poori tarah taiyaar hai. 150+ pages, har concept, har file, har logic — Hinglish mein, story format mein. **Ab aapki baari hai:** ise padho, samjho, aur sabse zaroori — **khud bol ke practice karo.** Interview success = understanding + confidence + communication. Teeno is document se ready hain.

Shubhkamnayein! 🙌

Part XI

Final Revision — Speed Round

56 100 Speed-Round Questions — 1 Line Answers

Concept Deep Dive

Interview ke aakhri 5 minute mein speed round hota hai — fast questions, crisp answers. Ye 100 Q&A bol-bol ke practice karo (10-15 min). Har answer 1 line ka.

56.1 Python (1-20)

1. **Q:** List vs tuple? **A:** Mutable vs immutable.
2. **Q:** Dict vs set? **A:** Key-value pairs vs unique elements.
3. **Q:** GIL? **A:** Ek thread at a time executes Python bytecode.
4. **Q:** Generator? **A:** Lazy iterator via yield.
5. **Q:** Decorator? **A:** Function wrapper.
6. **Q:** args? **A:** Positional args tuple.
7. **Q:** kwargs? **A:** Keyword args dict.
8. **Q:** Shallow copy? **A:** Top-level copy, nested shared.
9. **Q:** Deep copy? **A:** Everything copied.
10. **Q:** Type hints runtime? **A:** Ignored, static tools ke liye.
11. **Q:** with statement? **A:** Context manager setup/teardown.
12. **Q:** try/except/else/finally? **A:** Error handling + success + cleanup.
13. **Q:** f-string? **A:** Embedded expressions in strings.
14. **Q:** .get() vs []? **A:** Default vs KeyError.
15. **Q:** slicing [::-1]? **A:** Reverse.
16. **Q:** lambda? **A:** Anonymous one-line function.
17. **Q:** map/filter? **A:** Apply/predicate over iterable.
18. **Q:** global vs local scope? **A:** Module vs function scope.
19. **Q:** isinstance vs type? **A:** Inheritance-aware vs exact.
20. **Q:** __init__? **A:** Constructor.

56.2 LLM & AI (21-45)

1. **Q:** Token? **A:** Text ka processing unit.

2. **Q:** Temperature? **A:** Randomness control.
3. **Q:** Context window? **A:** Max input model dekh sakta hai.
4. **Q:** Hallucination? **A:** Confident galat output.
5. **Q:** System prompt? **A:** Role/behavior instructions.
6. **Q:** Few-shot? **A:** Examples ke saath prompt.
7. **Q:** RAG? **A:** Retrieve then generate.
8. **Q:** Embedding? **A:** Text ka vector.
9. **Q:** Cosine similarity? **A:** Direction similarity 0-1.
10. **Q:** Chunking? **A:** Text ko parts mein katna.
11. **Q:** Chunk overlap? **A:** Context continuity.
12. **Q:** Vector DB? **A:** Similarity search store.
13. **Q:** HNSW? **A:** Approximate nearest neighbor index.
14. **Q:** Attention? **A:** Words ke beech relationship.
15. **Q:** Transformer? **A:** Attention-based architecture.
16. **Q:** Fine-tuning? **A:** Trained model ko update.
17. **Q:** RLHF? **A:** Human feedback se alignment.
18. **Q:** Quantization? **A:** Weight precision kam.
19. **Q:** Prompt injection? **A:** Instructions hijack karna.
20. **Q:** LLM-as-judge? **A:** LLM se outputs score.
21. **Q:** Zero-shot? **A:** Bina example ke.
22. **Q:** Chain-of-thought? **A:** Step-by-step reasoning.
23. **Q:** Multimodal? **A:** Text + image + audio.
24. **Q:** Temperature 0 vs 1? **A:** Deterministic vs creative.
25. **Q:** Structured output? **A:** JSON/schema-based output.

56.3 LangGraph & Agents (46-70)

1. **Q:** StateGraph? **A:** State + nodes + edges ka graph.
2. **Q:** Node? **A:** State leke update dene wala function.
3. **Q:** Edge? **A:** Nodes ke beech connection.
4. **Q:** Conditional edge? **A:** Runtime routing.
5. **Q:** Reducer? **A:** State merge rule.
6. **Q:** add_messages? **A:** Messages append reducer.
7. **Q:** TypedDict? **A:** Typed dict contract.
8. **Q:** invoke()? **A:** Final result synchronously.
9. **Q:** stream()? **A:** Intermediate results yield.

10. **Q:** compile()? **A:** Graph execution plan banana.
11. **Q:** START/END? **A:** Entry/exit pseudo-nodes.
12. **Q:** Loop guard? **A:** MAX_REVISIONS infinite loop rokta.
13. **Q:** Agent? **A:** Role + LLM + tools wala unit.
14. **Q:** Multi-agent? **A:** Specialized agents ka system.
15. **Q:** Handoff? **A:** Ek agent ka output agle ko.
16. **Q:** Shared state? **A:** Blackboard pattern.
17. **Q:** Fan-out? **A:** Parallel node execution.
18. **Q:** Checkpoint? **A:** State persistence.
19. **Q:** Researcher agent? **A:** Search queries + Tavily + RAG.
20. **Q:** Analyst agent? **A:** Themes/contradictions/gaps brief.
21. **Q:** Fact-checker? **A:** Closed-world claim verification.
22. **Q:** Writer agent? **A:** Final report generation.
23. **Q:** Closed-world assumption? **A:** Sirf diye sources se verify.
24. **Q:** Revision loop? **A:** Fact-check fail par re-research.
25. **Q:** CrewAI vs LangGraph? **A:** Roles vs graph control.

56.4 RAG & Vector Search (71-85)

1. **Q:** Ingestion pipeline? **A:** PDF to chunks to vectors.
2. **Q:** Retrieval? **A:** Query embedding + top-k.
3. **Q:** k value? **A:** Kitne chunks return (5/3).
4. **Q:** Collections? **A:** research_docs + past_research.
5. **Q:** Persistence? **A:** Disk par save (Chroma).
6. **Q:** all-MiniLM-L6-v2? **A:** 384-dim embedding model.
7. **Q:** Hybrid search? **A:** Vector + keyword.
8. **Q:** Re-ranking? **A:** Cross-encoder refine.
9. **Q:** Query expansion? **A:** Multiple diverse queries.
10. **Q:** Metadata filter? **A:** Pre-filter by date/type.
11. **Q:** Recall@k? **A:** Relevant docs retrieved ratio.
12. **Q:** Lost in middle? **A:** LLM middle context ignores.
13. **Q:** Embedding dims? **A:** 384 (MiniLM).
14. **Q:** Semantic search? **A:** Meaning-based matching.
15. **Q:** Faiss vs Chroma? **A:** Library vs DB with persistence.

56.5 Django & Web (86-100)

1. **Q:** Django? **A:** Batteries-included web framework.
2. **Q:** DRF? **A:** REST API layer for Django.
3. **Q:** Serializer? **A:** Data conversion + validation.
4. **Q:** Migration? **A:** Schema version control.
5. **Q:** ORM? **A:** Python objects to DB.
6. **Q:** APIView? **A:** Class-based view.
7. **Q:** Token auth? **A:** Token header authentication.
8. **Q:** CORS? **A:** Cross-origin policy.
9. **Q:** 400 vs 401? **A:** Bad request vs unauthorized.
10. **Q:** 201? **A:** Created.
11. **Q:** QuerySet? **A:** Lazy DB query.
12. **Q:** order_by? **A:** Sort results.
13. **Q:** SQLite? **A:** File-based embedded DB.
14. **Q:** PostgreSQL? **A:** Production relational DB.
15. **Q:** Gunicorn? **A:** WSGI server.

57 Part-by-Part Summary — 60 Second Revision

Part	Core takeaway
I (Setup)	Project structure, env setup, teen interfaces ka overview
II (Concepts)	LLM, prompting, RAG, MCP, Django fundamentals
III (Code)	Har file line-by-line: config, rag, 4 agents, workflow, UI, MCP, evaluator, Django
IV (Deep Dives)	LangGraph internals, embeddings math, prompts verbatim, real runs, system design
V (Interviews)	68 Q&A + pitches + whiteboard problems
VI (Reference)	API endpoints, 16 tests, deployment
VII (Playbook)	Extension guide, 60-term glossary, comparisons, cheat sheets
VIII (Theory)	Transformer/attention, advanced RAG, debugging stories
IX (Extended)	30 more Q&A, troubleshooting, 30-min tutorial, file map
X (CS Revision)	DSA patterns, OS/DBMS/CN quick revision
XI (Demo)	Demo scripts, mock interview, case studies
XII (Python)	Python features from project code, full code walkthroughs
XIII (Extra)	20 more Q&A, dependencies, project metrics
XIV (Revision)	100 speed-round Q&A (ye section)

58 The Final Page — Jo Sabse Zaroori Hai

⚙️ System Design Deep Dive

Interview ke din 3 cheezein yaad rakho:

1. **Numbers:** 4 agents, 3 interfaces, 16 tests, 176s, 15 sources, 3 revisions max, 384 dims, GPT-OSS-120B, 6 endpoints.
2. **Stories:** (a) MCP version bug + fix; (b) Benchmark loss + root cause; (c) Model 404 + live verification; (d) venv fallback + documentation.
3. **Honesty:** Limitations bolo (single-run benchmark, closed-world fact-check, sync pipeline) — interviewer ko honest engineer pasand aata hai.

Aur sabse important: Confidence. Tumne ye project banaya hai, tumne har line likhi hai, aur ab tumhare paas 150 pages ka gyaan hai. Interview sirf ek baatachit hai — apni mehnat ka result dikha rahe ho. **Deep breath, smile, and go!**

Bahut saari shubhkamnayein! 🙌

📖 Kahani Se Samjho

Document complete. 150+ pages, 58 sections, 14 parts — Hinglish mein, story style mein, har concept aur har code line ke saath. **Yeh document ab aapka hai** — ise apna banao, samjho, aur interview mein chhapao.

Jai Hind!

Part XII

Exercises & What-If Scenarios

59 app.py + Models — Jo Abhi Tak Full Nahi Dekha

59.1 app.py — Session State Flow

</> Code Samjho Line-by-Line

```

1 # 1. Page config + theme
2 st.set_page_config(page_title="Multi-Agent Research", layout="wide")
3
4 # 2. Session state lazy init
5 if "theme" not in st.session_state:
6     st.session_state.theme = "light"
7 if "messages" not in st.session_state:
8     st.session_state.messages = []
9 if "final_report" not in st.session_state:
10    st.session_state.final_report = ""
11
12 # 3. Sidebar: theme picker + PDF upload
13 theme = st.sidebar.selectbox("Theme", ["light", "dark"],
14                               index=["light", "dark"].index(st.
15                                   session_state.theme))
16 if theme != st.session_state.theme:
17     st.session_state.theme = theme
18     st.rerun() # theme apply karne ke liye rerun
19
20 uploaded = st.sidebar.file_uploader("Upload PDF", type=["pdf"])
21 if uploaded:
22     save + ingest → st.success(f"{n} chunks added")
23
24 # 4. Topic input + Run
25 topic = st.text_input("Research topic")
26 if st.button("Run") and topic:
27     # reset state
28     st.session_state.messages = []
29     # stream pipeline
30     for event in research_graph.stream(initial_state(topic)):
31         for node, output in event.items():
32             st.session_state.agent_statuses[node] = "done"
33             with st.chat_message(node):
34                 st.write(output.get("messages", [""])[-1])
35     st.session_state.final_report = ... # last state se
36
37 # 5. Report render + download
38 if st.session_state.final_report:
39     st.markdown(st.session_state.final_report)
40     st.download_button("Download MD", ..., file_name="report.md")

```

Listing 49: app.py — key flow

Interview points: (1) `st.rerun()` theme change par — Streamlit idiom; (2) `file_uploader`

+ ingest — RAG loop close; (3) chat-message styling se agent progress visual; (4) download_button — result export.

59.2 backend/api/models.py — Full Walkthrough

Code Samjho Line-by-Line

```

1 from django.db import models
2 from django.contrib.auth.models import User
3
4 class ResearchSession(models.Model):
5     topic = models.CharField(max_length=500)
6     report = models.TextField()
7     messages = models.JSONField(default=list)          # agent log
8     user = models.ForeignKey(User, null=True, blank=True,
9                               on_delete=models.SET_NULL)
10    sources_count = models.IntegerField(default=0)
11    revision_count = models.IntegerField(default=0)
12    created_at = models.DateTimeField(auto_now_add=True)
13
14    def __str__(self):
15        return f"Session: {self.topic[:50]}"
16
17 class BenchmarkEvaluation(models.Model):
18     topic = models.CharField(max_length=500)
19     single_agent_report = models.TextField()
20     multi_agent_report = models.TextField()
21     single_agent_depth = models.IntegerField()
22     multi_agent_depth = models.IntegerField()
23     single_agent_verifiability = models.IntegerField()
24     multi_agent_verifiability = models.IntegerField()
25     verdict = models.CharField(max_length=40)
26     created_at = models.DateTimeField(auto_now_add=True)
27
28    def __str__(self):
29        return f"Benchmark: {self.topic[:50]} ({self.verdict})"

```

Listing 50: models.py — dono models

Design decisions:

- JSONField messages — flexible list storage (no separate table needed).
- on_delete=models.SET_NULL — user delete ho to session rahe (data loss nahi).
- null=True, blank=True — anonymous sessions allowed.
- auto_now_add — created_at automatic, immutable.
- IntegerFields for scores — 0-10 scale, DB-level simplicity.

60 Build-It-Yourself — Mini Versions With Solutions

📖 Kahani Se Samjho

Interview practice ke liye: project ke 4 core components ko **zero-dependency** mini versions mein implement karo (30-45 min each). Ye exercises aapki understanding ko prove karte hain — agar mini version bana sakte ho, to bada version samajhte ho.

60.1 Exercise 1: Mini Vector Store

🔗 Code Samjho Line-by-Line

Task: Bina ChromaDB ke, ek class jo documents store kare aur cosine similarity se top-k retrieve kare. **Hint:** numpy dot products. **Solution:**

```

1 import numpy as np
2
3 class MiniVectorStore:
4     def __init__(self):
5         self.docs = []      # list[str]
6         self.vectors = []   # list[np.ndarray]
7
8     def add(self, text: str, vector: np.ndarray):
9         self.docs.append(text)
10        self.vectors.append(vector / np.linalg.norm(vector))
11
12    def search(self, query_vec: np.ndarray, k: int = 3) → list[str]:
13        q = query_vec / np.linalg.norm(query_vec)
14        scores = [float(v @ q) for v in self.vectors] # cosine
15        idx = sorted(range(len(scores)), key=lambda i: -scores[i])[:k]
16        return [self.docs[i] for i in idx]
17
18 # use:
19 # store.add("RAG is retrieval augmented generation", embed("RAG..."))
20 # store.search(embed("what is RAG"), k=2)

```

Listing 51: mini vector store

Key concepts proven: cosine similarity = normalized dot product; top-k sorting; O(n) brute force (HNSW production mein).

60.2 Exercise 2: Mini Agent Loop

🔗 Code Samjho Line-by-Line

Task: Bina LangGraph ke, ek simple loop: research → analyze → verify → revise (max 3). **Solution:**

```

1 def run_pipeline(topic, call_llm, search):
2     state = {"topic": topic, "sources": [], "analysis": "",
3             "passed": False, "revisions": 0}
4     while not state["passed"] and state["revisions"] <= 3:
5         state["sources"] = search(topic) # researcher

```

```

6     state["analysis"] = call_llm("analyze", state["sources"])
7     verdict = call_llm("verify", state["analysis"], state["sources"]
8         [])
9     state["passed"] = "PASSED" if verdict.upper()
10    state["revisions"] += 1
11    return {"report": call_llm("write", state["analysis"], state["sources"]),
12        "revisions": state["revisions"]}

```

Listing 52: mini agent loop

Interview question: “LangGraph isse kya better karta hai?” — streaming, visualization, parallelism, checkpointing, declarative flow.

60.3 Exercise 3: Mini Benchmark Judge

Code Samjho Line-by-Line

Task: Do reports ko score karne wala heuristic judge (bina LLM ke). **Solution:**

```

1 import re
2
3 def judge(report: str) → dict:
4     sections = len(re.findall(r'^#{1,3} ', report, re.M))
5     bullets = len(re.findall(r'^[-*] ', report, re.M))
6     words = len(report.split())
7     citations = len(re.findall(r'\[\d+\]|https?://', report))
8     depth = min(10, sections // 3 + bullets // 5 + words // 300)
9     verifiability = min(10, citations * 2)
10    return {"depth": depth, "verifiability": verifiability}

```

Listing 53: mini heuristic judge

Interview point: Heuristics deterministic hain (tests ke liye), LLM judge quality deta hai — is project ka evaluator dono combine karta hai (LLM first, heuristic fallback).

61 What-If Scenarios — Aage Badhne Ke Raaste

61.1 Scenario 1: Agent Memory Add Karna

Concept Deep Dive

Problem: Agents har run mein past se forget karte hain (sirf rag_context_past_research milta hai).

Solution sketch: (1) **Short-term:** LangGraph checkpointing — har run ki state save, resume; (2) **Long-term:** Agent-specific summaries ChromaDB mein (“what did analyst learn about topic X”), retrieval on next run; (3) **Conversation memory:** add_messages history pipeline ko. Interview answer: “Memory = retrieval over past states + summaries, ek flag ke saath enable/disable.”

61.2 Scenario 2: Planner Agent Add Karna

Concept Deep Dive

Problem: Complex topics (“AI in healthcare AND finance”) ek hi researcher pass mein poori nahi cover hote.

Solution sketch: (1) **Planner node** graph ke START par — topic ko sub-tasks mein toda (“healthcare AI”, “finance AI”, “joint analysis”); (2) Har sub-task ka apna mini-pipeline (parallel fan-out); (3) **Aggregator node** — sub-reports ko final report mein merge. Interview point: “Hierarchical multi-agent — plan → execute → aggregate. LangGraph fan-out/fan-in edges se possible.”

61.3 Scenario 3: Multi-Hop RAG

Concept Deep Dive

Problem: Answer ke liye 2+ documents ka combination chahiye (“Company X ka revenue kaun sa product se aaya” — pehle product list, phir revenue).

Solution sketch: (1) Round 1: retrieve + answer attempt with confidence; (2) Low confidence → extracted entities se follow-up query; (3) Round 2: retrieve + combine. Loop guard max 3. Interview line: “Naive RAG single-hop hai; multi-hop query decomposition se complex reasoning improve hoti hai — same revision-loop pattern se implement hota hai jo fact-checker mein hai.”

61.4 Scenario 4: Real-Time Streaming Reports

Concept Deep Dive

Problem: 176s wait — user ko real-time partial report dikhao.

Solution sketch: (1) Har node ka output stream (already `graph.stream`); (2) UI mein node-by-node sections render (researcher ke sources, analyst brief, etc.); (3) Writer ka **token-level streaming** (Groq streaming) — last section live type hote dikhe. Interview point: “Progressive disclosure — user ko har step ka value dikhta hai, wait boring nahi.”

61.5 Scenario 5: Multi-Language Support

Concept Deep Dive

Problem: Hindi/Hinglish queries par reports English mein.

Solution sketch: (1) **Embedding model:** multilingual (multilingual-e5 / bge-m3) — cross-lingual retrieval; (2) **Language param:** writer prompt mein “report language = Hindi/English”; (3) **Translation layer** optional. Interview point: “Embedding model change = retrieval quality change — benchmark se verify karna zaroori (Retrieval eval), assumptions nahi.”



Memory Trick

Final exercise challenge: In 5 scenarios mein se 2 ko implement karke demo banao (even paper sketch enough) — interview mein “I’ve thought about scaling this” ka proof milega. Koi bhi “what if” question aaye — answer ready hai.

Part XIII

The Final Mile — Guided Reading & Cheat Card

62 Guided Code Reading — Poora Project 30 Min Mein

📖 Kahani Se Samjho

Ye section ek **narrative code reading** hai — main aapko project ke core files ko padh kar sunata hoon, jaise interview demo mein karte. Sequence: `config` → `state` → `rag` → `agents` → `graph` → `interfaces`.

62.1 Step 1: `config/setting.py` — Sab Ka Aadhaar

🔗 Code Samjho Line-by-Line

Padho: `load_dotenv()` → `get_env_var(key, default, required)` → `MODEL_NAME`, `MAX_SEARCH_RESULTS`, `MAX_REVISIONS`, `CHROMA_PERSIST_DIR`, `EMBEDDING_MODEL`, `CHUNK_SIZE`, `CHUNK_OVERLAP`.

Bolo: “Config module ek hi jagah hai — har environment variable ka ek getter with 3-level fallback (OS env, Streamlit secrets, `.env`). Required flags fail-fast karte hain. Isi se model, chunk sizes, aur API keys poori app mein control hote hain. Isliye config change = code change nahi.”

62.2 Step 2: `state/schema.py` — The Contract

🔗 Code Samjho Line-by-Line

Padho: `ResearchState(TypedDict)` — 10 fields.

Bolo: “Ye `TypedDict` poore pipeline ka data contract hai. Har agent ise padhta hai, apne fields return karta hai. `messages` field `Annotated[list, add_messages]` hai — matlab append semantics, replace nahi. Contract clear hai isliye agents independent develop hote hain.”

62.3 Step 3: `rag/vector_store.py` — The Memory

</> Code Samjho Line-by-Line

Padho: `get_vectorstore` → `ingest_documents` → `retrieve_context` → `store_research_session` → `get_collection_stats` → `clear_collection`.

Bolo: “ChromaDB do collections ke saath: `research_docs` (uploaded PDFs) aur `past_research` (sessions ki memory). Ingestion: `PyPDFLoader` → `RecursiveCharacterTextSplitter` (1000/200) → all-MiniLM-L6-v2 embeddings → Chroma. Retrieval: cosine similarity, `k=5` docs + `k=3` past. Har function try/except safe hai — vector store down ho to pipeline chalti rehti hai.”

62.4 Step 4: The Four Agents

</> Code Samjho Line-by-Line

Padho: Har agent ka `get_llm` + node function + `SYSTEM_PROMPT`.

Bolo: “Har agent ka pattern same hai: (1) `get_llm()` — ChatGroq with `MODEL_NAME` + apna temperature; (2) node function — state se input le, LLM invoke, fallback ready; (3) return partial state update. Temperatures: fact-checker 0.1, analyst 0.2, researcher 0.3, writer 0.4 — task ke nature se. Researcher Tavily search + RAG retrieval combine karta hai; analyst top-10 se dense brief; fact-checker closed-world verify + `VERDICT`; writer final report + memory store.”

62.5 Step 5: `graph/workflow.py` — The Engine

</> Code Samjho Line-by-Line

Padho: `build_graph()` → nodes → edges → conditional → compile → `research_graph = build_graph()`.

Bolo: “StateGraph: 4 nodes, linear flow researcher → analyst → fact_checker, phir conditional edge — fact-check passed ho to writer, nahi to researcher (max 3 revisions). `should_continue` traffic cop hai. Module-level singleton `research_graph` — teeno interfaces isi ek compiled graph ko call karte hain, code duplication zero.”

62.6 Step 6: The Three Interfaces

</> Code Samjho Line-by-Line

Padho: `app.py` (stream), `mcp_server.py` (invoke + tools), `backend/api/views.py` (invoke + persist).

Bolo: “Teen consumption patterns, ek core: Streamlit real-time streaming dikhata hai; MCP tools AI assistants ko deta hai; Django API programmatic access + persistence deta hai. Har interface sirf `research_graph` ko call karta hai — pipeline internals se decoupled. Ye adapter pattern ka real example hai.”

63 Quick-Fire Round 2 — 20 Most Likely Questions

Interview Question

1. **“Ek line mein project?”** “Multi-agent research system: web search, analysis, fact-check, revision loop, report — teen interfaces ke saath.”

Interview Question

2. **“Sabse acha decision?”** “Fact-checker + revision loop — quality control architecture mein built-in hai.”

Interview Question

3. **“Sabse bura decision?”** “Synchronous pipeline — 176s block. Production mein async hota.”

Interview Question

4. **“Bina kya kaam nahi karta?”** “API keys (Groq + Tavily) — fallback paths hain par quality gir jaati hai.”

Interview Question

5. **“Kya seekha?”** “Evaluation-first sochna — benchmark ne assumptions todi (single-agent jeet gaya ek run mein).”

Interview Question

6. **“LLM model kya? Kyun?”** “GPT-OSS-120B via Groq — live test mein clean output, fast, free tier.”

Interview Question

7. **“Embedding kya?”** “all-MiniLM-L6-v2 — 384 dims, CPU fast, local free.”

Interview Question

8. **“Vector store kya?”** “ChromaDB — 2 collections, cosine similarity.”

Interview Question

9. **“Tests kitne?”** “16 pytest — auth 6, research 4, benchmark 6.”

Interview Question

10. **“Endpoints kitne?”** “6 — health, register, login, execute, benchmark, history.”

 Interview Question

11. **“Kyun Django?”** “ORM + migrations + auth built-in — rapid + structured.”

 Interview Question

12. **“Kyun Streamlit?”** “Real-time streaming + zero frontend + Python.”

 Interview Question

13. **“Kyun MCP?”** “AI assistants ko tools dene ka open standard — ek baar server, sab clients.”

 Interview Question

14. **“Kyun LangGraph?”** “Conditional loop + streaming + graph thinking.”

 Interview Question

15. **“Chunk size kyun 1000?”** “Balance — precise retrieval + pura context; 200 overlap continuity.”

 Interview Question

16. **“Max revisions kyun 3?”** “Infinite loop guard + cost control.”

 Interview Question

17. **“Benchmark kya dikhaya?”** “Ek run mein single-agent superior — handoff quality matter karti hai.”

 Interview Question

18. **“Security?”** “Secrets gitignored + 3-level fallback; CORS allowlist; prompt injection aware.”

 Interview Question

19. **“Scale kaise?”** “Async pipeline, PostgreSQL, managed vector DB, multi-instance Streamlit.”

 Interview Question

20. **“Kyun hire karna chahiye?”** “Main end-to-end systems banata hoon, measure karta hoon, aur problems se nahi bhagta — inhe solve karta hoon.”

64 Project Pitfalls — Jo Galtiyan Humne Ki

Pitfall

fastmcp version mismatch

Lesson

Dependency upgrades ke baad smoke test hamesha

Model name documented par unavailable	Live API se verify karo, docs se nahi
Windows venv activate silent fail	Cross-platform run docs + exact interpreter path
Port conflicts	Configurable ports + health checks
Time-frozen tests / ordering ties	Deterministic ordering (secondary key)
LaTeX underscore captions	Escaping rules environment-specific
Emoji print crash (Windows console)	PYTHONIOENCODING=utf-8
Real API calls in tests	Mock everything external
Single benchmark run verdict	Statistical significance zaroori
Secrets in code/docs	gitignore + .env.example + scanning

65 Regeneration Guide + One-Page Cheat Card

65.1 Is Document Ko Kaise Regenerate Karein

Code Samjho Line-by-Line

```
# 1. Edit karo: main file + notes_parts/*.tex
# 2. Compile (MiKTeX/TeX Live)
pdflatex -interaction=nonstopmode Multi_Agent_Master_Notes.tex
pdflatex -interaction=nonstopmode Multi_Agent_Master_Notes.tex # 2nd
pass: TOC/refs

# 3. Structure
# Multi_Agent_Master_Notes.tex <- preamble + parts via \input{}
# notes_parts/part*.tex <- content (16 parts)
```

Tips: (1) Naya part: notes_parts/ mein file banao + {notes_parts/name} main file mein; (2) Tables: longtable rows 2 backslashes row end; (3) Underscores text mein _ escape; (4) Compile errors: log mein 1.<line> dekho.

65.2 One-Page Cheat Card (Interview Se Pehle)

System Design Deep Dive

THE NUMBERS: 4 agents | 3 interfaces | 16 tests | 6 endpoints | 2 collections | 384 dims | 1000/200 chunks | 3 max revisions | 176s run | 15 sources | GPT-OSS-120B.

THE FLOW: topic → researcher (3 queries + Tavily + RAG) → analyst (themes/contradictions/gaps) → fact-checker (VERDICT, closed-world) → [loop max 3] → writer (report + memory).

THE ARCHITECTURE: Ek compiled LangGraph singleton, 3 adapters (Streamlit stream / MCP tools / Django REST) + ChromaDB memory + config central.

THE STORIES: MCP version bug | Model 404 + live verify | Benchmark loss + root cause | venv silent fail | LaTeX underscore saga.

THE HONESTY: Limitations — sync pipeline, single-run benchmark, closed-world fact-check, naive RAG (no re-ranking).

THE CLOSER: “Maine system banaya, fail hote dekha, root cause nikala, fix propose kiya

— aur har cheez measure kiya. Ye hi engineering hai.”

Kahani Se Samjho

Document samapt. 16 parts, 66 sections, 150+ pages — Hinglish mein, story style mein, project ka har logic, har code line, aur interview ke liye har sawaal ka jawab. **Ab jao aur jeeto.** 🏆

Part XIV

Backend Files + Daily Revision Plan

66 Remaining Backend Files — Serializers, URLs, Conftest

66.1 serializers.py — Full Walkthrough

Code Samjho Line-by-Line

```
1 from rest_framework import serializers
2 from .models import ResearchSession, BenchmarkEvaluation
3
4 class ResearchSessionSerializer(serializers.ModelSerializer):
5     class Meta:
6         model = ResearchSession
7         fields = ["id", "topic", "report", "messages",
8                 "sources_count", "revision_count", "created_at"]
9
10 class BenchmarkEvaluationSerializer(serializers.ModelSerializer):
11     class Meta:
12         model = BenchmarkEvaluation
13         fields = ["id", "topic", "single_agent_report",
14                 "multi_agent_report", "single_agent_depth",
15                 "multi_agent_depth", "single_agent_verifiability",
16                 "multi_agent_verifiability", "verdict", "created_at"]
```

Listing 54: serializers.py — dono serializers

Design: (1) `ModelSerializer` — model fields se auto field mapping + validation; (2) Explicit fields list — kya expose karna hai control (user field exclude); (3) `user` intentionally serializer mein nahi — request se attach hota hai, client se nahi. Interview point: “Serializer = API ka contract — kya dikhega, kya validation, kya chhupa hoga, sab yahan control hota hai.”

66.2 urls.py — Full Walkthrough

Code Samjho Line-by-Line

```
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     path("health/", views.HealthCheckView.as_view()),
6     path("auth/register/", views.RegisterView.as_view()),
7     path("auth/login/", views.LoginView.as_view()),
8     path("research/execute/", views.ExecuteResearchAPIView.as_view()),
9     path("research/benchmark/", views.BenchmarkAPIView.as_view()),
10    path("benchmark/history/", views.BenchmarkHistoryAPIView.as_view())
```

11

Listing 55: api/urls.py

Routing structure: (1) `health/` top-level; (2) `auth/` namespace — `register/login`; (3) `research/` namespace — `execute/benchmark`; (4) `benchmark/history/` read-only. Ye REST-consistent nested design hai. Main `urls.py` mein `path("api/", include("api.urls"))` — prefix ek jagah.

66.3 conftest.py — Mocking Infrastructure

Code Samjho Line-by-Line

```

1 import pytest
2 from unittest.mock import patch, MagicMock
3
4 @pytest.fixture
5 def mock_research_graph():
6     mock = MagicMock()
7     mock.invoke.return_value = {
8         "final_report": "# Report\n\nTest content",
9         "messages": ["Agent ran"],
10    }
11    with patch("graph.workflow.research_graph", mock):
12        yield mock
13
14 @pytest.fixture
15 def exploding_research_graph():
16     mock = MagicMock()
17     mock.invoke.side_effect = RuntimeError("pipeline crashed")
18     with patch("graph.workflow.research_graph", mock):
19         yield mock
20
21 @pytest.fixture
22 def mock_benchmark():
23     with patch("benchmark.evaluator.run_single_agent_baseline",
24               return_value="# Baseline report"),
25         patch("benchmark.evaluator.evaluate_outputs",
26               return_value={"single_agent_depth": 9, ...}):
27         yield

```

Listing 56: conftest.py — fixtures

Fixture design: (1) `MagicMock` — auto attributes/methods; (2) `side_effect` — exception simulation (error paths); (3) `return_value` — success paths; (4) `patch` context manager — test ke baad automatic restore. Interview point: “Fixtures = test setup ka reusable building block. Mocking se tests fast (9s), free, aur deterministic hote hain — real API calls sirf manual E2E mein.”

66.4 Settings/wsgi/asgi/manage.py — Quick Tour

 Code Samjho Line-by-Line

Listing 57: Django entry files

```
# manage.py: CLI entry (runserver, migrate, shell)
python manage.py runserver 8001

# settings.py: INSTALLED_APPS (rest_framework, corsheaders, api),
# DATABASES (sqlite3), CORS_ALLOWED_ORIGINS, rest_framework auth
  classes

# wsgi.py: sync WSGI app (Gunicorn production ke liye)
# asgi.py: async ASGI app (uvicorn / real-time ke liye)
```

Interview points: (1) WSGI vs ASGI — sync vs async server interface; (2) INSTALLED_APPS = Django ka plugin system; (3) CORS config — Streamlit origin allow; (4) REST framework settings — DefaultAuthentication (token/session).

67 Daily Revision Plan — Interview Tak

Day	Revision focus
Day 1	Parts I-III — setup, concepts, code walkthrough (2x speed read)
Day 2	Part IV — deep dives + prompts verbatim + system design
Day 3	Part V — 68 Q&A (bol ke practice) + pitches + whiteboard
Day 4	Parts VI-VII — API reference, tests, deployment, glossary
Day 5	Parts VIII-IX — transformer theory, debugging stories, 30 more Q&A
Day 6	Parts X-XI — DSA patterns, core CS, demo scripts, mock interview
Day 7	Parts XII-XVI — Python deep dive, quick-fire, cheat card, full mock
Daily	30-min tutorial script 1x + 10 numbers + 4 stories

 Memory Trick**Interview day checklist:**

1. Numbers yaad karo (cheat card section 66).
2. 30-sec pitch mirror ke saamne 2 baar.
3. Ek mock interview kisi dost ke saath.
4. Laptop ready: code + demo + tests chalne ready.
5. Setup: local server ports free, API keys loaded, internet on.
6. Deep breath. **Tum taiyaar ho.**

 Kahani Se Samjho

17 parts, 68 sections, 150+ pages. Har logic explain kiya, har code line samjhaya, har interview sawaal ka jawab diya — sab Hinglish mein, story style mein. Is document ne aapko **gyaan** diya hai; ab **confident** bolo. Interview sirf ek conversation hai jisme aap apna kaam

dikha rahe ho. **All the best!** 🎓

Part XV

Self-Assessment — Khud Ko Test Karo

68 30-Question Quiz — Bina Dekhe Jawab Do

 Concept Deep Dive

Rules: (1) Paper par jawab likho (2-3 lines each); (2) Answer section 69.2 se check karo; (3) 20+ sahi = interview ready; 15-20 = ek aur revision; <15 = parts dobara.

68.1 Questions (1-30)

1. ResearchState mein `messages` field par kya reducer lagta hai aur kyun?
2. Researcher 3 sub-queries kaise banata hai? Query generator fail ho to kya hota hai?
3. Analyst apne input mein kitne sources limit karta hai? Kyun?
4. Fact-checker ki “closed-world assumption” kya hai?
5. `should_continue` function kis condition par “end” return karta hai?
6. `MAX_REVISIONS` default kya hai? Ye kya prevent karta hai?
7. Temperature har agent mein kitna hai? Kyun alag-alag?
8. chromaDB ke 2 collections ke naam aur unke uses?
9. Chunk size/overlap kya hai? Overlap kyun zaroori?
10. Embedding model ka naam aur dimensions?
11. Cosine similarity ka formula batao (words mein).
12. `app.py` mein `session_state` kyun chahiye? Lazy init pattern kya hai?
13. `graph.stream()` vs `graph.invoke()` mein difference?
14. MCP ke 3 core primitives kya hain?
15. MCP server mein `fastmcp` ka kaunsa parameter fix kiya tha aur kyun?
16. evaluator mein LLM judge fail ho to kya hota hai?
17. Benchmark ke 2 dimensions kya hain?
18. Benchmark verdict ke 3 possible values?
19. Django mein migration kya hai? Is project mein kaunsi migration hai?
20. API ke 6 endpoints likho (method + path).
21. Register karne par password kaise store hota hai?
22. Test mein real API calls kyun nahi hote? Kaise mock hote hain?
23. `pytest-django` test ke liye kaunsi database use karta hai?
24. History endpoint `order_by` mein secondary key kya hai aur kyun?

25. Bina LangGraph ke pipeline kaise ban sakti thi? 2 problems batao.
26. Prompt injection ka is project mein kya risk hai?
27. Is project ki 3 interfaces kya hain aur kaunsi consumption pattern?
28. Pipeline latency kyun 176s hai? Kaunse 2 fixes sabse effective honge?
29. RAG ki 3 problems aur is project ke 2 mitigations?
30. Is project ka “moat” kya hai (product thinking)?

68.2 Answers (1-30)

1. `Annotated[list, add_messages]` — messages append semantics ke saath merge hote hain, replace nahi.
2. `LLM_QUERY_GEN_PROMPT` se 3 diverse queries; fail ho to fallback: topic + “latest research” + “analysis”.
3. `research_data[:10] + rag_context[:5]` — context window control + token cost.
4. Claims sirf diye gaye source data se verify hote hain — model ka training knowledge count nahi hota.
5. `fact_check_passed` true, ya `revision_count` $\geq$ `MAX_REVISIONS` — dono par “end”.
6. 3 — infinite loop prevent + cost control.
7. fact-checker 0.1, analyst 0.2, researcher 0.3, writer 0.4 — task ke nature se (factual vs creative).
8. `research_docs` (uploaded PDFs), `past_research` (sessions ki memory).
9. 1000 chars / 200 overlap — context continuity, sentence na kate.
10. all-MiniLM-L6-v2, 384 dimensions.
11. normalized dot product = cosine similarity — direction similarity.
12. Har rerun par state reset hota; `session_state` persist karta hai; `if key not in st.session_state` pattern.
13. stream: har node ka event real-time; invoke: final state ek baar mein.
14. Tools (capabilities), Resources (readable data), Prompts (templates).
15. `description=` → `instructions=` — fastmcp 3.x rename.
16. Heuristic scoring fallback — regex se sections/citations count, deterministic.
17. Depth + Verifiability (0-10 each).
18. `SINGLE_AGENT_SUPERIOR`, `MULTI_AGENT_SUPERIOR`, `TIE`.
19. Schema version control; `0001_initial` (ResearchSession + BenchmarkEvaluation tables).
20. GET health/, POST auth/register/, POST auth/login/, POST research/execute/, POST research/benchmark/, GET benchmark/history/.
21. `set_password` — hashed (plain text kabhi nahi).
22. Mocked (MagicMock + patch) — fast, free, deterministic; real calls sirf manual E2E.
23. Throwaway test database (real db.sqlite3 untouched).
24. `-created_at`, `-id` — same-timestamp rows deterministic.

25. Manual loop: flow logic khoya, streaming/checkpointing khud banana, testing hard.
26. Untrusted sources (Tavily/PDFs) model instructions ko hijack kar sakte hain — data instructions se alag + fact-checker closed-world.
27. Streamlit (human UI), MCP (AI assistants), Django REST (programmatic) — ek compiled graph, 3 adapters.
28. 3 Tavily calls + 4 LLM calls + embeddings sequential. Fixes: parallel sub-queries + async pipeline.
29. Problems: retrieval quality, lost-in-middle, no evaluation. Mitigations: multi-source retrieval + analyst compression + fact-checker.
30. Workflow design (fact-check loop), evaluation data accumulation, 3-interface integration.

Kahani Se Samjho

Result: 20+ sahi — **taiyaar ho!** Ab confidence ke saath interview do. Aur koi doubt ho to is document ka koi bhi part kholo — sab kuch yahan hai.

Ye document aapka hai. Isse apna banao. Aur jeet jao. 🏆